SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE
FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Evidenčné číslo: FEI-16607-115092

REDUKCIA PARSOVACIEHO PROBLÉMU PRE BEZKONTEXTOVÉ GRAMATIKY NA SAT PROBLÉM

Diplomová práca

2026 **Bc. Jozef Nyitrai**

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE
FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Evidenčné číslo: FEI-16607-115092

REDUKCIA PARSOVACIEHO PROBLÉMU PRE BEZKONTEXTOVÉ GRAMATIKY NA SAT PROBLÉM

Diplomová práca

Študijný program:	aplikovaná informatika
Študijný odbor:	informatika
Školiace pracovisko:	Ústav informatiky a matematiky
Vedúci záverečnej práce:	Ing. Viliam Hromada, PhD.
Konzultant:	Ing. Viliam Hromada, PhD.

2026 **Bc. Jozef Nyitrai**

Slovenská technická univerzita v Bratislave
Ústav informatiky a matematiky

Fakulta elektrotechniky a informatiky
2025/2026

ZADANIE DIPLOMOVEJ PRÁCE

Autor práce:	Bc. Jozef Nyitrai
Študijný program:	aplikovaná informatika
Študijný odbor:	informatika
Evidenčné číslo:	FEI-16607-115092
ID študenta:	115092
Vedúci práce:	Ing. Viliam Hromada, PhD.
Vedúci pracoviska:	doc. Ing. Milan Vojvoda, PhD.
Názov práce:	**Redukcia parsovacieho problému pre bezkontextové gramatiky na SAT problém**
Jazyk, v ktorom sa práca vypracuje:	slovenský jazyk

Špecifikácia zadania:

Cieľom práce je preskúmať možnosť riešenia parsovacieho problému pre bezkontextové gramatiky redukciou na SAT-problém formou naštudovania a implementácie existujúceho prístupu, prípadne implementáciou vlastnej metódy.

Úlohy:
1. Naštudujte si parsovací problém pre bezkontextové gramatiky.
2. Naštudujte si podľa dostupnej literatúry aspoň jednu metódu redukcie parsovacieho problému pre bezkontextové gramatiky na SAT-problém.
3. Vybranú metódu implementujte.
4. Experimentálne vyhodnoťte efektívnosť implementácie a prístupu riešenia parsovacieho problému pre bezkontextové gramatiky redukciou na SAT-problém.

Literatúra:

1. AXELSSON, Roland; HELJANKO, Keijo; LANGE, Martin. Analyzing context-free grammars using an incremental SAT solver. In: Automata, Languages and Programming: 35th International Colloquium, ICALP 2008, Reykjavik, Iceland, July 7-11, 2008, Proceedings, Part II 35. Springer Berlin Heidelberg, 2008. p. 410-422.
2. IMADA, Keita; NAKAMURA, Katsuhiko. Learning context free grammars by using SAT solvers. In: 2009 International Conference on Machine Learning and Applications. IEEE, 2009. p. 267-272.

Termín odovzdania práce:	15. 05. 2026
Dátum schválenia zadania práce:	**12. 03. 2026**
Zadanie práce schválil:	**prof. Dr. rer. nat. Martin Drozda** garant študijného programu

Poďakovanie

Úprimne ďakujem svojmu školiteľovi, Ing. Viliam Hromada, PhD., za odborné vedenie, cenné rady a podporu pri vypracovaní tejto práce. Ďakujem tiež svojej priateľke za podporu a pochopenie, ako aj svojej rodine za trpezlivosť a podporu počas celého štúdia.

Abstrakt

Práca sa zaoberá riešením parsovacieho problému pre bezkontextové gramatiky prostredníctvom redukcie na SAT-problém. Najprv analyzujeme teoretické základy formálnych jazykov, parsovania a Booleovskej splniteľnosti. Následne implementujeme prístup založený na práci Axelsson et al. [1], ktorý transformuje bezkontextovú gramatiku v Chomského normálnej forme na konjunktívnu normálnu formu (CNF) vhodnú pre SAT riešič. Navrhnuté riešenie umožňuje riešiť rozhodovací aj konštrukčný variant parsovania vrátane rekonštrukcie derivačného stromu. Efektívnosť prístupu experimentálne hodnotíme pomocou benchmarkingu viacerých SAT riešičov a porovnávame ju s klasickým CYK algoritmom.

Kľúčové slová

bezkontextové gramatiky, parsovanie, SAT-problém, redukcia, CYK algoritmus, CNF

Abstract

This thesis addresses the parsing problem for context-free grammars by reducing it to the SAT problem. First, we analyze the theoretical foundations of formal languages, parsing, and Boolean satisfiability. We then implement an approach based on the work of Axelsson et al. [1], which transforms a context-free grammar in Chomsky normal form into conjunctive normal form (CNF) suitable for a SAT solver. The proposed solution supports both decision and constructive variants of parsing, including derivation tree reconstruction. The efficiency of the approach is evaluated experimentally using benchmarking of multiple SAT solvers and compared with the classical CYK algorithm.

Keywords

context-free grammars, parsing, SAT problem, reduction, CYK algorithm, CNF

Obsah

Zoznam obrázkov 9
Zoznam značiek a skratiek 11

Úvod **12**

1 Základy formálnych jazykov a gramatík **14**
1.1 Pojem formálneho jazyka 14
1.2 Formálna gramatika 14
1.3 Chomského hierarchia gramatík 15
1.4 Význam bezkontextových gramatík 16
1.5 Bezkontextové gramatiky a ich vlastnosti 17
1.5.2 Chomského normálna forma 19
1.5.3 Význam Chomského normálnej formy pri parsovaní a redukciách 19

2 Parsovací problém **20**
2.1 Definícia parsovania 20
2.2 Rozhodovací a konštrukčný problém 20
2.2.1 Rozhodovací problém 20
2.2.2 Konštrukčný problém 21
2.3 Klasické parsovacie algoritmy 21
2.3.1 CYK algoritmus 21
2.3.2 Časová zložitosť parsovania 21
2.4 Deterministické parsovacie algoritmy (LL a LR) 22
2.5 Porovnanie parsovacích prístupov 22

3 SAT-problém **24**
3.1 Booleovská logika 24
3.2 Literál, klauzula a CNF formula 24
3.3 Definícia rozhodovacieho SAT-problému 25
3.4 Význam SAT-problému 25
3.5 Moderné SAT riešiče 25
3.6 Oblasti použitia SAT-problému 26

4 Pojem redukcie a jej význam **27**
4.1 Definícia redukcie problémov 27

4.2 Redukcie medzi rozhodovacími problémami ... 27
4.3 Dôvody redukcie na SAT ... 27
4.4 Parsovací problém pomocou SAT ... 28
4.4.1 Parsovanie ako množina vzťahov ... 28
4.4.2 Vyjadrenie pomocou logických obmedzení ... 29
4.4.3 Výhody SAT prístupu ... 29
4.4.4 Obmedzenia SAT prístupu ... 29

5 Prehľad súvisiacich prác 30

5.1 Analyzing Context-Free Grammars Using an Incremental SAT Solver ... 30
5.2 Learning Context-Free Grammars by Using SAT Solvers ... 30
5.3 Porovnanie prístupov ... 31

6 Návrh vlastného algoritmu 32

6.1 Teoretické východiská navrhovaného algoritmu ... 32
6.2 Reprezentácia parsovania pomocou booleovských premenných ... 33
6.2.1 Základné premenné modelu ... 34
6.2.2 Inicializované pravdivostné hodnoty ... 34
6.2.3 Pomocné premenné pre binárne pravidlá ... 35
6.2.4 Intuitívny význam reprezentácie ... 36
6.3 Konštrukcia klauzúl ... 36
6.3.1 Klauzuly pre fixné pravdivostné hodnoty ... 37
6.3.2 Klauzuly pre binárne pravidlá ... 38
6.3.3 Väzba medzi premennými $\boldsymbol{X}$ a $\boldsymbol{Z}$ ... 38
6.3.4 Charakter výslednej formule ... 38
6.3.5 Prevod do DIMACS formátu ... 39
6.4 Korektnosť navrhovaného algoritmu ... 39
6.4.1 Soundness ... 39
6.4.2 Completeness ... 40
6.5 Časová a priestorová zložitosť navrhovaného algoritmu ... 41
6.5.1 Počet premenných ... 42
6.5.2 Počet klauzúl ... 42
6.5.3 Celková zložitosť ... 43
6.5.4 Interpretácia výsledkov ... 44
6.5.5 Diskusia ... 44

7 Návrh softvéru 45

7.1 Funkcionálne požiadavky na systém ... 45

7.2 Nefunkcionálne požiadavky na systém 46
7.3 Diagram použitia 46
7.4 Diagram tried 48
7.5 Vývojové diagramy systému 50

8 Implementácia 53

8.1 Interné objekty gramatiky 53
8.2 Spracovanie vstupnej gramatiky 55
8.3 Generovanie SAT modelu 66
8.4 Benchmarking 76
8.4.1 Porovnanie SAT riešičov 76
8.4.2 Porovnanie SAT prístupu a algoritmu CYK 88

9 Experimentálne vyhodnotenie 95

9.1 Metodológia experimentov 95
9.2 Analýza korelácií medzi parametrami modelu 97
9.2.1 Vzťah medzi veľkosťou formule a časom riešenia 97
9.2.2 Vzťah medzi premennými a klauzulami 100
9.2.3 Interpretácia výsledkov 106
9.3 Porovnanie SAT riešičov 106
9.4 Porovnanie SAT prístupu a algoritmu CYK 109

Záver 113

Literatúra 114

Prílohy I

Príloha A: Zdrojový kód algoritmov II

Zoznam obrázkov

Obrázok 1-1 Nejednoznačnosť bezkontextových gramatík 18

Obrázok 7-1 Use-case diagram navrhovaného systému. 47

Obrázok 7-2 Triedny diagram hlavných komponentov systému. 48

Obrázok 7-3 Hlavný vývojový diagram systému. 50

Obrázok 7-4 Detailný vývojový diagram spracovania jedného prípadu. 51

Obrázok 9-1 Korelácie medzi parametrami modelu 97

Obrázok 9-2 Závislosť času riešenia od počtu klauzúl pre gramatiku big_grammar pri jednotlivých SAT riešičoch. 98

Obrázok 9-3 Závislosť času riešenia od počtu klauzúl pre ostatné testované gramatiky pri jednotlivých SAT riešičoch. 98

Obrázok 9-4 Závislosť času riešenia od počtu booleovských premenných pre gramatiku big_grammar pri jednotlivých SAT riešičoch. 99

Obrázok 9-5 Závislosť času riešenia od počtu booleovských premenných pre ostatné testované gramatiky pri jednotlivých SAT riešičoch. 99

Obrázok 9-6 Závislosť času riešenia od dĺžky vstupného slova pre gramatiku big_grammar pri jednotlivých SAT riešičoch. 101

Obrázok 9-7 Závislosť času riešenia od dĺžky vstupného slova pre ostatné testované gramatiky pri jednotlivých SAT riešičoch. 101

Obrázok 9-8 Súhrnné porovnanie závislosti času riešenia od dĺžky vstupného slova pre gramatiku big_grammar a ostatné testované gramatiky. 102

Obrázok 9-9 Závislosť času riešenia od dĺžky vstupného slova pri spoločnom zobrazení všetkých gramatík. 103

Obrázok 9-10 Porovnanie nárastu času riešenia podľa jednotlivých gramatík. 104

Obrázok 9-11 : Závislosť času riešenia od dĺžky vstupného slova zobrazená samostatne pre jednotlivé gra-matiky. 105

Obrázok 9-12 Závislosť mediánového času riešenia od dĺžky vstupného reťazca pre jednotlivé SAT riešiče 107

Obrázok 9-13 Porovnanie mediánového času riešenia jednotlivých SAT riešičov naprieč všetkými testovacími prípadmi 108

Obrázok 9-14 Porovnanie mediánu času riešenia algoritmu CYK a SAT prístupu v závislosti od dĺžky vstupného slova pre gramatiku big_grammar. 110

Obrázok 9-15 Rozklad času SAT prístupu na čas konštrukcie formule a čas samotného riešenia v porovnaní s algoritmom CYK. 111

Zoznam značiek a skratiek

CFG - Bezkontextová gramatika (Context-Free Grammar)

SAT - Problém splniteľnosti Booleovskej formule (Boolean Satisfiability Problem)

CNF - Konjunktívna normálna forma (Conjunctive Normal Form)

CHNF - Chomského normálna forma (Chomsky Normal Form)

CYK - Cocke–Younger–Kasami algoritmus

CDCL - Conflict-Driven Clause Learning

DPLL - Davis–Putnam–Logemann–Loveland

LHS – Left Hand Side

RHS – Right Hand Side

N **-** Množina neterminálov

Σ **-** Množina terminálov

P **-** Množina pravidiel

S **-** Počiatočný symbol gramatiky

Úvod

Parsovanie patrí medzi základné problémy teoretickej informatiky a zohráva kľúčovú úlohu v mnohých oblastiach, ako sú spracovanie programovacích jazykov, kompilátory, analýza prirodzeného jazyka či spracovanie štruktúrovaných dát. Úlohou parsovania je určiť, či daný vstupný reťazec patrí do jazyka definovaného gramatikou, a prípadne nájsť jeho derivačnú štruktúru. V prípade bezkontextových gramatík ide o dobre preskúmaný problém, pre ktorý existujú efektívne klasické algoritmy, ako napríklad CYK algoritmus s kubickou časovou zložitosťou.

V posledných desaťročiach sa však do popredia dostávajú aj alternatívne prístupy založené na redukcii problémov na iné formálne modely. Jedným z najvýznamnejších je redukcia na problém splniteľnosti Booleovskej formule (SAT). SAT problém predstavuje jeden zo základných problémov teoretickej informatiky a zároveň prvý dokázaný NP-úplný problém. Vďaka intenzívnemu výskumu a vývoju moderných SAT riešičov, ktoré využívajú pokročilé heuristiky a optimalizačné techniky, sa SAT stal praktickým nástrojom na riešenie širokej škály problémov z rôznych oblastí, ako sú verifikácia hardvéru, plánovanie, bioinformatika či umelá inteligencia.

Myšlienka redukcie parsovacieho problému na SAT problém spočíva v tom, že proces derivácie reťazca možno reprezentovať pomocou logických premenných a obmedzení. Každé možné použitie pravidla a každá možná segmentácia vstupného reťazca môže byť zakódovaná ako booleovská premenná, pričom vzťahy medzi nimi sú vyjadrené pomocou klauzúl v konjunktívnej normálnej forme. Takto vzniknutý model je následne možné riešiť pomocou všeobecného SAT riešiča.

Cieľom tejto práce je preskúmať možnosť riešenia parsovacieho problému pre bezkontextové gramatiky prostredníctvom redukcie na SAT problém. V rámci práce sa najprv oboznámime s teoretickými základmi formálnych jazykov a gramatík, parsovania a SAT problému. Následne sa zameriame na existujúce prístupy k redukcii parsovania na SAT, najmä na metódu navrhnutú v práci Axelsson et al. [1], ktorá využíva systematickú konštrukciu booleovského modelu z bezkontextovej gramatiky.

Na základe týchto poznatkov implementujeme vlastné riešenie, ktoré transformuje bezkontextovú gramatiku v Chomského normálnej forme (CHNF) na konjunktívnu normálnu formu (CNF) vhodnú pre SAT riešič. Navrhnutý prístup umožňuje riešiť nielen rozhodovací problém, teda určiť, či reťazec patrí do jazyka, ale aj konštrukčný problém v podobe rekonštrukcie derivačného stromu.

Súčasťou práce je aj experimentálne vyhodnotenie implementovaného riešenia. V rámci experimentov porovnávame výkonnosť viacerých moderných SAT riešičov na formulách

generovaných z rôznych gramatík a vstupných reťazcov. Okrem toho porovnávame navrhovaný SAT prístup s klasickým CYK algoritmom, čo nám umožňuje posúdiť jeho praktickú efektivitu a identifikovať jeho silné a slabé stránky.

1 Základy formálnych jazykov a gramatík

Teória formálnych jazykov a gramatík patrí medzi základné oblasti teoretickej informatiky a tvorí nevyhnutný základ pre štúdium výpočtových modelov, syntaktickej analýzy a formálneho popisu jazykových štruktúr. V tejto kapitole zavádzame pojem formálneho jazyka, definujeme formálnu gramatiku, predstavujeme Chomského hierarchiu gramatík a podrobne diskutujeme význam bezkontextových gramatík v súčasných informatických aplikáciách.

1.1 Pojem formálneho jazyka

Formálny jazyk definujeme ako množinu konečných reťazcov nad konečnou abecedou symbolov. Nech Σ označuje konečnú množinu symbolov (abecedu), potom Σ^* predstavuje množinu všetkých konečných reťazcov nad touto abecedou a formálny jazyk L je ľubovoľná podmnožina množiny Σ^* [2]. Táto definícia umožňuje opisovať jazyky čisto z hľadiska ich syntaktickej štruktúry, bez ohľadu na význam jednotlivých reťazcov.

Formálne jazyky sa využívajú ako abstraktný model v informatike, predovšetkým pri analýze výpočtových problémov a vlastností algoritmov. Ako uvádza Sipser [3], abstrakcia od sémantiky umožňuje presne skúmať otázky rozhodnuteľnosti a výpočtovej zložitosti. Formálny jazyk môže byť konečný alebo nekonečný a je možné na ňom definovať operácie ako zjednotenie, prienik, zreťazenie alebo iterácia (Kleeneho uzáver), ktoré zohrávajú významnú úlohu pri konštrukcii zložitejších jazykov [2].

1.2 Formálna gramatika

Na opis a generovanie formálnych jazykov používame formálne gramatiky. Formálnu gramatiku definujeme ako usporiadanú štvoricu

$$G = \langle N, \Sigma, P, S \rangle$$

kde N je konečná množina neterminálnych symbolov, Σ je konečná množina terminálnych symbolov, P je konečná množina produkčných pravidiel a $S \in N$ je začiatočný symbol gramatiky [2].

Produkčné pravidlá určujú spôsob, akým môžeme nahrádzať neterminálne symboly reťazcami terminálnych a neterminálnych symbolov. Proces postupnej aplikácie produkčných pravidiel nazývame derivácia. Jazyk generovaný gramatikou G, označovaný ako $L(G)$,

definujeme ako množinu všetkých reťazcov z terminálnych symbolov, ktoré je možné odvodiť zo začiatočného symbolu pomocou konečného počtu derivácií [4]. Formálne gramatiky tak poskytujú presný mechanizmus na generovanie všetkých syntakticky korektných reťazcov daného jazyka.

1.3 Chomského hierarchia gramatík

Chomského hierarchia predstavuje klasifikáciu formálnych gramatík podľa obmedzení kladených na tvar produkčných pravidiel. Túto hierarchiu zaviedol Noam Chomsky v rámci formálneho štúdia prirodzených jazykov [5]. Hierarchia rozdeľuje gramatiky do štyroch typov, pričom každá vyššia trieda zahŕňa triedy nižšie.

Regulárne gramatiky

Regulárne gramatiky (typ 3) predstavujú najjednoduchšiu triedu v Chomského hierarchii. Produkčné pravidlá majú tvar $A \to aB$, $A \to a$ alebo $A \to \varepsilon$, kde $A, B \in N$ a $a \in \Sigma$ [2]. Jazyky generované regulárnymi gramatikami sú presne tie jazyky, ktoré rozpoznávajú konečné automaty. Regulárne gramatiky sa uplatňujú najmä pri lexikálnej analýze a spracovaní jednoduchých syntaktických vzorov.

Bezkontextové gramatiky

Bezkontextové gramatiky (typ 2) majú produkčné pravidlá tvaru $A \to \alpha$, kde $A \in N$ a $\alpha \in (N \cup \Sigma)*$. Charakteristickou črtou je skutočnosť, že ľavá strana pravidla obsahuje presne jeden neterminálny symbol, nezávisle od jeho kontextu [3].

Bezkontextové gramatiky sú výrazovo silnejšie než regulárne gramatiky a umožňujú opis rekurzívnych a hierarchických štruktúr. Ako uvádzajú Aho et al. [4], práve bezkontextové gramatiky tvoria teoretický základ syntaktickej analýzy väčšiny programovacích jazykov. Jazyky generované bezkontextovými gramatikami rozpoznávajú zásobníkové automaty, čo poukazuje na ich úzku väzbu na výpočtové modely so zásobníkom.

Kontextové gramatiky

Kontextové gramatiky (typ 1) predstavujú triedu gramatík, v ktorých aplikácia pravidla môže závisieť od kontextu prepisovaného neterminálneho symbolu. Všeobecne možno ich pravidlá zapisovať v tvare $\alpha A \beta \to \alpha \gamma \beta$, kde $A \in N$, $\alpha, \beta \in (N \cup \Sigma)^*$, $\gamma \in (N \cup \Sigma)^+$, teda

$|\gamma| \geq 1$. Symbol A sa v tomto prípade môže prepísať na reťazec γ iba vtedy, ak sa nachádza v kontexte danom reťazcami α a β [2].

Kontextové gramatiky sú výrazovo silnejšie než bezkontextové gramatiky, pretože pravidlá nemusia prepisovať neterminál nezávisle od jeho okolia. Na druhej strane sú stále obmedzenejšie než frázové gramatiky, keďže ich pravidlá vo všeobecnosti nesmú skracovať odvodený reťazec. Z praktického hľadiska sú však menej používané než bezkontextové gramatiky, pretože ich parsovanie je výpočtovo podstatne náročnejšie.

Frázové gramatiky

Frázové gramatiky (typ 0) predstavujú najvšeobecnejšiu triedu gramatík v Chomského hierarchii. Ich pravidlá majú všeobecný tvar $\alpha \rightarrow \beta$, kde $\alpha, \beta \in (N \cup \Sigma)^*$ a reťazec α obsahuje aspoň jeden neterminálny symbol [2]. Na rozdiel od regulárnych a bezkontextových gramatík teda ľavá strana pravidla nemusí pozostávať iba z jedného neterminálu. V porovnaní s kontextovými gramatikami sú frázové gramatiky ešte všeobecnejšie, pretože nevyžadujú, aby pravidlá zachovávali alebo zväčšovali dĺžku prepisovaného reťazca.

Jazyky generované frázovými gramatikami sa nazývajú rekurzívne vyčísliteľné jazyky a zodpovedajú výpočtovej sile Turingových strojov [4]. Z tohto dôvodu predstavujú najvyššiu úroveň expresívnej sily v rámci Chomského hierarchie a zahŕňajú všetky ostatné typy gramatík. V praxi sa však používajú len zriedkavo, pretože rozhodovanie viacerých vlastností týchto jazykov je vo všeobecnosti nerozhodnuteľné alebo výpočtovo veľmi náročné [3].

1.4 Význam bezkontextových gramatík

Bezkontextové gramatiky zohrávajú kľúčovú úlohu v informatike, pretože predstavujú kompromis medzi vyjadrovacou silou a výpočtovou zvládnuteľnosťou.

Syntax programovacích jazykov

Syntax takmer všetkých moderných programovacích jazykov je formálne definovaná pomocou bezkontextových gramatík. Tieto gramatiky umožňujú zachytiť vnorenú štruktúru programov, ako sú bloky, podmienené príkazy a cykly [4]. Syntaktické analyzátory v prekladačoch využívajú parsovacie algoritmy založené na CFG, napríklad LL, LR alebo všeobecné algoritmy ako CYK [6].

Modelovanie prirodzených jazykov

Bezkontextové gramatiky sa pôvodne vyvinuli v lingvistike ako nástroj na opis syntaktickej štruktúry prirodzených jazykov. Umožňujú modelovať hierarchické vzťahy medzi vetnými členmi, ako sú frázy a podfrázy [7]. Hoci prirodzené jazyky obsahujú aj kontextovo závislé javy, CFG tvoria základ mnohých syntaktických analyzátorov v oblasti spracovania prirodzeného jazyka [7].

Štruktúrované formáty dát (XML, JSON)

Bezkontextové gramatiky nachádzajú uplatnenie aj pri definovaní a validácii štruktúrovaných dátových formátov. Jazyk dobre formovaných XML dokumentov je bezkontextový, keďže vyžaduje správne párovanie otváracích a zatváracích značiek [6]. Podobne aj syntaktická špecifikácia formátu JSON je definovaná pomocou formálnej gramatiky, čo umožňuje automatické generovanie parserov a validátorov [8].

1.5 Bezkontextové gramatiky a ich vlastnosti

Bezkontextové gramatiky predstavujú jednu z najdôležitejších tried formálnych gramatík v Chomského hierarchii, keďže poskytujú vhodnú rovnováhu medzi vyjadrovacou silou a algoritmickou zvládnuteľnosťou. V tejto kapitole sa zameriavame na proces derivácie, derivačné stromy, pojem nejednoznačnosti a význam normálnych foriem, predovšetkým Chomského normálnej formy, v kontexte parsovania a redukcií.

Derivácia a derivačný strom

Základným pojmom pri práci s bezkontextovými gramatikami je derivácia. Deriváciu chápeme ako postupnú aplikáciu produkčných pravidiel, pomocou ktorej zo začiatočného symbolu S získavame reťazce zložené z terminálnych a neterminálnych symbolov. Ak existuje konečná postupnosť derivácií, ktorá vedie od symbolu S k reťazcu $w \in \Sigma *$, potom hovoríme, že reťazec w patrí do jazyka generovaného gramatikou G, teda $w \in L(G)$ [2].

Rozlišujeme ľavú a pravú deriváciu. Pri ľavej derivácii sa v každom kroku prepisuje vždy ľavý neterminálny symbol v aktuálnom reťazci, zatiaľ čo pri pravej derivácii sa prepisuje vždy pravý neterminál [4]. Tieto pojmy sú dôležité najmä pri návrhu deterministických parsovacích algoritmov, ako sú LL a LR parsery.

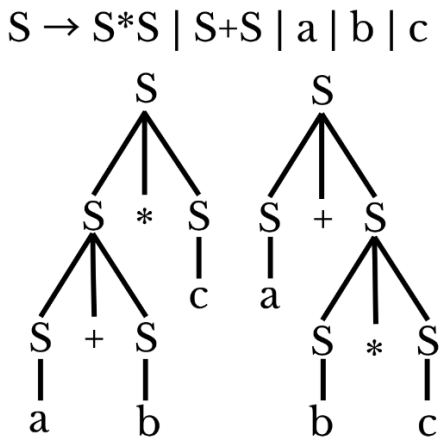
Derivačný strom (parse tree) predstavuje stromovú reprezentáciu derivácie reťazca. Koreň stromu je označený začiatočným symbolom S, vnútorné uzly sú označené neterminálmi

a listy stromu sú označené terminálnymi symbolmi alebo prázdnym symbolom ε [2]. Derivačný strom jednoznačne zachytáva hierarchickú štruktúru generovaného reťazca a abstrahuje od konkrétneho poradia aplikácie produkčných pravidiel. Dve rôzne derivácie môžu viesť k rovnakému derivačnému stromu, čo poukazuje na fakt, že derivačný strom predstavuje „sémantickejší“ pohľad na syntaktickú štruktúru reťazca než samotná sekvencia derivácií [4].

Nejednoznačnosť bezkontextových gramatík

Bezkontextová gramatika sa nazýva nejednoznačná (ambiguous), ak existuje aspoň jeden reťazec $w \in L(G)$, pre ktorý existujú dva rôzne derivačné stromy, resp. dve rôzne ľavé (alebo pravé) derivácie [2,3]. Nejednoznačnosť predstavuje dôležitý teoretický aj praktický problém, keďže rôzne derivačné stromy môžu zodpovedať rôznym sémantickým interpretáciám vstupu.

Typickým príkladom nejednoznačnosti je gramatika aritmetických výrazov bez explicitného určenia priority operátorov. Pre výraz ako $a + b * c$ môže gramatika bez ďalších obmedzení generovať dva rôzne derivačné stromy, zodpovedajúce dvom rôznym významom výrazu [4].



Obrázok 1-1 Nejednoznačnosť bezkontextových gramatík

Nejednoznačnosť je v mnohých praktických aplikáciách nežiaduca, najmä v prípade programovacích jazykov, kde sa vyžaduje jednoznačná syntaktická interpretácia vstupu.

Je dôležité poznamenať, že existujú bezkontextové jazyky, ktoré sú inherentne nejednoznačné, t. j. neexistuje pre ne žiadna jednoznačná bezkontextová gramatika [3]. Tento fakt predstavuje teoretické obmedzenie bezkontextových gramatík a má dôsledky aj pri návrhu parsovacích algoritmov.

Normálne formy bezkontextových gramatík

Pri teoretickej analýze bezkontextových gramatík sa často využívajú tzv. normálne formy, ktoré obmedzujú tvar produkčných pravidiel bez zmeny generovaného jazyka. Pre každý bezkontextový jazyk existuje ekvivalentná gramatika v danej normálnej forme [2].

1.5.2 Chomského normálna forma

Jednou z najdôležitejších normálnych foriem je Chomského normálna forma (Chomsky Normal Form, CHNF). Bezkontextová gramatika je v Chomského normálnej forme, ak všetky jej produkčné pravidlá majú jeden z nasledujúcich tvarov:

$$A \to BC, \text{ kde } A, B, C \in N$$

$$A \to a, \text{ kde } A \in N \text{ a } a \in \Sigma$$

pričom sa povoľuje aj pravidlo $S \to \varepsilon$, ak prázdny reťazec patrí do jazyka, ale v tomto prípade S nemôže byť na pravej strane žiadneho pravidla [2,3].

Transformácia ľubovoľnej bezkontextovej gramatiky do Chomského normálnej formy je vždy možná a nemení jazyk generovaný gramatikou, s výnimkou prípadnej úpravy zaobchádzania s prázdnym reťazcom [2]. Proces transformácie zahŕňa odstránenie ε-pravidiel, jednoduchých pravidiel a nadbytočných symbolov, ako aj rozklad dlhších pravých strán pravidiel na binárne pravidlá.

1.5.3 Význam Chomského normálnej formy pri parsovaní a redukciách

Chomského normálna forma zohráva významnú úlohu pri návrhu parsovacích algoritmov a pri formálnych redukciách. Jedným z najznámejších algoritmov pracujúcich výhradne s gramatikami v CHNF je algoritmus CYK (Cocke–Younger–Kasami), ktorý rozhoduje, či daný reťazec patrí do jazyka generovaného bezkontextovou gramatikou v čase $O(n^3)$, kde n je dĺžka vstupného reťazca [4,6].

Okrem parsovania je CHNF mimoriadne vhodná aj pre formálne redukcie, keďže jednotný a obmedzený tvar produkčných pravidiel umožňuje jednoduché zobrazenie gramatických pravidiel na logické obmedzenia. Táto vlastnosť je obzvlášť dôležitá pri redukcii parsovacieho problému na SAT-problém, kde je potrebné systematicky reprezentovať aplikáciu produkčných pravidiel pomocou booleovských premenných a klauzúl [6]. Použitie Chomského normálnej formy tak výrazne zjednodušuje konštrukciu formúl a umožňuje efektívnejšiu implementáciu redukčných postupov.

2 Parsovací problém

Parsovanie predstavuje jednu zo základných úloh pri spracovaní formálnych jazykov. V informatike zohráva kľúčovú úlohu najmä pri syntaktickej analýze programovacích jazykov, spracovaní prirodzeného jazyka a validácii štruktúrovaných dátových formátov. V tejto kapitole definujeme parsovací problém pre bezkontextové gramatiky, diskutujeme jeho rôzne formulácie a predstavujeme klasické algoritmy používané na jeho riešenie.

2.1 Definícia parsovania

Parsovanie (syntaktická analýza) je proces, ktorého cieľom je určiť syntaktickú štruktúru vstupného reťazca vzhľadom na danú gramatiku. Formálne povedané, pri parsovaní skúmame, či daný reťazec w patrí do jazyka generovaného gramatikou G, teda či platí $w \in L(G)$ [2]. Ak reťazec patrí do jazyka, parsovanie zároveň umožňuje rekonštruovať derivačnú štruktúru reťazca, ktorá je zvyčajne reprezentovaná vo forme derivačného stromu.

Podľa Aho et al. [4] predstavuje parsovanie centrálnu fázu syntaktickej analýzy v kompilátoroch, kde sa analyzuje postupnosť tokenov generovaných lexikálnym analyzátorom. Výsledkom parsovania je syntaktický strom, ktorý zachytáva hierarchickú štruktúru vstupného programu.

2.2 Rozhodovací a konštrukčný problém

Parsovací problém môžeme formulovať dvoma základnými spôsobmi: ako rozhodovací problém alebo ako konštrukčný problém.

2.2.1 Rozhodovací problém

Rozhodovací problém parsovania sa pýta, či daný reťazec patrí do jazyka generovaného danou gramatikou. Formálne ide o otázku: Je reťazec w generovaný gramatikou G?

Tento problém je známy ako membership problem pre bezkontextové jazyky. Pre danú bezkontextovú gramatiku G a reťazec w sa snažíme rozhodnúť, či existuje derivácia zo začiatočného symbolu S, ktorá vedie k reťazcu w [2].

Rozhodovací problém je základom mnohých praktických aplikácií, napríklad pri kontrole syntaktickej správnosti programov alebo validácii vstupných dát.

2.2.2 Konštrukčný problém

Konštrukčný problém parsovania ide o krok ďalej. Namiesto samotného rozhodnutia sa snažíme nájsť konkrétnu syntaktickú štruktúru vstupného reťazca. Otázka má teda tvar: Aké sú možné derivačné stromy pre reťazec w?

Výsledkom konštrukčného parsovania je jeden alebo viac derivačných stromov reprezentujúcich rôzne možné syntaktické interpretácie vstupu. Tento problém je dôležitý najmä v prípade nejednoznačných gramatík, kde môže existovať viacero rôznych syntaktických interpretácií toho istého reťazca [3].

2.3 Klasické parsovacie algoritmy

V priebehu vývoja teórie formálnych jazykov bolo navrhnutých viacero algoritmov na riešenie parsovacieho problému. Jedným z najznámejších všeobecných algoritmov pre bezkontextové gramatiky je algoritmus CYK (Cocke–Younger–Kasami).

2.3.1 CYK algoritmus

CYK algoritmus je dynamický programovací algoritmus, ktorý rozhoduje, či daný reťazec patrí do jazyka generovaného bezkontextovou gramatikou. Algoritmus predpokladá, že gramatika je zapísaná v Chomského normálnej forme (CHNF) [2].

Algoritmus pracuje s tabuľkou veľkosti $n \times n$, kde n je dĺžka vstupného reťazca. Každá bunka tabuľky reprezentuje množinu neterminálnych symbolov, ktoré môžu generovať príslušný podreťazec vstupu. Výpočet prebieha zdola nahor – najprv sa analyzujú jednotlivé terminálne symboly a následne sa postupne kombinujú väčšie podreťazce pomocou pravidiel typu $A \rightarrow BC$ [6].

CYK algoritmus je významný najmä z teoretického hľadiska, keďže poskytuje univerzálny postup pre parsovanie ľubovoľných bezkontextových gramatík po ich transformácii do Chomského normálnej formy.

2.3.2 Časová zložitosť parsovania

Z hľadiska výpočtovej zložitosti patrí parsovanie bezkontextových jazykov medzi problémy riešiteľné v polynomiálnom čase. Pre všeobecné bezkontextové gramatiky má CYK algoritmus časovú zložitosť $O(n^3)$, kde n je dĺžka vstupného reťazca [2,6].

Táto kubická zložitosť vyplýva z potreby skúmať všetky možné rozdelenia podreťazcov a kombinácie pravidiel. Hoci existujú špeciálne triedy bezkontextových gramatík, pre ktoré

je možné parsovanie vykonať v lineárnom čase (napríklad LL alebo LR gramatiky), v najvšeobecnejšom prípade zostáva zložitosť $O(n^3)$ [4].

2.4 Deterministické parsovacie algoritmy (LL a LR)

Okrem všeobecných parsovacích algoritmov existuje aj skupina algoritmov určených pre špeciálne triedy bezkontextových gramatík. Medzi najznámejšie patria LL a LR parsovacie algoritmy, ktoré sa široko používajú najmä pri syntaktickej analýze programovacích jazykov.

LL parsovanie patrí medzi tzv. top-down parsovacie metódy. Pri tomto prístupe sa parsovanie začína od začiatočného symbolu gramatiky a postupne sa snažíme odvodiť vstupný reťazec aplikovaním pravidiel. Označenie LL znamená, že parser číta vstup zľava doprava (Left-to-right) a vytvára ľavú deriváciu (Leftmost derivation) [4].

LL parsery sú relatívne jednoduché na implementáciu a často sa realizujú ako rekurzívne zostupné parsery. Ich nevýhodou však je, že fungujú len pre obmedzenú triedu bezkontextových gramatík, označovanú ako *LL(k)* gramatiky, kde *k* predstavuje počet symbolov vstupu, ktoré parser môže dopredu analyzovať [2]. Mnohé praktické gramatiky programovacích jazykov je preto potrebné pred použitím LL parsovania upraviť, napríklad odstránením ľavej rekurzie.

Na rozdiel od LL parsovania patrí LR parsovanie medzi bottom-up parsovacie metódy. Pri tomto prístupe parser postupne redukuje vstupný reťazec na začiatočný symbol gramatiky pomocou sekvencie redukcií pravidiel. Označenie LR znamená, že parser číta vstup zľava doprava (Left-to-right) a vytvára pravú deriváciu v obrátenom poradí (Rightmost derivation in reverse) [4].

LR parsery sú výrazne výkonnejšie než LL parsery a dokážu spracovať širšiu triedu bezkontextových gramatík. Medzi najznámejšie varianty patria algoritmy SLR, LALR a Canonical LR. Tieto metódy sa používajú v mnohých generátoroch parserov, napríklad v nástrojoch YACC alebo Bison, ktoré automaticky generujú syntaktické analyzátory z formálne definovaných gramatík [4,6].

Nevýhodou LR parserov je ich väčšia implementačná zložitosť a potreba konštruovať rozsiahle parsovacie tabuľky. Napriek tomu predstavujú štandardný nástroj pri implementácii kompilátorov.

2.5 Porovnanie parsovacích prístupov

Jednotlivé parsovacie algoritmy sa líšia najmä v oblasti vyjadrovacej sily, výpočtovej zložitosti a praktickej použiteľnosti.

Algoritmus CYK patrí medzi všeobecné algoritmy pre spracovanie bezkontextových gramatík. Jeho hlavným cieľom je riešiť rozhodovací problém, teda určiť, či daný reťazec patrí do jazyka generovaného gramatikou, formálne či $w \in L(G)$ [2]. Algoritmus predpokladá, že gramatika je prevedená do Chomského normálnej formy, a pracuje pomocou dynamického programovania nad tabuľkou podreťazcov. Výsledkom algoritmu je informácia o existencii derivácie, nie však explicitná konštrukcia derivačného stromu pre pôvodnú gramatiku. Výhodou CYK algoritmu je jeho univerzálnosť – dokáže spracovať ľubovoľnú bezkontextovú gramatiku po jej transformácii do CHNF. Nevýhodou je jeho kubická časová zložitosť $O(n^3)$, ktorá môže byť problematická pri spracovaní dlhých vstupov [2].

Na druhej strane LL a LR parsery sú deterministické parsovacie algoritmy určené pre špeciálne triedy gramatík. Ich výhodou je výrazne nižšia časová zložitosť, ktorá je v praxi lineárna vzhľadom na dĺžku vstupu. Táto vlastnosť je kľúčová najmä pri syntaktickej analýze programovacích jazykov, kde je potrebné spracovať veľké množstvo zdrojového kódu efektívne [4].

Nevýhodou deterministických parserov je však obmedzenie na určité triedy gramatík. Nie všetky bezkontextové gramatiky je možné spracovať pomocou LL alebo LR parsovania bez dodatočných úprav gramatiky.

Zhrnutím môžeme povedať, že klasické parsovacie algoritmy predstavujú kompromis medzi všeobecnosťou a efektivitou. Všeobecné algoritmy, ako je CYK, poskytujú univerzálne riešenie parsovacieho problému, avšak za cenu vyššej výpočtovej zložitosti. Deterministické algoritmy LL a LR sú efektívnejšie, no ich použiteľnosť je obmedzená štruktúrou gramatiky.

Tieto obmedzenia klasických prístupov vytvárajú priestor pre alternatívne metódy riešenia parsovacieho problému. Jednou z perspektívnych možností je formulovať parsovanie ako problém splniteľnosti booleovských formúl a riešiť ho pomocou moderných SAT riešičov, čomu sa budeme venovať v nasledujúcej kapitole.

3 SAT-problém

SAT-problém (Boolean Satisfiability Problem) patrí medzi najvýznamnejšie problémy teoretickej informatiky a zohráva dôležitú úlohu v oblasti algoritmov, optimalizácie a formálnej verifikácie. V tejto kapitole predstavujeme základné pojmy booleovskej logiky, definujeme rozhodovací SAT-problém a stručne vysvetľujeme jeho význam v kontexte výpočtovej zložitosti a praktických aplikácií.

3.1 Booleovská logika

Booleovská logika predstavuje formálny systém určený na manipuláciu s logickými výrokmi, ktorých hodnoty môžu byť pravda alebo nepravda. Základnými operátormi booleovskej logiky sú konjunkcia (AND), disjunkcia (OR) a negácia (NOT). Tieto operácie umožňujú konštruovať zložité logické výrazy z jednoduchších premenných [9].

Booleovská logika tvorí základ mnohých oblastí informatiky, najmä digitálnych obvodov, formálnej verifikácie a automatizovaného dokazovania. Ako uvádzajú Biere et al. [9], väčšina problémov z oblasti verifikácie a optimalizácie môže byť transformovaná do formy booleovskej formuly, ktorej splniteľnosť je následne analyzovaná pomocou SAT riešičov.

3.2 Literál, klauzula a CNF formula

Základnými stavebnými prvkami SAT-problému sú literály, klauzuly a logické formuly.

Literál je buď booleovská premenná, alebo jej negácia. Ak označíme premennú x, potom literály sú x alebo $\neg x$.

Klauzula je disjunkcia (logické OR) literálov. Napríklad výraz

$$(x \vee \neg y \vee z)$$

predstavuje klauzulu pozostávajúcu z troch literálov.

Konjunktívna normálna forma (CNF) je logická formula, ktorá je konjunkciou klauzúl, teda má tvar

$$(C_1 \wedge C_2 \wedge \ldots \wedge C_{\mathrm{n}})$$

kde každá C_{i} je klauzula [3,9]. CNF predstavuje štandardnú reprezentáciu logických formúl používanú v SAT riešičoch, pretože umožňuje efektívne aplikovať algoritmy na kontrolu splniteľnosti.

3.3 Definícia rozhodovacieho SAT-problému

Rozhodovací SAT-problém môžeme formulovať nasledovne: Je daná booleovská formula v konjunktívnej normálnej forme splniteľná?

Inými slovami, hľadáme priradenie pravdivostných hodnôt booleovským premenným tak, aby celá formula nadobudla hodnotu pravda [3].

SAT-problém bol prvým problémom dokázaným ako NP-úplný. Tento výsledok publikoval Stephen Cook v roku 1971 vo svojej známej práci *The Complexity of Theorem-Proving Procedures* [10]. Cookov výsledok ukazuje, že každý problém z triedy NP je možné v polynomiálnom čase redukovať na SAT-problém. Krátko nato Richard Karp identifikoval ďalšie známe NP-úplné problémy a potvrdil centrálnu úlohu SAT-problému v teórii výpočtovej zložitosti [11].

NP-úplnosť znamená, že SAT je jeden z najťažších problémov v triede NP. Ak by existoval polynomiálny algoritmus na riešenie SAT-problému, potom by bolo možné v polynomiálnom čase riešiť všetky problémy z triedy NP [3].

3.4 Význam SAT-problému

SAT-problém má mimoriadny význam v informatike, pretože predstavuje univerzálny formalizmus na riešenie širokej triedy problémov. Mnohé praktické problémy je možné transformovať do formy booleovskej formuly a následne riešiť pomocou SAT riešiča.

Jednou z hlavných výhod SAT-prístupu je možnosť redukcie rôznych problémov na SAT. Tento prístup umožňuje transformovať problém z inej oblasti informatiky na problém splniteľnosti booleovskej formuly. Následne je možné využiť existujúce optimalizované SAT riešiče bez potreby navrhovať nový špecializovaný algoritmus [9].

V posledných desaťročiach sa výkonnosť SAT riešičov dramaticky zlepšila. Moderné SAT riešiče dokážu riešiť formuly obsahujúce milióny premenných a klauzúl, čo umožňuje ich využitie v reálnych priemyselných aplikáciách [9].

3.5 Moderné SAT riešiče

Moderné SAT riešiče sú založené na rôznych heuristikách a optimalizačných technikách. Jedným z najvýznamnejších prístupov je algoritmus CDCL (Conflict-Driven Clause Learning), ktorý predstavuje rozšírenie klasického algoritmu DPLL (Davis–Putnam–Logemann–Loveland) [9].

CDCL algoritmus využíva mechanizmus učenia z konfliktov. Ak počas hľadania riešenia dôjde ku konfliktu, algoritmus analyzuje príčinu konfliktu a vytvorí novú klauzulu, ktorá zabraňuje opakovaniu rovnakého konfliktu v budúcnosti. Tento mechanizmus výrazne zvyšuje efektivitu riešenia a umožňuje SAT riešičom rýchlo eliminovať veľké časti priestoru riešení [9,12].

Okrem toho moderné SAT riešiče využívajú rôzne heuristiky na výber premenných, techniky propagácie jednotkových klauzúl a optimalizované dátové štruktúry na spracovanie veľkých formúl.

3.6 Oblasti použitia SAT-problému

Vďaka vysokému výkonu moderných SAT riešičov sa SAT-problém stal základným nástrojom v mnohých oblastiach informatiky. Jednou z najdôležitejších aplikácií je verifikácia hardvéru, kde sa SAT riešiče používajú na kontrolu správnosti digitálnych obvodov a procesorových architektúr [9]. Podobne sa SAT využíva aj pri verifikácii softvéru, kde umožňuje automaticky overovať vlastnosti programov a identifikovať potenciálne chyby. Ďalšou významnou oblasťou je plánovanie a rozvrhovanie, kde SAT riešiče pomáhajú riešiť komplexné optimalizačné problémy, napríklad pri plánovaní výroby alebo logistike. SAT metódy sa používajú aj v bioinformatike, napríklad pri analýze genetických dát alebo modelovaní biologických sietí. Napokon významné uplatnenie nachádzajú aj v oblasti umelej inteligencie, kde sa SAT využíva pri riešení problémov splniteľnosti obmedzení, plánovania a automatizovaného dokazovania tvrdení [9].

4 Pojem redukcie a jej význam

Redukcia problémov predstavuje základný nástroj teoretickej informatiky, ktorý umožňuje porovnávať zložitosť rôznych výpočtových problémov a prenášať riešenia medzi nimi. V tejto kapitole definujeme pojem redukcie, vysvetľujeme jej význam a poukazujeme na dôvody, prečo je redukcia na SAT-problém prakticky aj teoreticky výhodná.

4.1 Definícia redukcie problémov

Redukciu problémov chápeme ako transformáciu jedného problému na iný problém takým spôsobom, že riešenie druhého problému umožňuje vyriešiť pôvodný problém. Formálne hovoríme, že problém A je redukovateľný na problém B, ak existuje efektívny (typicky polynomiálny) algoritmus, ktorý každú inštanciu problému A prevedie na inštanciu problému B tak, že odpoveď na problém B zodpovedá odpovedi na problém A [3].

Redukcie zohrávajú kľúčovú úlohu najmä v teórii výpočtovej zložitosti. Ako ukazuje Cookova veta, každý problém z triedy NP je možné v polynomiálnom čase redukovať na SAT-problém, čím sa SAT stáva centrálnym problémom tejto triedy [10]. Redukcie teda umožňujú klasifikovať problémy podľa ich náročnosti a identifikovať tzv. NP-úplné problémy.

4.2 Redukcie medzi rozhodovacími problémami

V kontexte tejto práce nás zaujímajú najmä redukcie medzi rozhodovacími problémami. Rozhodovací problém je taký problém, ktorého výstupom je odpoveď „áno“ alebo „nie“. Parsovací problém vo svojej základnej forme patrí práve do tejto kategórie, keďže sa pýtame, či daný reťazec patrí do jazyka generovaného gramatikou.

Redukcia medzi rozhodovacími problémami zachováva pravdivostnú hodnotu odpovede. To znamená, že ak je pôvodná inštancia problému riešiteľná, potom aj transformovaná inštancia musí byť riešiteľná a naopak [3]. Táto vlastnosť je kľúčová pri návrhu korektných redukčných postupov.

4.3 Dôvody redukcie na SAT

Jedným z hlavných dôvodov, prečo sa v súčasnosti často využíva redukcia problémov na SAT, je existencia vysoko optimalizovaných SAT riešičov. Moderné SAT riešiče, založené

napríklad na algoritmoch typu CDCL, dokážu efektívne spracovať veľmi veľké logické formuly a sú výsledkom dlhoročného výskumu a optimalizácie [9].

Redukcia na SAT umožňuje využiť tieto existujúce nástroje bez potreby navrhovať nový špecializovaný algoritmus pre každý konkrétny problém. Namiesto toho sa problém transformuje do jednotného formátu (CNF formula), ktorý je možné riešiť univerzálnym spôsobom [9].

Ďalším dôvodom je flexibilita tohto prístupu. SAT formulácie umožňujú jednoducho pridávať nové obmedzenia, čím sa rozširuje riešený problém bez zásadnej zmeny algoritmu. Tento aspekt je obzvlášť dôležitý pri komplexných problémoch, kde sa požiadavky môžu meniť.

Z teoretického hľadiska redukcie umožňujú lepšie pochopiť vzťahy medzi problémami a ich výpočtovú náročnosť. Umožňujú prenášať známe výsledky z jednej oblasti do druhej a identifikovať všeobecné riešiteľné a neriešiteľné triedy problémov [3].

Z praktického hľadiska redukcie prinášajú možnosť využiť existujúce optimalizované riešenia. V prípade SAT to znamená využitie moderných riešičov, ktoré sú schopné efektívne riešiť problémy veľkého rozsahu. Tento prístup je dnes štandardom v oblastiach ako formálna verifikácia alebo plánovanie [9].

4.4 Parsovací problém pomocou SAT

Parsovací problém pre bezkontextové gramatiky môžeme interpretovať ako problém určovania vzťahov medzi symbolmi a pozíciami v reťazci. Tento pohľad umožňuje formulovať parsovanie ako kombináciu logických obmedzení, čo prirodzene vedie k jeho reprezentácii pomocou SAT-problému.

4.4.1 Parsovanie ako množina vzťahov

Pri parsovaní analyzujeme, aké produkčné pravidlá boli použité a akým spôsobom súvisia jednotlivé časti vstupného reťazca. Každý derivačný strom môžeme chápať ako množinu vzťahov medzi neterminálnymi symbolmi a podreťazcami vstupu.

Tieto vzťahy zahŕňajú najmä:

- priradenie neterminálnych symbolov k intervalom v reťazci,
- aplikácie produkčných pravidiel typu $A \rightarrow BC$ alebo $A \rightarrow a$,
- hierarchické vzťahy medzi uzlami derivačného stromu.

Takýto pohľad umožňuje interpretovať parsovanie ako problém splnenia množiny podmienok, ktoré musia byť súčasne splnené.

4.4.2 Vyjadrenie pomocou logických obmedzení

Vzťahy vznikajúce pri parsovaní môžeme prirodzene reprezentovať pomocou binárnych premenných. Napríklad premenná môže reprezentovať, či určitý neterminál generuje konkrétny podreťazec vstupu.

Tieto premenné následne viažeme logickými obmedzeniami, ktoré vyplývajú z produkčných pravidiel gramatiky. Napríklad pravidlo $A \rightarrow BC$ môžeme interpretovať ako podmienku, že neterminál A generuje podreťazec iba vtedy, ak existuje rozdelenie reťazca na dve časti generované neterminálmi B a C.

Takto vzniká booleovská formula, ktorej splniteľnosť zodpovedá existencii derivačného stromu pre daný reťazec.

4.4.3 Výhody SAT prístupu

Použitie SAT pri riešení parsovacieho problému prináša viacero výhod. Jednou z nich je alternatívna riešiaca stratégia, ktorá sa líši od klasických dynamických alebo deterministických parsovacích algoritmov.

Ďalšou výhodou je dobrá paralelizovateľnosť. Moderné SAT riešiče sú navrhnuté tak, aby efektívne využívali viacjadrové procesory, čo umožňuje zrýchliť riešenie veľkých problémov [9].

SAT prístup tiež umožňuje jednoduché rozšírenia problému. Napríklad je možné pridať obmedzenia na počet derivácií, preferovať určité pravidlá alebo kombinovať parsovanie s inými typmi obmedzení bez zásadnej zmeny algoritmu.

4.4.4 Obmedzenia SAT prístupu

Napriek svojim výhodám má SAT prístup aj určité obmedzenia. Jedným z hlavných problémov je veľkosť výslednej booleovskej formule, ktorá môže rásť veľmi rýchlo s dĺžkou vstupného reťazca a počtom produkčných pravidiel.

Ďalším faktorom sú režijné náklady SAT riešiča. Hoci moderné riešiče sú veľmi efektívne, ich inicializácia a spracovanie veľkých formúl môže byť časovo náročné.

Napokon, SAT prístup nie je vždy rýchlejší než klasické parsovacie algoritmy. Pre špeciálne triedy gramatík, ako sú LL alebo LR gramatiky, môžu byť deterministické parsery výrazne efektívnejšie.

5 Prehľad súvisiacich prác

V tejto kapitole predstavujeme prehľad vybraných vedeckých prác, ktoré sa zaoberajú využitím SAT riešičov pri práci s bezkontextovými gramatikami. Cieľom tejto kapitoly je identifikovať základné prístupy k redukcii problémov súvisiacich s bezkontextovými gramatikami na SAT-problém a poukázať na ich spoločné znaky a rozdiely.

5.1 Analyzing Context-Free Grammars Using an Incremental SAT Solver

Práca Axelssona, Heljanka a Langeho [1] sa zameriava na analýzu vlastností bezkontextových gramatík pomocou inkrementálneho SAT riešiča. Hlavným cieľom autorov je ukázať, že rôzne vlastnosti gramatík, ako napríklad generovanie konkrétnych reťazcov alebo existencia určitých derivácií, je možné formulovať ako SAT-problém. Autori navrhujú model, v ktorom sa derivácie bezkontextovej gramatiky reprezentujú pomocou booleovských premenných. Každá premenná reprezentuje určitý aspekt derivácie, napríklad priradenie neterminálu k podreťazcu vstupu. Produkčné pravidlá gramatiky sú následne transformované na logické obmedzenia, ktoré zabezpečujú konzistentnosť derivácie.

Kľúčovým prvkom tejto práce je využitie inkrementálneho SAT riešenia. Namiesto opakovaného vytvárania a riešenia nezávislých SAT formulácií autori postupne rozširujú existujúcu formulu o nové obmedzenia. Tento prístup umožňuje efektívnejšie riešiť sekvenciu súvisiacich problémov, napríklad pri postupnom zvyšovaní dĺžky analyzovaného reťazca. Výsledky ukazujú, že SAT prístup môže byť efektívny pri analýze určitých vlastností gramatík, najmä ak sa využíva inkrementálne riešenie a moderné SAT riešiče.

5.2 Learning Context-Free Grammars by Using SAT Solvers

Práca Imadu a Nakamuru [13] sa zameriava na iný, ale príbuzný problém, konkrétne na učenie bezkontextových gramatík pomocou SAT riešičov. Cieľom tejto práce je odvodiť gramatiku z daného množstva pozitívnych a negatívnych príkladov reťazcov. Autori formulujú problém učenia ako SAT-problém tak, že definujú množinu booleovských premenných reprezentujúcich produkčné pravidlá potenciálnej gramatiky. Následne konštruujú logické obmedzenia, ktoré zabezpečujú, že výsledná gramatika generuje všetky pozitívne príklady a zároveň negeneruje negatívne príklady.

Základnou myšlienkou prístupu je transformovať problém syntézy gramatiky na problém splniteľnosti logickej formuly. SAT riešič potom nájde takú konfiguráciu produkčných pravidiel, ktorá spĺňa všetky zadané podmienky. Tento prístup demonštruje flexibilitu SAT formalizmu, keďže umožňuje riešiť nielen parsovací problém, ale aj problém učenia gramatík, ktorý je výrazne komplexnejší.

5.3 Porovnanie prístupov

Obe analyzované práce využívajú spoločný princíp redukcie problémov súvisiacich s bezkontextovými gramatikami na SAT-problém, avšak líšia sa vo svojom cieli a konkrétnom prístupe.

Práca Axelssona et al. [1] sa zameriava na analýzu existujúcej gramatiky, pričom cieľom je overiť určité vlastnosti alebo existenciu derivácií. Naopak práca Imadu a Nakamuru [13] rieši problém syntézy gramatiky na základe dát, teda ide o konštrukčný problém vyššej úrovne.

Z hľadiska modelovania obe práce využívajú reprezentáciu pomocou booleovských premenných a logických obmedzení. Rozdiel spočíva najmä v tom, čo tieto premenné reprezentujú. V prvom prípade ide o vzťahy medzi neterminálmi a podreťazcami, zatiaľ čo v druhom prípade ide o samotné produkčné pravidlá gramatiky.

Významným rozdielom je aj použitie inkrementálneho SAT riešenia v práci Axelssona et al., čo umožňuje efektívnejšie spracovanie sekvencie súvisiacich problémov. Tento aspekt nie je prítomný v práci Imadu a Nakamuru, kde sa rieši jednorazová formulácia problému učenia.

Obe práce však potvrdzujú, že SAT predstavuje silný a flexibilný nástroj na riešenie problémov súvisiacich s bezkontextovými gramatikami. Zároveň poukazujú na rôzne možnosti modelovania týchto problémov pomocou logických formúl, čo vytvára priestor pre ďalší výskum a návrh nových prístupov.

6 Návrh vlastného algoritmu

6.1 Teoretické východiská navrhovaného algoritmu

Navrhovaný algoritmus vychádza z myšlienky transformovať parsovací problém pre bezkontextové gramatiky na problém splniteľnosti booleovskej formuly (SAT). Cieľom algoritmu je rozhodnúť, či daný reťazec patrí do jazyka generovaného gramatikou, teda riešiť tzv. membership problem, pričom zároveň umožňuje rekonštruovať derivačný strom reprezentujúci syntaktickú štruktúru vstupu.

Teoretický návrh algoritmu vychádza z prístupu opísaného v práci Axelsson et al. [1], ktorá sa zaoberá analýzou bezkontextových gramatík pomocou inkrementálneho SAT riešiča. Z uvedeného článku preberáme najmä základnú myšlienku reprezentovať vzťah medzi neterminálnymi symbolmi a podreťazcami vstupného slova pomocou booleovských premenných a následne vyjadriť pravidlá gramatiky ako logické obmedzenia. Táto práca teda nenavrhuje úplne novú redukciu parsovacieho problému na SAT, ale implementuje a upravuje existujúcu myšlienku pre vlastný systém, ktorý pracuje s gramatikou v CHNF, generuje CNF formulu, exportuje ju do DIMACS formátu a experimentálne vyhodnocuje správanie rôznych SAT riešičov.

Základom navrhovaného prístupu je pozorovanie, že parsovanie bezkontextovej gramatiky možno chápať ako množinu vzťahov medzi neterminálnymi symbolmi a podreťazcami vstupného reťazca. Podobne ako pri algoritme CYK, aj v tomto prípade uvažujeme, či určitý neterminálny symbol generuje konkrétny interval vstupu. Na rozdiel od klasického dynamického programovania však tieto vzťahy neukladáme do tabuľky, ale reprezentujeme ich pomocou booleovských premenných a logických obmedzení.

Navrhovaný algoritmus pracuje s bezkontextovou gramatikou prevedenou do Chomského normálnej formy. Tento krok je nevyhnutný, pretože produkčné pravidlá v CHNF majú jednotný tvar $A \to BC$ alebo $A \to a$, čo umožňuje systematickú konštrukciu logických obmedzení. Transformáciu gramatiky do CHNF zabezpečuje samostatný parser, ktorý odstraňuje ε-produkcie, jednotkové pravidlá a zjednodušuje štruktúru gramatiky bez zmeny generovaného jazyka.

Kľúčovým prvkom modelu, ktorý preberáme z prístupu Axelsson et al. [1], je zavedenie booleovských premenných typu

$$X_{A,i,j}$$

ktoré reprezentujú tvrdenie, že neterminálny symbol A generuje podreťazec vstupu od pozície i po j. V našej implementácii túto myšlienku prispôsobujeme indexovaniu od nuly a práci s tokenizovaným vstupným slovom. Týmto spôsobom prevádzame problém parsovania

na problém určovania pravdivostných hodnôt týchto premenných tak, aby boli splnené všetky obmedzenia vyplývajúce z produkčných pravidiel gramatiky.

Logické obmedzenia sú následne konštruované tak, aby presne zodpovedali pravidlám derivácie. Terminálne pravidlá zabezpečujú konzistenciu medzi symbolmi vstupného reťazca a neterminálmi, zatiaľ čo binárne pravidlá reprezentujú spôsob, akým možno väčšie podreťazce rozdeliť na menšie časti. Výsledná množina obmedzení je prevedená do konjunktívnej normálnej formy (CNF), čím vzniká booleovská formula vhodná pre SAT riešič.

Riešenie tejto formule pomocou SAT riešiča umožňuje určiť, či existuje taká kombinácia premenných, ktorá spĺňa všetky obmedzenia. Ak je formula splniteľná, znamená to, že existuje derivačný strom pre daný reťazec, a teda reťazec patrí do jazyka generovaného gramatikou. Navyše, z nájdeného ohodnotenia premenných je možné spätne rekonštruovať konkrétny derivačný strom, čím algoritmus poskytuje nielen odpoveď na rozhodovací problém, ale aj riešenie konštrukčného problému parsovania.

Navrhovaný prístup tak predstavuje alternatívu ku klasickým parsovacím algoritmom, ako je CYK, pričom využíva silu moderných SAT riešičov. Hlavnou výhodou tohto prístupu je jeho flexibilita – logické obmedzenia je možné jednoducho rozšíriť o ďalšie podmienky bez zásadnej zmeny algoritmu. Na druhej strane však veľkosť výslednej formule môže rásť s dĺžkou vstupu a počtom pravidiel, čo predstavuje potenciálne obmedzenie z hľadiska výpočtovej efektivity.

Z článku [1] teda preberáme základný logický pohľad na parsovanie pomocou premenných reprezentujúcich odvodenie podreťazcov a ideu prevodu pravidiel gramatiky na SAT obmedzenia. Vlastná časť tejto práce spočíva najmä v implementácii parsera, transformácii vstupných gramatík do CHNF, konštrukcii konkrétnych klauzúl v programe, exporte do DIMACS formátu, rekonštrukcii derivačného stromu a v experimentálnom porovnaní SAT riešičov a algoritmu CYK.

6.2 Reprezentácia parsovania pomocou booleovských premenných

Navrhovaný algoritmus reprezentuje parsovací problém pre bezkontextovú gramatiku v Chomského normálnej forme pomocou množiny booleovských premenných. Táto reprezentácia je inšpirovaná prácou Axelsson et al. [1], v ktorej sa odvodenie podreťazcov vstupného slova vyjadruje pomocou logických premenných. Základnou myšlienkou je priradiť booleovské premenné tvrdeniam o tom, či určitý neterminálny symbol generuje konkrétny podreťazec vstupného slova. Týmto spôsobom sa problém syntaktickej analýzy prevádza na problém hľadania takého pravdivostného ohodnotenia premenných, ktoré spĺňa všetky obmedzenia vyplývajúce z gramatických pravidiel.

Nech $G = \langle N, \Sigma, P, S \rangle$ je bezkontextová gramatika prevedená do Chomského normálnej formy a nech vstupné slovo tvorí postupnosť tokenov

$$w = w_0 w_1 \dots w_{n-1}.$$

V implementácii používame indexovanie od nuly, preto podreťazce identifikujeme dvojicami indexov (i, j), kde $0 \leq i \leq j < n$. Každý takýto interval reprezentuje súvislý úsek vstupného slova od pozície i po pozíciu j.

6.2.1 Základné premenné modelu

Jadrom celej SAT formulácie sú premenné tvaru $X_{A,i,j}$. Takýto typ premenných vychádza z intervalovej reprezentácie parsovania použitej v práci Axelsson et al. [1]. V našej implementácii tieto premenné definujeme pre každý neterminál $A \in N$ a každý interval $0 \leq i \leq j < n$. Význam tejto premennej je nasledovný:

$$X_{A,i,j} = \text{true} \Leftrightarrow A \text{ generuje podreťazec } w_i \dots w_j.$$

Inými slovami, premenná $X_{A,i,j}$ reprezentuje tvrdenie, že neterminálny symbol A dokáže odvodiť časť vstupu medzi pozíciami i a j. Táto interpretácia priamo zodpovedá spôsobu, akým sa pri klasickom algoritme CYK vypĺňa tabuľka derivovateľnosti podreťazcov. Rozdiel spočíva v tom, že namiesto explicitnej tabuľky používame booleovské premenné, ktoré sa stávajú súčasťou SAT formulácie.

Pre každý neterminál a pre každý interval (i, j) vytvárame práve jednu takúto premennú. Počet základných premenných je teda úmerný počtu neterminálov a počtu všetkých možných intervalov vstupného slova. Keďže počet intervalov v slove dĺžky n je $\frac{n(n+1)}{2}$, celkový počet základných premenných je rádu

$$O(|N| \cdot n^2).$$

Táto množina premenných tvorí základný opis parsovacieho priestoru. Každé pravdivostné ohodnotenie týchto premenných predstavuje určitý kandidátny opis toho, ktoré neterminály generujú ktoré časti vstupu. Úlohou klauzúl je následne zabezpečiť, aby boli prípustné len tie ohodnotenia, ktoré zodpovedajú platnej derivácii podľa pravidiel gramatiky.

6.2.2 Inicializované pravdivostné hodnoty

Samotná interpretácia premenných $X_{A,i,j}$ vychádza z prístupu [1], avšak v našej implementácii niektorým z týchto premenných priraďujeme pravdivostné hodnoty už počas konštrukcie modelu. Ide o implementačné predspracovanie, ktoré využíva priamo známe

informácie zo vstupného slova a zo štruktúry pravidiel v CHNF. Tým znižujeme priestor hľadania a zároveň priamo zapracúvame informácie vyplývajúce zo vstupného slova a terminálnych pravidiel.

Po prvé, premenná $X_{S,0,n-1}$ je nastavená na hodnotu true, pretože cieľom celého modelu je overiť, či štartovací symbol S generuje celé vstupné slovo. Táto podmienka zodpovedá membership problému a predstavuje cieľovú podmienku celej SAT formulácie.

Po druhé, pri terminálnych pravidlách tvaru $A \to a$ nastavujeme premenné $X_{A,i,i}$ na hodnotu true práve vtedy, keď token na pozícii i je zhodný s terminálom a. Ak takáto zhoda neexistuje, príslušné diagonálne premenné ostávajú nepravdivé. Týmto spôsobom už na úrovni jednotkových klauzúl zakódujeme základné informácie o tom, ktoré neterminály môžu generovať jednoprvkové podreťazce.

Po tretie, ak pre daný neterminál A neexistuje žiadne binárne pravidlo tvaru $A \to BC$, tak všetky premenné $X_{A,i,j}$ pre intervaly dĺžky väčšej než jeden symbol nastavujeme na hodnotu false. Takýto neterminál totiž v Chomského normálnej forme nemôže generovať podreťazec dĺžky aspoň dva. Aj tento krok znižuje počet premenných, ktoré musí riešič efektívne zvažovať.

Z teoretického hľadiska ide o formu predspracovania modelu, v ktorej využívame jednoduché lokálne dôsledky pravidiel gramatiky ešte pred vytvorením zložitejších klauzúl. Tým sa celá formulácia stáva kompaktnejšou a prehľadnejšou.

6.2.3 Pomocné premenné pre binárne pravidlá

Okrem základných premenných typu $X_{A,i,j}$ zavádzame v implementácii aj pomocné booleovské premenné pre binárne pravidlá. Tieto premenné vznikajú pri spracovaní pravidiel tvaru $A \to BC$ a pri všetkých možných rozdeleniach intervalu (i, j) na dve časti. Pre každé pravidlo a pre každý index rozdelenia k, kde $i \leq k < j$, zavádzame pomocnú premennú, ktorú môžeme teoreticky označiť napríklad ako

$$Z_{A \to BC,i,k,j}.$$

Význam tejto premennej je:

$$Z_{A \to BC,i,k,j} = \text{true} \Leftrightarrow X_{B,i,k} \wedge X_{C,k+1,j}.$$

Táto premenná teda reprezentuje konkrétnu možnosť, že podreťazec $w_i \dots w_j$ vzniká použitím pravidla $A \to BC$ a rozdelením v bode k. Zavedenie takýchto pomocných premenných je výhodné najmä preto, že zjednodušuje prevod logických podmienok do CNF. Namiesto toho, aby sme vytvárali veľmi dlhé a zložité formuly, rozkladáme ich na menšie lokálne vzťahy medzi premennými, ktoré následne prepájame klauzulami.

V implementácii sú tieto pomocné premenné prideľované dynamicky po vytvorení všetkých základných premenných $X_{A,i,j}$. Ich počet závisí od počtu binárnych pravidiel a od počtu všetkých možných trojíc (i, k, j). Preto je ich celkový počet v najhoršom prípade rádu

$$O(|P_{bin}| \cdot n^3),$$

kde $|P_{bin}|$ označuje počet binárnych pravidiel gramatiky.

6.2.4 Intuitívny význam reprezentácie

Takto zvolená reprezentácia nám umožňuje interpretovať parsovanie ako problém konzistencie medzi dvoma typmi tvrdení. Základné premenné $X_{A,i,j}$ hovoria o tom, ktoré neterminály generujú ktoré intervaly vstupu, zatiaľ čo pomocné premenné $Z_{A \to BC,i,k,j}$ zachytávajú konkrétne spôsoby, akými môže byť táto generácia realizovaná pomocou binárnych pravidiel.

Výhodou tejto reprezentácie je, že veľmi prirodzene kopíruje štruktúru derivačného stromu. Každý pravdivý literál $X_{A,i,j}$ možno chápať ako uzol derivačného stromu pokrývajúci určitý interval vstupu, a každá pravdivá pomocná premenná $Z_{A \to BC,i,k,j}$ predstavuje rozklad tohto uzla na dve dcérske vetvy. Z tohto dôvodu je možné po nájdení splniteľného ohodnotenia nielen rozhodnúť, že vstupné slovo patrí do jazyka, ale aj spätne zrekonštruovať konkrétny derivačný strom.

Zároveň táto reprezentácia zachováva úzku väzbu na klasický parsovací problém. Namiesto explicitného výpočtu tabuľky derivovateľnosti, ako je to v prípade algoritmu CYK, konštruujeme množinu premenných a klauzúl, ktorých splniteľnosť zodpovedá existencii tej istej odvodenosti. Navrhovaný model teda možno chápať ako logickú reformuláciu substringového pohľadu na bezkontextové parsovanie.

6.3 Konštrukcia klauzúl

Konštrukcia klauzúl vychádza z logickej interpretácie parsovania opísanej v práci Axelsson et al. [1]. Z tejto práce preberáme základnú myšlienku, že pravidlá gramatiky možno vyjadriť ako obmedzenia nad premennými reprezentujúcimi odvodenie intervalov vstupného slova. V tejto práci túto myšlienku konkretizujeme pre vlastnú implementáciu, ktorá používa gramatiku v CHNF, pomocné Tseitinove premenné a výstup vo formáte DIMACS.

Nech $G = \langle N, \Sigma, P, S \rangle$ je bezkontextová gramatika v CHNF a $w = w_0 w_1 \dots w_{n-1}$ vstupný reťazec. Výsledná CNF formula je množinou klauzúl, ktoré obmedzujú možné ohodnotenia premenných tak, aby zodpovedali platnej derivácii.

Konštrukciu klauzúl rozdelíme do viacerých skupín podľa ich významu.

6.3.1 Klauzuly pre fixné pravdivostné hodnoty

Tieto klauzuly predstavujú implementačné zapracovanie počiatočných a terminálnych podmienok do SAT modelu. V našom riešení ich zapisujeme explicitne ako jednotkové klauzuly, čím sa časť pravdivostných hodnôt fixuje ešte pred samotným riešením formule.

Cieľová podmienka

Premenná reprezentujúca generovanie celého reťazca štartovacím symbolom $X_{S,0,n-1}$ je nastavená na pravdu, čo v CNF zapisujeme ako jednotkovú klauzulu:

$$\left(X_{S,0,n-1}\right)$$

Táto klauzula zabezpečuje, že hľadáme také ohodnotenie, v ktorom štartovací symbol generuje celý vstup.

Terminálne pravidlá

Pre každé pravidlo tvaru $A \to a$ a pre každú pozíciu i, kde $w_i = a$, nastavujeme: $X_{A,i,i} =$ true čo reprezentujeme klauzulou:

$$\left(X_{A,i,i}\right)$$

Naopak, ak neexistuje terminálne pravidlo, ktoré by generovalo symbol w_i, príslušné premenné nastavujeme na false:

$$\left(\neg X_{A,i,i}\right)$$

Týmto spôsobom zakódujeme základnú kompatibilitu medzi gramatikou a vstupným reťazcom.

Neplatné premenné

Ak pre neterminál A neexistuje žiadne pravidlo tvaru $A \to BC$, potom pre všetky intervaly dĺžky väčšej než jeden symbol platí: $X_{A,i,j} =$ false, $i < j$, čo zapisujeme ako:

$$\left(\neg X_{A,i,j}\right)$$

Tým eliminujeme nemožné derivácie už na úrovni základných premenných.

6.3.2 Klauzuly pre binárne pravidlá

Prevod binárnych pravidiel $A \rightarrow BC$ na obmedzenia nad intervalmi vychádza z myšlienky použitej v [1], podľa ktorej odvodenie väčšieho intervalu možno vyjadriť existenciou deliaceho bodu a odvodením dvoch menších intervalov. Pre každú trojicu indexov i, k, j, kde $i \leq k < j$, zavádzame pomocnú premennú $Z_{A \rightarrow BC,i,k,j}$ ktorá reprezentuje tvrdenie:

$$Z_{A \rightarrow BC,i,k,j} \leftrightarrow (X_{B,i,k} \wedge X_{C,k+1,j})$$

Túto ekvivalenciu prevádzame do CNF pomocou troch klauzúl:

$$\begin{gathered}(\neg Z_{A,i,k,j} \vee X_{B,i,k}) \\ (\neg Z_{A,i,k,j} \vee X_{C,k+1,j}) \\ (\neg X_{B,i,k} \vee \neg X_{C,k+1,j} \vee Z_{A,i,k,j})\end{gathered}$$

Tieto klauzuly zabezpečujú, že pomocná premenná Z je pravdivá práve vtedy, keď sú pravdivé obe zodpovedajúce podpremenné.

6.3.3 Väzba medzi premennými X a Z

Pre každú premennú $X_{A,i,j}$, kde $i < j$, vytvárame klauzulu, ktorá zabezpečuje, že ak je $X_{A,i,j}$ pravdivá, musí existovať aspoň jedna validná derivácia pomocou pravidiel:

$$X_{A,i,j} \rightarrow \bigvee_{(A \rightarrow BC) \in P} \bigvee_{k=i}^{j-1} Z_{A \rightarrow BC,i,k,j}$$

V CNF tvare túto implikáciu zapisujeme ako:

$$(\neg X_{A,i,j} \vee Z_1 \vee Z_2 \vee \cdots \vee Z_m)$$

kde $Z_1, \ldots, Z_m$ sú všetky pomocné premenné zodpovedajúce rôznym pravidlám a rozdeleniam intervalu.

Ak pre daný neterminál a interval neexistuje žiadna možnosť derivácie, pridávame klauzulu:

$$(\neg X_{A,i,j})$$

čo zabezpečuje, že takáto premenná nemôže byť pravdivá.

6.3.4 Charakter výslednej formule

Výsledná CNF formula je teda tvorená tromi hlavnými typmi klauzúl:

1. **jednotkové klauzuly**
   - fixujú pravdivostné hodnoty premenných,
2. **lokálne klauzuly pre pomocné premenné**
   - zabezpečujú korektnú reprezentáciu pravidiel $A \rightarrow BC$,
3. **globálne väzbové klauzuly**
   - prepájajú premenné $X_{A,i,j}$ s možnými deriváciami.

Takto konštruovaná formula presne reprezentuje parsovací problém. Každé jej splniteľné ohodnotenie zodpovedá platnej derivácii vstupného reťazca podľa danej gramatiky.

6.3.5 Prevod do DIMACS formátu

Po vygenerovaní všetkých klauzúl algoritmus prevedie formulu do štandardného DIMACS formátu, ktorý je bežne používaný SAT riešičmi. Každá premenná je reprezentovaná jedinečným číselným identifikátorom a každá klauzula je zapísaná ako disjunkcia literálov.

Tento krok umožňuje použiť rôzne SAT riešiče bez nutnosti meniť samotnú formuláciu problému. Rovnaká množina klauzúl môže byť tak použitá na experimentálne porovnanie výkonnosti rôznych riešičov, čo je jedným z cieľov tejto práce.

6.4 Korektnosť navrhovaného algoritmu

V tejto časti analyzujeme korektnosť navrhovaného algoritmu, ktorý redukuje parsovací problém pre bezkontextové gramatiky na problém splniteľnosti booleovskej formuly. Cieľom je ukázať, že navrhnutý model je správny v tom zmysle, že presne zachytáva vzťah medzi deriváciami v gramatike a splniteľnosťou výslednej SAT formule.

Korektnosť algoritmu dokazujeme pomocou dvoch základných vlastností: **soundness** a **completeness**.

6.4.1 Soundness

Vlastnosť soundness znamená, že každé splniteľné ohodnotenie výslednej SAT formule zodpovedá platnej derivácii v gramatike. Formálne:

Ak je SAT formula splniteľná, potom $w \in L(G)$.

Predpokladajme, že existuje ohodnotenie premenných, ktoré spĺňa všetky klauzuly vytvorené algoritmom. Ukážeme, že z tohto ohodnotenia možno zostrojiť derivačný strom pre vstupný reťazec.

Z definície modelu vieme, že premenná $X_{A,i,j}$ je pravdivá práve vtedy, keď neterminál A generuje podreťazec $w_i \dots w_j$. Keďže klauzula $(X_{S,0,n-1})$ je vždy súčasťou formule, musí platiť, že $X_{S,0,n-1} = \text{true}$. To znamená, že štartovací symbol generuje celý vstupný reťazec.

Ďalej, pre každú pravdivú premennú $X_{A,i,j}$ s $i < j$ existuje podľa konštrukcie klauzúl aspoň jedna pomocná premenná $Z_{A\to BC,i,k,j}$, ktorá je pravdivá a spĺňa podmienku:

$$Z_{A\to BC,i,k,j} \Rightarrow (X_{B,i,k} \wedge X_{C,k+1,j}).$$

Z toho vyplýva, že podreťazec $w_i \dots w_j$možno rozdeliť na dve časti, ktoré sú generované neterminálmi B a C. Tento proces môžeme rekurzívne aplikovať, až kým nedosiahneme podreťazce dĺžky jedna, kde sa použijú terminálne pravidlá.

Pre intervaly dĺžky jedna sú hodnoty premenných $X_{A,i,i}$ fixované podľa terminálnych pravidiel, čo zabezpečuje, že listy výsledného stromu zodpovedajú symbolom vstupného reťazca.

Týmto spôsobom možno z pravdivého ohodnotenia všetkých premenných postupne zostrojiť derivačný strom, ktorý reprezentuje platnú deriváciu reťazca w zo štartovacieho symbolu S. Z toho vyplýva, že reťazec patrí do jazyka generovaného gramatikou.

6.4.2 Completeness

Vlastnosť completeness znamená, že každá platná derivácia v gramatike vedie k splniteľnej SAT formule. Formálne:

Ak $w \in L(G)$, *potom je SAT formula splniteľná.*

Predpokladajme, že existuje derivačný strom pre reťazec w podľa gramatiky G. Ukážeme, že na základe tohto stromu možno zostrojiť také ohodnotenie premenných, ktoré spĺňa všetky klauzuly formule.

Pre každý uzol derivačného stromu, ktorý reprezentuje, že neterminál A generuje podreťazec $w_i \dots w_j$, nastavíme premennú: $X_{A,i,j} = \text{true}$. Pre všetky ostatné premenné môžeme nastaviť hodnotu false.

Ďalej, pre každé použitie pravidla $A \to BC$ v derivačnom strome, ktoré rozdeľuje interval (i, j) v bode k, nastavíme príslušnú pomocnú premennú: $Z_{A\to BC,i,k,j} = \text{true}$. Z konštrukcie derivačného stromu vyplýva, že pre každé takéto nastavenie platí: $X_{B,i,k} = \text{true}, X_{C,k+1,j} = \text{true}$, a teda sú splnené aj klauzuly:

$$(\neg Z_{A,i,k,j} \vee X_{B,i,k}), (\neg Z_{A,i,k,j} \vee X_{C,k+1,j}),$$
$$(\neg X_{B,i,k} \vee \neg X_{C,k+1,j} \vee Z_{A,i,k,j}).$$

Zároveň, keďže pre každú pravdivú premennú $X_{A,i,j}$ existuje v derivačnom strome konkrétny spôsob jej odvodenia, bude splnená aj klauzula:

$$(\neg X_{A,i,j} \vee Z_1 \vee \cdots \vee Z_m),$$

pretože aspoň jedna z premenných Z bude nastavená na true.

Terminálne klauzuly sú splnené, pretože derivačný strom rešpektuje terminálne pravidlá, a cieľová klauzula je splnená, keďže koreň stromu je označený symbolom S.

Z toho vyplýva, že existuje ohodnotenie premenných, ktoré spĺňa všetky klauzuly, a teda SAT formula je splniteľná.

Zhrnutie

Na základe vyššie uvedených argumentov môžeme konštatovať, že navrhovaný algoritmus je korektný. Platí, že splniteľnosť výslednej SAT formule je ekvivalentná existencii derivačného stromu pre vstupný reťazec.

Formálne:

$$\textit{SAT formula je splniteľná} \iff w \in L(G).$$

Táto vlastnosť zaručuje, že navrhovaná redukcia parsovacieho problému na SAT-problém je správna a že výsledky získané pomocou SAT riešiča presne zodpovedajú výsledkom klasických parsovacích algoritmov.

6.5 Časová a priestorová zložitosť navrhovaného algoritmu

V tejto časti analyzujeme časovú a priestorovú zložitosť navrhovaného algoritmu, ktorý redukuje parsovací problém pre bezkontextové gramatiky na problém splniteľnosti booleovskej formule. Analýza sa zameriava najmä na počet premenných a klauzúl generovaných počas konštrukcie CNF formule, keďže tieto veličiny priamo ovplyvňujú náročnosť následného riešenia pomocou SAT riešiča.

Nech $G = \langle N, \Sigma, P, S \rangle$ je bezkontextová gramatika v Chomského normálnej forme a nech vstupné slovo má dĺžku n.

6.5.1 Počet premenných

Počet základných premenných

Základné premenné modelu majú tvar:

$$X_{A,i,j}, A \in N,\ 0 \leq i \leq j < n.$$

Počet všetkých intervalov (i, j) v reťazci dĺžky n je:

$$\frac{n(n+1)}{2} = O(n^2).$$

Pre každý neterminál $A \in N$ definujeme premennú pre každý takýto interval, preto celkový počet základných premenných je:

$$O(|\ N\ | \cdot n^2).$$

Tieto premenné reprezentujú všetky možné kombinácie neterminálov a podreťazcov, ktoré môžu potenciálne vzniknúť počas derivácie.

Počet pomocných premenných

Pri spracovaní binárnych pravidiel tvaru $A \rightarrow BC$ zavádzame pomocné premenné:

$$Z_{A \rightarrow BC, i, k, j},$$

kde $i \leq k < j$. Pre každý interval (i, j) existuje najviac $j - i$ možných hodnôt indexu k, čo je v najhoršom prípade $O(n)$. Keďže počet intervalov je $O(n^2)$, celkový počet trojíc (i, k, j) je $O(n^3)$.

Pre každé binárne pravidlo vytvárame takúto množinu pomocných premenných, preto celkový počet pomocných premenných je:

$$O(|\ P_{bin}\ | \cdot n^3),$$

kde $|\ P_{bin}\ |$ označuje počet pravidiel typu $A \rightarrow BC$.

6.5.2 Počet klauzúl

Klauzuly vznikajú v niekoľkých krokoch, pričom každý z nich prispieva k celkovej zložitosti.

Jednotkové klauzuly

Pre fixné hodnoty premenných (terminálne pravidlá, štartovací symbol, nemožné prípady) generujeme jednotkové klauzuly. Ich počet je najviac úmerný počtu základných premenných:

$$O(|N| \cdot n^2).$$

Klauzuly pre pomocné premenné

Každá pomocná premenná $Z_{A \to BC,i,k,j}$ je viazaná na premenné $X_{B,i,k}$ a $X_{C,k+1,j}$ pomocou troch klauzúl:

$$(\neg Z \vee X_{B,i,k}), (\neg Z \vee X_{C,k+1,j}), (\neg X_{B,i,k} \vee \neg X_{C,k+1,j} \vee Z).$$

Pre každú pomocnú premennú teda vznikajú konštantne mnohé klauzuly, čo znamená, že počet týchto klauzúl je:

$$O(|P_{bin}| \cdot n^3).$$

Väzbové klauzuly

Pre každú premennú $X_{A,i,j}$ s $i < j$ generujeme klauzulu:

$$(\neg X_{A,i,j} \vee Z_1 \vee Z_2 \vee \cdots \vee Z_m),$$

kde $Z_1, \ldots, Z_m$ reprezentujú všetky možné derivácie pomocou pravidiel a rozdelení intervalu.

Počet takýchto klauzúl je úmerný počtu premenných $X_{A,i,j}$, teda:

$$O(|N| \cdot n^2),$$

pričom dĺžka jednotlivých klauzúl môže byť až $O(n)$, keďže zahŕňa všetky možné hodnoty k.

6.5.3 Celková zložitosť

Z vyššie uvedenej analýzy vyplýva, že dominantným faktorom z hľadiska počtu premenných aj klauzúl je spracovanie binárnych pravidiel a všetkých možných rozdelení intervalov.

Celkový počet premenných je:

$$O(|N| \cdot n^2 + |P_{bin}| \cdot n^3),$$

a celkový počet klauzúl je:

$$O(|P_{bin}| \cdot n^3).$$

Z toho vyplýva, že pri fixnej gramatike (teda konštantnom $|N|$a $|P|$) je asymptotická zložitosť modelu vzhľadom na dĺžku vstupu:

$$O(n^3).$$

6.5.4 Interpretácia výsledkov

Kubická závislosť od dĺžky vstupného reťazca vyplýva z kombinácie dvoch faktorov:

1. počtu všetkých podreťazcov vstupu ($O(n^2)$),
2. počtu možných rozdelení každého podreťazca ($O(n)$).

Táto štruktúra je analogická klasickému algoritmu CYK, ktorý taktiež pracuje s trojicami indexov (i, k, j) a má časovú zložitosť $O(n^3)$.

Na rozdiel od CYK algoritmu však navrhovaný prístup presúva výpočtovú náročnosť do konštrukcie logickej formule a následného riešenia pomocou SAT riešiča. Výsledná zložitosť preto nezávisí iba od veľkosti formule, ale aj od použitých heuristík a konkrétnej implementácie SAT riešiča.

6.5.5 Diskusia

Navrhovaný algoritmus má z teoretického hľadiska rovnakú asymptotickú zložitosť ako klasické parsovacie algoritmy pre všeobecné bezkontextové gramatiky. Jeho hlavná výhoda spočíva v tom, že umožňuje využiť optimalizované SAT riešiče a flexibilne rozširovať model o ďalšie obmedzenia.

Nevýhodou je potenciálne veľký počet premenných a klauzúl, najmä pri dlhších vstupoch alebo gramatikách s veľkým počtom pravidiel. Táto vlastnosť môže viesť k zvýšenej pamäťovej aj časovej náročnosti, čo je potrebné zohľadniť pri praktickom nasadení.

7 Návrh softvéru

V tejto časti sa zameriavame na návrh implementácie systému, ktorý realizuje redukciu parsovacieho problému pre bezkontextové gramatiky na SAT problém. Cieľom tejto časti nie je opakovať teoretický opis algoritmu, ale ukázať, ako je navrhnutý algoritmus premietnutý do konkrétneho softvérového riešenia. Opisujeme základnú architektúru systému, tok spracovania dát, hlavné komponenty implementácie a ich vzájomné prepojenie.

Implementované riešenie je navrhnuté modulárne. Jednotlivé časti systému vykonávajú samostatné úlohy, ako je načítanie vstupnej gramatiky, jej transformácia do Chomského normálnej formy (CHNF), generovanie booleovských premenných, tvorba klauzúl v konjunktívnej normálnej forme (CNF), export výslednej formule do DIMACS formátu, spustenie SAT riešiča a interpretácia výsledku. Takéto rozdelenie umožňuje jednoduchšiu údržbu systému, samostatné testovanie jednotlivých komponentov a neskoršie rozšírenie implementácie napríklad o ďalšie SAT riešiče alebo nové typy experimentov.

7.1 Funkcionálne požiadavky na systém

Načítanie gramatiky zo súboru – Systém musí umožniť načítanie bezkontextovej gramatiky z textového súboru. Súbor obsahuje definíciu terminálov, neterminálov, počiatočného symbolu a pravidiel.

Spracovanie viacerých gramatík v jednom súbore – Systém musí podporovať vstupný súbor obsahujúci viacero gramatík oddelených samostatnými blokmi.

Syntaktická kontrola vstupu – Systém musí overiť, či vstupná gramatika obsahuje požadované sekcie a či sú pravidlá zapísané v očakávanom formáte.

Overenie symbolov v pravidlách – Systém musí skontrolovať, či symboly použité v pravidlách patria medzi deklarované terminály alebo neterminály.

Transformácia do CHNF – Systém musí transformovať načítanú gramatiku do Chomského normálnej formy, aby bolo možné následne generovať SAT model.

Spracovanie vstupného slova – Systém musí prijať vstupné slovo a rozdeliť ho na jednotlivé tokeny.

Ošetrenie prázdneho vstupného slova – Ak je vstupné slovo prázdne, systém musí rozhodnúť výsledok podľa toho, či gramatika prijíma prázdny reťazec.

Generovanie booleovských premenných – Systém musí vytvoriť premenné $X_{A,i,j}$, ktoré reprezentujú tvrdenie, že neterminál A generuje podreťazec od pozície i po pozíciu j.

Generovanie CNF klauzúl – Systém musí vytvoriť klauzuly v konjunktívnej normálnej forme, ktoré reprezentujú pravidlá gramatiky a podmienky platnej derivácie.

Export do DIMACS formátu – Systém musí uložiť výslednú CNF formulu do štandardného DIMACS formátu, aby ju bolo možné spracovať externými SAT riešičmi.

Riešenie pomocou SAT riešiča – Systém musí umožniť odovzdanie CNF formule SAT riešiču a spracovanie výsledku splniteľnosti.

Rekonštrukcia derivačného stromu – V prípade splniteľnej formule musí systém umožniť spätnú rekonštrukciu derivačného stromu z ohodnotenia premenných.

7.2 Nefunkcionálne požiadavky na systém

Modularita – Systém musí byť rozdelený na samostatné moduly, napríklad parser, transformáciu do CHNF, generovanie SAT modelu, export DIMACS a benchmarking.

Rozšíriteľnosť – Implementácia musí umožňovať jednoduché doplnenie ďalších SAT riešičov.

Reprodukovateľnosť – Rovnaký vstup musí viesť k rovnakej CNF formule a rovnakému DIMACS výstupu.

Kompatibilita so štandardmi – Systém musí používať štandardné výstupné formáty, najmä DIMACS pre SAT formuly a CSV pre experimentálne výsledky.

Efektívne spracovanie väčších vstupov – Systém musí byť schopný pracovať s gramatikami a vstupnými slovami rôznej veľkosti v rámci dostupných výpočtových zdrojov.

Zrozumiteľné chybové hlásenia – Pri nesprávnom vstupe musí systém poskytnúť informáciu o príčine chyby, napríklad chýbajúca sekcia alebo nesprávne zapísané pravidlo.

Platformová prenositeľnosť – Keďže implementácia je vytvorená v jazyku Python, systém musí byť spustiteľný na bežných operačných systémoch po nainštalovaní potrebných závislostí.

Automatizovateľnosť experimentov – Systém musí umožňovať spustenie viacerých benchmarkov bez nutnosti manuálneho zásahu do každého testovacieho prípadu.

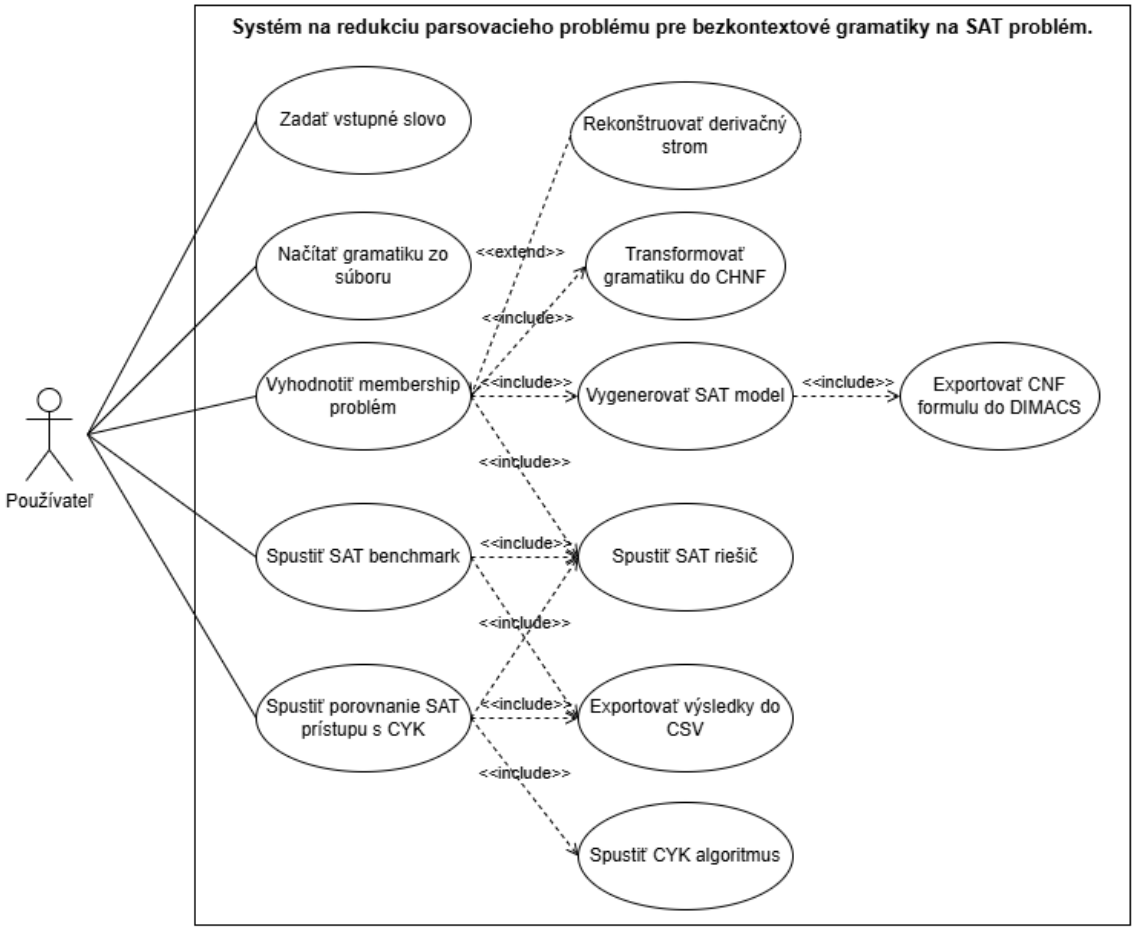
Prehľadnosť výstupov – Výstupné súbory musia byť organizované tak, aby bolo možné jednoducho identifikovať použitú gramatiku, vstupné slovo a výsledky experimentov.

Možnosť ďalšej analýzy výsledkov – Uložené výsledky musia byť vo formáte vhodnom na ďalšie spracovanie, vizualizáciu a štatistické vyhodnotenie.

7.3 Diagram použitia

Use-case diagram slúži na znázornenie hlavných spôsobov použitia navrhovaného systému z pohľadu používateľa. V našom prípade je hlavným aktérom používateľ, ktorý pracuje so systémom buď v režime jednorazového riešenia parsovacieho problému, alebo v režime benchmarkingu. Diagram zachytáva základné funkcionality systému, ako je načítanie

gramatiky, zadanie vstupného slova, vyhodnotenie príslušnosti slova do jazyka, generovanie SAT modelu, export formule do DIMACS formátu a spustenie experimentálneho vyhodnotenia.



Obrázok 7-1 Use-case diagram navrhovaného systému.

Základným prípadom použitia je vyhodnotenie príslušnosti vstupného slova do jazyka generovaného gramatikou. Tento scenár zahŕňa načítanie gramatiky zo súboru, transformáciu gramatiky do Chomského normálnej formy (CHNF), zadanie vstupného slova, vytvorenie SAT modelu a následné riešenie vzniknutej CNF formule pomocou SAT riešiča. Výsledkom je informácia o tom, či vstupné slovo patrí alebo nepatrí do jazyka danej gramatiky.

Súčasťou základného scenára je aj možnosť exportovať vytvorenú CNF formulu do DIMACS formátu. Tento export je dôležitý najmä preto, že umožňuje oddeliť fázu generovania SAT modelu od fázy jeho riešenia. Vďaka tomu je možné rovnakú formulu neskôr spracovať rôznymi SAT riešičmi, čo sa využíva najmä v experimentálnej časti práce.

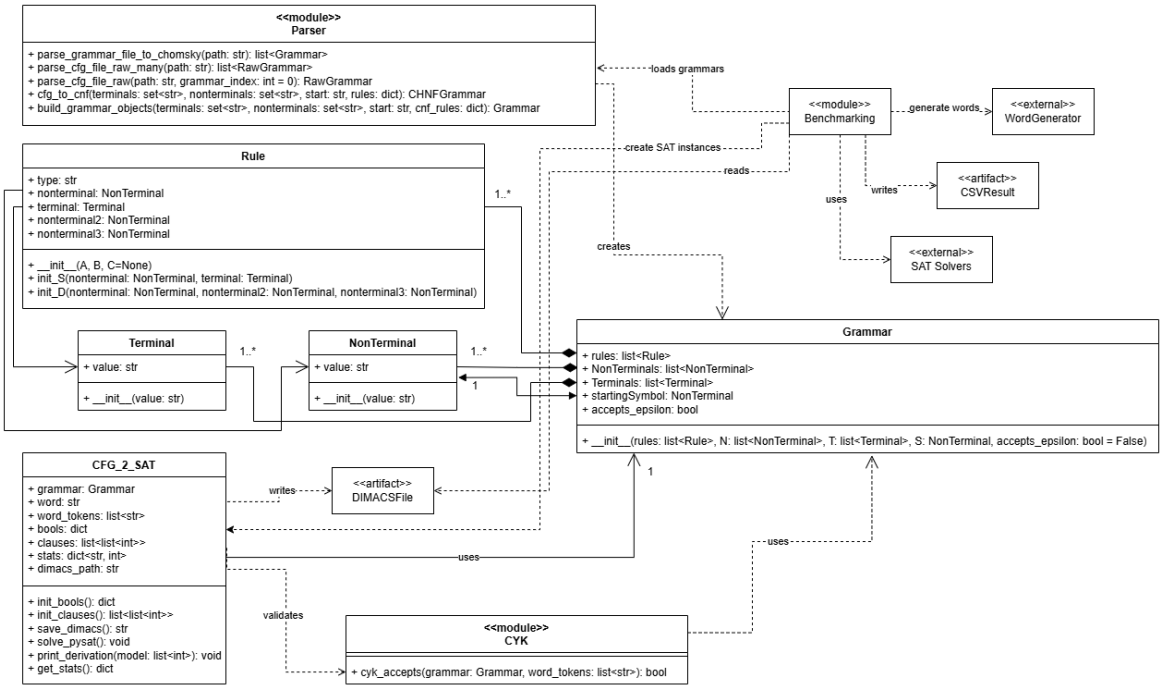
Rozšírením základného scenára je rekonštrukcia derivačného stromu. Táto funkcionalita je dostupná iba v prípade, že SAT riešič označí formulu ako splniteľnú. V takom prípade

systém z pravdivostného ohodnotenia premenných spätne rekonštruuje derivačný strom, ktorý reprezentuje konkrétnu deriváciu vstupného slova podľa danej gramatiky.

Okrem jednorazového riešenia systém podporuje aj experimentálne použitie. Používateľ môže spustiť SAT benchmark, ktorého cieľom je porovnať viacero SAT riešičov nad rovnakými DIMACS formulami. Ďalším podporovaným scenárom je porovnanie navrhovaného SAT prístupu s klasickým algoritmom CYK. V týchto prípadoch systém automatizovane spracúva väčšie množstvo vstupných slov, zaznamenáva výsledky a ukladá ich do CSV súborov na ďalšiu analýzu.

7.4 Diagram tried

Triedny diagram slúži na znázornenie hlavných tried, modulov a dátových štruktúr, ktoré tvoria implementované riešenie. Diagram nezobrazuje všetky pomocné funkcie a interné premenné systému, ale sústreďuje sa na najdôležitejšie komponenty, ich zodpovednosti a vzájomné vzťahy.



Obrázok 7-2 Triedny diagram hlavných komponentov systému.

Základ internej reprezentácie gramatiky tvoria triedy `Terminal`, `NonTerminal`, `Rule` a `Grammar`. Trieda `Terminal` reprezentuje terminálny symbol a trieda `NonTerminal` reprezentuje neterminálny symbol. Obidve triedy obsahujú atribút `value`, ktorý uchováva textovú hodnotu daného symbolu.

Trieda `Rule` reprezentuje jedno pravidlo gramatiky v CHNF. V implementácii rozlišujeme dva typy pravidiel. Prvý typ je terminálne pravidlo tvaru $A \to a$, kde ľavá strana je neterminál a pravá strana je terminál. Druhý typ je binárne pravidlo tvaru $A \to BC$, kde pravá strana pozostáva z dvoch neterminálov. Tento spôsob reprezentácie zodpovedá požiadavkám algoritmu, ktorý pracuje s gramatikou prevedenou do Chomského normálnej formy.

Trieda `Grammar` predstavuje kompletnú gramatiku. Obsahuje zoznam pravidiel, množinu neterminálov, množinu terminálov, počiatočný symbol a informáciu o tom, či gramatika prijíma prázdny reťazec. Trieda `Grammar` je preto centrálnou dátovou štruktúrou, ktorú využívajú ďalšie časti systému.

Modul `Parser` zabezpečuje načítanie vstupnej gramatiky zo súboru, jej syntaktické spracovanie, transformáciu do CHNF a vytvorenie objektu typu `Grammar`. Parser teda predstavuje vstupnú vrstvu systému, ktorá prevádza textový zápis gramatiky na internú objektovú reprezentáciu.

Centrálnou triedou systému je `CFG_2_SAT`. Táto trieda prijíma objekt `Grammar` a vstupné slovo. Na ich základe vytvára booleovské premenné, generuje klauzuly CNF formule a v prípade potreby exportuje výslednú formulu do súboru vo formáte DIMACS. Trieda zároveň uchováva štatistické údaje o počte premenných, klauzúl a pravidiel, ktoré sú následne využité pri experimentálnom vyhodnotení.

Modul `CYK` obsahuje referenčnú implementáciu klasického CYK algoritmu. V systéme sa používa najmä na overenie správnosti výsledku SAT prístupu a na experimentálne porovnanie výkonnosti oboch metód. Keďže modul pracuje s objektom `Grammar`, je v diagrame znázornená jeho závislosť od tejto triedy.

Modul `Benchmarking` predstavuje experimentálnu vrstvu systému. Jeho úlohou je automatizovať generovanie testovacích vstupov, vytváranie SAT inštancií, spúšťanie SAT riešičov a ukladanie výsledkov do CSV súborov. Benchmarking využíva parser na načítanie gramatík, triedu `CFG_2_SAT` na vytváranie SAT modelov a modul `CYK` pri porovnaní membership problému.

V diagrame sú znázornené aj dva typy pomocných artefaktov. `DIMACSFile` reprezentuje vygenerovaný súbor obsahujúci CNF formulu vo formáte DIMACS. Tento súbor vytvára trieda `CFG_2_SAT` a následne ho môže využívať benchmarkovacia vrstva. `CSVResult` reprezentuje výstupný súbor s výsledkami experimentov, napríklad časmi riešenia, počtom premenných a počtom klauzúl.

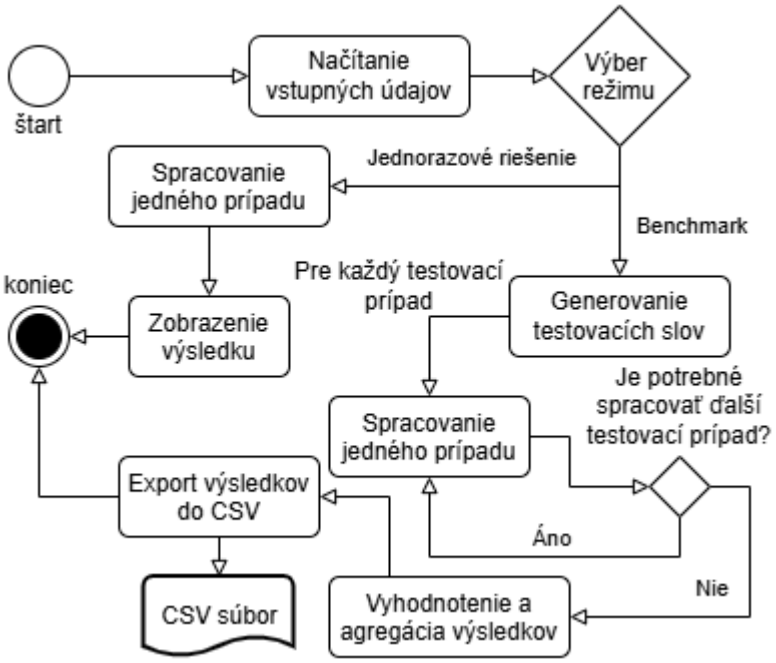
Externými komponentmi systému sú `WordGenerator` a `SAT Solvers`. `WordGenerator` slúži na generovanie testovacích slov pre benchmarky a ide o externý nástroj vytvorený mimo tejto práce [14]. Komponent `SAT Solvers` reprezentuje použité SAT riešiče, ktoré spracúvajú vygenerované CNF formuly.

Triedny diagram tak ukazuje, že implementácia je rozdelená na viacero samostatných častí. Parser zabezpečuje vstup a transformáciu gramatiky, `CFG_2_SAT` realizuje samotnú redukciu na SAT problém, `CYK` slúži ako referenčný algoritmus a `Benchmarking` zabezpečuje experimentálne vyhodnotenie. Takéto rozdelenie zvyšuje prehľadnosť systému a umožňuje jednoduchšie rozšírenie implementácie v budúcnosti.

7.5 Vývojové diagramy systému

Vývojové diagramy dopĺňajú triedny a use-case diagram o pohľad na dynamické správanie systému. Zatiaľ čo triedny diagram opisuje štruktúru tried, modulov a ich vzťahov, vývojový diagram znázorňuje postupnosť krokov, ktoré systém vykonáva pri spracovaní vstupu. V práci používame dva vývojové diagramy. Prvý diagram opisuje hlavný priebeh systému a druhý diagram detailnejšie zachytáva spracovanie jedného konkrétneho prípadu.

Hlavný vývojový diagram zobrazuje dva základné režimy práce systému. Prvým režimom je jednorazové riešenie, pri ktorom používateľ zadá jednu gramatiku a jedno vstupné slovo. Druhým režimom je benchmark, ktorý opakovane využíva rovnaký proces spracovania jedného prípadu nad väčším počtom testovacích vstupov.



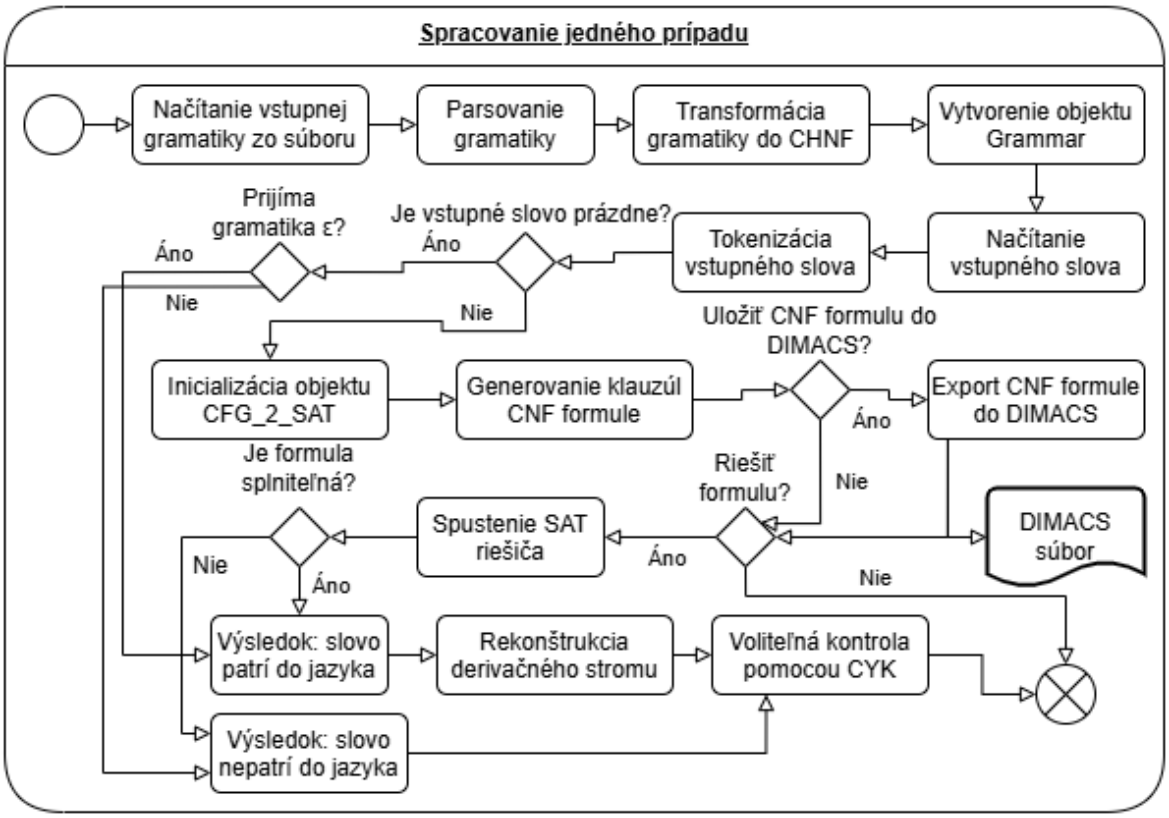
Obrázok 7-3 Hlavný vývojový diagram systému.

V režime jednorazového riešenia systém načíta vstupné údaje, používateľ zadá konkrétne vstupné slovo a systém spustí spracovanie jedného prípadu. Výsledkom je informácia o tom,

či dané slovo patrí do jazyka generovaného gramatikou. Ak je výsledná formula splniteľná, systém môže navyše rekonštruovať derivačný strom.

V benchmarkovacom režime systém najprv pripraví množinu testovacích vstupov [14]. Následne sa pre každý testovací prípad spúšťa rovnaký proces spracovania jedného prípadu ako pri jednorazovom riešení. Po spracovaní všetkých vstupov systém agreguje výsledky a uloží ich do CSV súboru. Tento režim umožňuje porovnávať rôzne SAT riešiče alebo porovnať navrhovaný SAT prístup s algoritmom CYK.

Druhý vývojový diagram detailnejšie opisuje samotný proces spracovania jedného prípadu. Tento proces predstavuje jadro systému a využíva sa v oboch režimoch, teda pri jednorazovom riešení aj pri benchmarkingu. Vstupom je gramatika a vstupné slovo. Gramatika je najprv načítaná, spracovaná parserom a transformovaná do Chomského normálnej formy (CHNF). Následne sa vytvorí interný objekt Grammar, ktorý slúži ako vstup pre triedu `CFG_2_SAT`.



Obrázok 7-4 Detailný vývojový diagram spracovania jedného prípadu.

Po načítaní vstupného slova systém vykoná jeho tokenizáciu. Ak je vstupné slovo prázdne, nie je potrebné generovať SAT formulu. V takom prípade systém rozhodne výsledok priamo na základe toho, či gramatika prijíma prázdny reťazec. Ak gramatika prijíma ε, slovo patrí do jazyka. V opačnom prípade slovo do jazyka nepatrí.

Ak vstupné slovo nie je prázdne, systém inicializuje objekt `CFG_2_SAT`. V tejto fáze sa vytvoria premenné $X_{A,i,j}$, ktoré reprezentujú tvrdenie, že neterminál A generuje podreťazec od pozície i po pozíciu j. Následne sa vytvoria klauzuly výslednej CNF formule.

Po zostavení CNF formule môže systém formulu uložiť do DIMACS formátu. Tento krok je dôležitý najmä pri benchmarkingu, pretože DIMACS umožňuje spúšťať rôzne SAT riešiče nad rovnakou formulou. Následne môže byť formula odovzdaná SAT riešiču. Ak je formula splniteľná, systém vyhodnotí, že vstupné slovo patrí do jazyka gramatiky, a môže rekonštruovať derivačný strom. Ak formula splniteľná nie je, systém vyhodnotí, že vstupné slovo nie je gramatikou generované.

Súčasťou spracovania môže byť aj voliteľná kontrola pomocou algoritmu CYK. Táto kontrola slúži najmä na overenie správnosti implementácie a porovnanie výsledkov navrhovaného SAT prístupu s klasickým parsovacím algoritmom. Vývojové diagramy tak ukazujú nielen samotný postup výpočtu, ale aj spôsob, akým sú jednotlivé časti systému prepojené v rámci jednorazového riešenia a experimentálneho vyhodnotenia.

8 Implementácia

V tejto kapitole sa zameriavame na praktickú implementáciu navrhovaného algoritmu, ktorý redukuje parsovací problém pre bezkontextové gramatiky na problém splniteľnosti booleovskej formule. Cieľom tejto časti je opísať architektúru riešenia, jednotlivé komponenty systému a spôsob, akým je realizovaná transformácia vstupného problému až po jeho riešenie pomocou SAT riešiča.

Implementácia vychádza z modulárneho prístupu, v ktorom jednotlivé fázy spracovania tvoria logický reťazec operácií. Celý proces začína načítaním vstupnej gramatiky a jej spracovaním pomocou parsera, ktorý zabezpečuje transformáciu gramatiky do Chomského normálnej formy (CHNF). Tento krok je nevyhnutný pre následnú systematickú konštrukciu logických obmedzení. Po prevode gramatiky do CHNF nasleduje fáza generovania SAT modelu. V tejto fáze vytvárame množinu booleovských premenných reprezentujúcich vzťahy medzi neterminálnymi symbolmi a podreťazcami vstupného slova. Na základe produkčných pravidiel následne generujeme klauzuly v konjunktívnej normálnej forme (CNF), ktoré spolu tvoria výslednú logickú formulu.

Takto vytvorená CNF formula je následne exportovaná do štandardného DIMACS formátu, ktorý umožňuje jej spracovanie rôznymi SAT riešičmi. Po vyriešení formule SAT riešičom získavame ohodnotenie premenných, ktoré interpretujeme ako riešenie pôvodného parsovacieho problému. V prípade splniteľnej formule je možné z tohto ohodnotenia spätne rekonštruovať derivačný strom pre vstupný reťazec.

Celý proces možno schematicky vyjadriť nasledovne:

vstupná gramatika + reťazec → parser → CHNF → SAT model
→ CNF (DIMACS) → SAT riešič → výsledok

Takto navrhnutá architektúra umožňuje jasné oddelenie jednotlivých častí systému a zároveň poskytuje flexibilitu pri experimentoch, najmä pri porovnávaní rôznych SAT riešičov nad rovnakou množinou generovaných formulí.

8.1 Interné objekty gramatiky

Pred samotným spracovaním vstupnej gramatiky je potrebné definovať internú objektovú reprezentáciu, s ktorou pracujú ďalšie časti systému. Táto reprezentácia je implementovaná v zdrojovom súbore:

src/domain/grammar_types.py

Cieľom tejto časti implementácie je previesť textový zápis gramatiky na jednoduché dátové objekty, ktoré môžu byť ďalej použité parserom, modulom CFG_2_SAT, algoritmom CYK a benchmarkovacou časťou systému. Základnými objektmi sú triedy Terminal, NonTerminal, Rule a Grammar.

Trieda Terminal reprezentuje terminálny symbol gramatiky. Obsahuje jediný atribút value, ktorý uchováva textovú hodnotu terminálu. Vstupom konštruktora je parameter value: str a trieda nevracia žiadnu samostatnú hodnotu, ale vytvára objekt terminálneho symbolu.

```
class Terminal:
    def __init__(self, value: str):
        self.value = value
```

Trieda NonTerminal reprezentuje neterminálny symbol gramatiky. Rovnako ako trieda Terminal, aj táto trieda obsahuje atribút value, v ktorom je uložený názov neterminálu. Vstupom konštruktora je parameter value: str.

```
class NonTerminal:
    def __init__(self, value: str):
        self.value = value
```

Trieda Rule reprezentuje jedno pravidlo gramatiky v CHNF. Keďže po transformácii do CHNF pracujeme iba s pravidlami tvaru $A \to a$ alebo $A \to BC$, trieda Rule podporuje dva typy pravidiel. Ak konštruktor dostane dva argumenty, vytvorí terminálne pravidlo typu SINGULAR. Ak dostane tri argumenty, vytvorí binárne pravidlo typu DOUBLE.

```
class Rule:
    def __init__(self, A, B, C=None):
        if C is None:
            self.init_S(A, B)
        else:
            self.init_D(A, B, C)

    def init_S(self, nonterminal: NonTerminal, terminal: Terminal):
        self.type = 'SINGULAR'
        self.nonterminal = nonterminal
        self.terminal = terminal
```

```
    def init_D(self, nonterminal: NonTerminal,
               nonterminal2: NonTerminal,
               nonterminal3: NonTerminal):
        self.type = 'DOUBLE'
        self.nonterminal = nonterminal
        self.nonterminal2 = nonterminal2
        self.nonterminal3 = nonterminal3
```

Najvyššiu úroveň internej reprezentácie tvorí trieda Grammar. Tá reprezentuje celú gramatiku po transformácii do CHNF. Konštruktor triedy má tvar:

```
class Grammar:
    def __init__(self, rules: list[Rule], N: list[NonTerminal],
                 T: list[Terminal], S: NonTerminal,
                 accepts_epsilon: bool = False):
        self.rules = rules
        self.NonTerminals = N
        self.Terminals = T
        self.startingSymbol = S
        self.accepts_epsilon = accepts_epsilon
```

Vstupom konštruktora je zoznam pravidiel, zoznam neterminálov, zoznam terminálov, počiatočný symbol a hodnota určujúca, či gramatika prijíma prázdny reťazec. Trieda tieto údaje ukladá do atribútov `rules`, `NonTerminals`, `Terminals`, `startingSymbol` a `accepts_epsilon`.

Atribút `accepts_epsilon` má osobitný význam pri spracovaní prázdneho vstupného slova. Keďže SAT model v tejto implementácii nepracuje s prázdnymi intervalmi vstupu, prípad prázdneho slova sa rieši osobitne práve na základe tejto hodnoty.

Interná reprezentácia gramatiky tak poskytuje jednotný dátový model pre všetky ďalšie časti systému. Parser vytvára objekty typu `Grammar`, trieda `CFG_2_SAT` z nich generuje SAT model, algoritmus CYK ich používa ako referenčný vstup a benchmarkovacia časť systému ich využíva pri automatizovanom experimentálnom vyhodnotení.

8.2 Spracovanie vstupnej gramatiky

Prvým krokom implementácie je spracovanie vstupnej gramatiky zo súboru a jej transformácia do tvaru vhodného pre ďalšie spracovanie. Na tento účel používame samostatný parser, ktorého úlohou je načítať gramatiku zo zvoleného textového formátu, syntakticky ju

overiť, previesť ju do internej reprezentácie a následne transformovať do Chomského normálnej formy (CHNF).

Spracovanie vstupnej gramatiky je implementované v zdrojovom súbore

`src/modules/parser.py`

Hlavnou verejnou funkciou tohto modulu je funkcia:

```
def parse_grammar_file_to_chomsky(path: str) -> List[Grammar]:
```

Táto funkcia predstavuje vstupný bod parsera pre ostatné časti systému. Jej vstupným parametrom je `path`, teda cesta k textovému súboru obsahujúcemu jednu alebo viac bezkontextových gramatík. Návratovou hodnotou je zoznam objektov typu `Grammar`, pričom každý objekt reprezentuje jednu gramatiku transformovanú do CHNF a pripravenú na ďalšie spracovanie v module `CFG_2_SAT`.

Formát vstupu

Parser prijíma vstupnú gramatiku vo forme textového súboru rozdeleného do pomenovaných sekcií. Podporované sú nasledujúce sekcie:

- terminal:
- nonterminal:
- start:
- rules:

Formát jednej gramatiky má teda tvar:

```
# Grammar = aa(b,c)+ / in CNF

terminal:
a b c
nonterminal:
S A B C
rules:
S - A B
A - a
B - A C
C - C C
C - b
C - c
start:
S
```

Sekcia terminal obsahuje množinu terminálnych symbolov, sekcia nonterminal obsahuje množinu neterminálnych symbolov, sekcia start určuje štartovací symbol a sekcia rules obsahuje produkčné pravidlá vo forme $LHS - RHS$, kde LHS je neterminál na ľavej strane a RHS je pravá strana pravidla. Parser akceptuje aj viacero gramatík v jednom súbore, pričom jednotlivé bloky sú oddelené riadkom obsahujúcim symboly '---, .

Vyhľadanie vstupného súboru

Vyhľadanie vstupného súboru je realizované funkciou:

```
def _resolve_input_path(path: str) -> Path:
```

Vstupným parametrom tejto funkcie je `path`, teda cesta k vstupnému súboru. Funkcia najprv overí, či zadaná cesta existuje priamo. Ak súbor na tejto ceste nenájde, pokúsi sa ho vyhľadať v predvolenom adresári:

`src/text/input/`

Návratovou hodnotou funkcie je objekt typu `Path`, ktorý reprezentuje platnú cestu k vstupnému súboru. Ak súbor neexistuje ani na jednej z očakávaných ciest, funkcia vyvolá výnimku `FileNotFoundError`.

Tento mechanizmus umožňuje používať buď absolútnu alebo relatívnu cestu k súboru, prípadne iba názov súboru umiestneného v štandardnom vstupnom adresári.

Predspracovanie vstupu

Pred samotným parsovaním parser odstraňuje komentáre a ignoruje prázdne riadky. Komentár je chápaný ako text za znakom # na danom riadku. Túto úlohu realizuje pomocná funkcia:

```
def _strip_comment(line: str) -> str:
```

Vstupom funkcie je jeden riadok textu. Návratovou hodnotou je ten istý riadok bez komentára a bez nadbytočných medzier na začiatku a na konci. Tento prístup umožňuje zapisovať ku gramatikám pomocné poznámky bez vplyvu na samotné parsovanie.

Rozdelenie vstupného súboru na samostatné bloky gramatík je realizované funkciou:

```
def _split_grammar_blocks(text: str) -> List[str]:
```

Vstupom funkcie je celý obsah vstupného súboru ako textový reťazec. Funkcia hľadá oddeľovacie riadky obsahujúce `---` a podľa nich rozdelí vstup na samostatné bloky. Návratovou hodnotou je zoznam textových blokov, kde každý blok reprezentuje jednu gramatiku.

Táto funkcia zároveň ignoruje prázdne bloky, čo zabraňuje vzniku chýb v prípade nadbytočných oddeľovacích riadkov.

Čítanie sekcií gramatiky

Po rozdelení vstupu na jednotlivé bloky parser spracúva každú gramatiku samostatne. Rozpoznanie sekcií zabezpečuje funkcia:

```
def _read_sections(text: str) -> Dict[str, List[str]]:
```

Vstupom tejto funkcie je text jedného gramatického bloku. Funkcia postupne prechádza jednotlivé riadky, odstraňuje komentáre a rozpoznáva názvy sekcií. Podporované sú aj varianty názvov sekcií, napríklad `terminal` aj `terminals`, resp. `nonterminal` aj `nonterminals`. Návratovou hodnotou funkcie je slovník, ktorého kľúčmi sú názvy základných sekcií:

```
terminal
nonterminal
rules
start
```

Hodnotami sú zoznamy riadkov patriacich do jednotlivých sekcií. Ak sa v súbore nachádza neznáma sekcia, funkcia vyvolá výnimku `ValueError`. Rovnako vyvolá chybu aj vtedy, ak sa riadok nachádza mimo akejkoľvek sekcie. Týmto spôsobom parser zabezpečuje základnú syntaktickú kontrolu vstupu.

Spracovanie terminálnych a neterminálnych symbolov

Obsah sekcií terminal a nonterminal sa spracúva pomocou funkcie:

```
def _split_csv(line: str) -> List[str]:
```

Vstupom funkcie je jeden riadok obsahujúci zoznam symbolov. Funkcia podporuje symboly oddelené medzerami aj čiarkami. Osobitne ošetruje aj prípad, keď je samotná čiarka použitá ako terminálny symbol.

Návratovou hodnotou funkcie je zoznam symbolov vo forme textových reťazcov. Z týchto zoznamov parser následne vytvorí množiny terminálov a neterminálov.

Kontrola samotnej gramatiky prebieha vo funkcii:

```
def _parse_cfg_text_raw(text: str) -> Tuple
    Set[str],
    Set[str],
    str,
    Dict[str, List[List[str]]]
 ]:
```

Vstupom funkcie je text jedného gramatického bloku. Návratovou hodnotou je štvorica:

$$(T, N, S, P),$$

kde:

- T je množina terminálov,
- N je množina neterminálov,
- S je počiatočný symbol,
- P je množina pravidiel v surovej reprezentácii.

V implementácii je množina pravidiel reprezentovaná ako slovník:

```
Dict[str, List[List[str]]]
```

Kľúčom slovníka je neterminál na ľavej strane pravidla a hodnotou je zoznam pravých strán. Každá pravá strana je reprezentovaná ako zoznam symbolov. Funkcia zároveň overuje, či sekcie terminálov a neterminálov nie sú prázdne. Ak niektorá z týchto sekcií chýba alebo je prázdna, funkcia vyvolá výnimku `ValueError`. Ak je v sekcii start uvedený počiatočný symbol, parser ho použije ako počiatočný symbol gramatiky. Ak sekcia start nie je uvedená, parser automaticky zvolí symbol S, ak sa nachádza medzi neterminálmi. V opačnom prípade použije prvý deklarovaný neterminál. Zvolený počiatočný symbol sa následne kontroluje voči množine neterminálov.

Spracovanie pravidiel

Pravidlá parser spracúva v sekcii rules. Každý riadok musí mať tvar *LHS – RHS,* kde ľavá strana predstavuje jeden neterminál a pravá strana predstavuje postupnosť terminálnych

a neterminálnych symbolov. Parser overuje, či neterminál na ľavej strane patrí medzi deklarované neterminály.

Tokenizácia pravej strany pravidla je realizovaná funkciou:

```
def _tokenize_rhs(rhs_raw: str,
                  nonterminals: Set[str],
                  terminals: Set[str]
) -> List[str]:
```

Vstupnými parametrami sú:

- rhs_raw – pravá strana pravidla ako textový reťazec,
- nonterminals – množina deklarovaných neterminálov,
- terminals – množina deklarovaných terminálov.

Návratovou hodnotou je zoznam symbolov pravej strany pravidla.

Funkcia najprv overí, či pravá strana nie je prázdna. Ak pravá strana obsahuje symbol ε, funkcia ho vráti ako samostatný token. Ak pravá strana obsahuje medzery, rozdelí ju podľa medzier. Ak pravá strana presne zodpovedá jednému deklarovanému terminálu alebo neterminálu, vráti ju ako jeden symbol.

V prípade staršieho zápisu bez medzier funkcia používa greedy tokenizáciu, pri ktorej sa snaží zľava doprava nájsť najdlhší možný deklarovaný symbol. Ak sa symbol nepodarí rozpoznať, funkcia vyvolá výnimku `ValueError`. Osobitne je ošetrený aj prázdny reťazec. V implementácii je reprezentovaný symbolom ε . Ak sa na pravej strane pravidla nachádza tento symbol, parser ho interpretuje ako pravidlo pre prázdny reťazec.

Spracovanie jednej alebo viacerých gramatík

Spracovanie celého vstupného súboru zabezpečuje funkcia:

```
def parse_cfg_file_raw_many(path: str) -> List[
    Tuple[Set[str], Set[str], str, Dict[str, List[List[str]]]]
 ]:
```

Vstupom funkcie je cesta k súboru s gramatikou. Funkcia najprv vyhľadá vstupný súbor pomocou `_resolve_input_path`, načíta jeho obsah a rozdelí ho na samostatné bloky pomocou `_split_grammar_blocks`.

Návratovou hodnotou je zoznam surových reprezentácií gramatík. Každá položka zoznamu má tvar (T, N, S, P).

Ak sa pri spracovaní niektorej gramatiky vyskytne chyba, funkcia doplní informáciu o poradí gramatiky v súbore a vyvolá výnimku `ValueError`.

Pre spätnú kompatibilitu je implementovaná aj funkcia:

```
def parse_cfg_file_raw(path: str,
                       grammar_index: int = 0
) -> Tuple[Set[str], Set[str], str, Dict[str, List[List[str]]]]:
```

Táto funkcia umožňuje vybrať jednu konkrétnu gramatiku zo súboru podľa indexu. Vstupom je cesta k súboru a index gramatiky. Návratovou hodnotou je surová reprezentácia jednej vybranej gramatiky.

Transformácia do CHNF

Po úspešnom načítaní parser automaticky prevádza gramatiku do Chomského normálnej formy (CHNF). Tento krok je kľúčový pre ďalšiu redukciu na SAT problém, keďže navrhnutý algoritmus pracuje s pravidlami jednotného tvaru.

Transformácia do CHNF je implementovaná funkciou:

```
def cfg_to_cnf(terminals: Set[str],
               nonterminals: Set[str],
               start: str,
               productions: Dict[str, List[List[str]]],
) -> Tuple[Set[str],
           Set[str],
           str,
           Dict[str, Set[Tuple[str, ...]]]
]:
```

Názov funkcie v zdrojovom kóde používa označenie cnf, keďže ide o pôvodné pomenovanie funkcie. V texte práce však pre gramatickú Chomského normálnu formu používame označenie CHNF, aby sme ju odlíšili od konjunktívnej normálnej formy (CNF) používanej pri SAT formulách.

Vstupnými parametrami funkcie sú:

- terminals – množina terminálov,
- nonterminals – množina neterminálov,
- start – počiatočný symbol,
- productions – surová reprezentácia pravidiel.

Návratovou hodnotou je štvorica:

```
Tuple[Set[str],
      Set[str],
      str,
      Dict[str, Set[Tuple[str, ...]]]]
```

Táto štvorica obsahuje upravenú množinu terminálov, upravenú množinu neterminálov, nový počiatočný symbol a pravidlá v tvare vhodnom pre CHNF. Transformácia prebieha v niekoľkých krokoch:

1. zavedenie nového počiatočného symbolu,
2. odstránenie pravidiel pre prázdny reťazec,
3. odstránenie jednoduchých pravidiel,
4. odstránenie nadbytočných a nedostupných symbolov,
5. nahradenie terminálov v dlhších pravidlách pomocnými neterminálmi,
6. binarizácia pravidiel s pravou stranou dlhšou ako dva symboly,
7. finálne filtrovanie pravidiel do požadovaného tvaru.

Najprv sa vytvorí nový počiatočný symbol S_0, ktorý odkazuje na pôvodný počiatočný symbol. Ak by už symbol S_0 v gramatike existoval, implementácia postupne pridáva ďalšie znaky 0, až kým nevytvorí jedinečný názov.

Následne sa vyhľadávajú nullable neterminály, teda také neterminály, ktoré môžu odvodiť prázdny reťazec. Na základe tejto množiny sa vytvárajú nové pravidlá, v ktorých sú nullable symboly podľa potreby vynechané. Pravidlo pre prázdny reťazec sa zachová iba pre nový počiatočný symbol, ak jazyk gramatiky obsahuje ε.

Ďalším krokom je odstránenie jednoduchých pravidiel tvaru $A \to B$, kde A aj B sú neterminály. Implementácia pre každý neterminál počíta jeho unit closure, teda množinu neterminálov dosiahnuteľných pomocou jednoduchých pravidiel, a následne prenáša nejednoduché pravidlá do príslušných neterminálov.

Potom sa odstránia neužitočné symboly. Implementácia najprv určí generujúce symboly, ktoré dokážu prispieť k odvodeniu terminálneho reťazca. Následne sa určia dosiahnuteľné symboly, ktoré sú dostupné z počiatočného symbolu. V gramatike sa ponechajú iba tie neterminály, ktoré sú zároveň generujúce aj dosiahnuteľné.

Ak sa terminál nachádza v pravidle s pravou stranou dlhšou ako jeden symbol, nahradí sa pomocným neterminálom. Pomocné neterminály majú názov odvodený od terminálu, napríklad T_a. Pre každý takýto pomocný neterminál sa vytvorí pravidlo smerujúce späť na príslušný terminál.

V poslednom kroku sa pravidlá s pravou stranou dlhšou ako dva symboly binarizujú. Implementácia zavádza nové pomocné neterminály, ktoré postupne rozkladajú dlhé pravidlo na reťazec binárnych pravidiel. Po finálnom filtrovaní zostanú iba pravidlá tvaru $A \to a$ alebo $A \to BC$. Osobitne môže zostať pravidlo $S \to \varepsilon$, ak jazyk obsahuje prázdny reťazec.

Vytvorenie internej objektovej reprezentácie

Po transformácii do CHNF parser vytvára interné objekty reprezentujúce terminály, neterminály, pravidlá a samotnú gramatiku. Túto úlohu realizuje funkcia:

```
def build_grammar_objects(
    terminals: Set[str],
    nonterminals: Set[str],
    start: str,
    cnfP: Dict[str, Set[Tuple[str, ...]]],
) -> Grammar:
```

Vstupom funkcie sú:

- množina terminálov,
- množina neterminálov,
- počiatočný symbol,
- pravidlá v CHNF reprezentácii.

Návratovou hodnotou je objekt typu Grammar. Funkcia najprv vytvorí slovníky objektov `Terminal` a `NonTerminal`. Následne prechádza pravidlá v CHNF. Ak má pravidlo pravú stranu dĺžky jedna, vytvorí objekt `Rule` reprezentujúci pravidlo tvaru $A \to a$. Ak má pravidlo pravú stranu dĺžky dva, vytvorí objekt `Rule` reprezentujúci pravidlo tvaru $A \to BC$.

Pravidlo pre prázdny reťazec sa priamo neprevádza na objekt `Rule`, pretože SAT model nepracuje s prázdnymi intervalmi vstupného slova. Informácia o tom, či gramatika prijíma prázdny reťazec, sa však uloží do atribútu `accepts_epsilon` . Táto informácia sa neskôr používa pri špeciálnom prípade, keď vstupné slovo neobsahuje žiadne tokeny.

Výsledný objekt Grammar obsahuje:

- zoznam objektov Rule,
- zoznam neterminálov,
- zoznam terminálov,
- počiatočný symbol,
- informáciu o prijímaní prázdneho reťazca.

Táto objektová reprezentácia sa následne využíva najmä v triede `CFG_2_SAT`.

Export medzivýstupov

Parser okrem internej reprezentácie vytvára aj pomocné textové výstupy. Exporty sú realizované funkciou:

```
def _write_export_files(
    input_path: Path,
    raw_grammars: List[Tuple[Set[str], Set[str], str, Dict[str,
List[List[str]]]]],
    cnf_blocks: List[Tuple[Set[str], Set[str], str, Dict[str,
Set[Tuple[str, ...]]]]],
) -> None:
```

Vstupom funkcie je cesta k pôvodnému vstupnému súboru, zoznam surových gramatík a zoznam gramatík po transformácii do CHNF. Funkcia nevracia žiadnu hodnotu, ale vytvára výstupné súbory na disku. Cieľové cesty pre exporty určuje funkcia:

```
def _export_output_paths(input_path: Path) -> tuple[Path, Path]:
```

Táto funkcia vracia dvojicu ciest:

- cestu pre Java formát,
- cestu pre CHNF výstup.

Konkrétne sa používajú adresáre:

`src/text/output/JavaGrammar/`

`src/text/output/Chomsky/`

Parser vytvára najmä tieto výstupy:

- textovú reprezentáciu pôvodnej gramatiky,
- textovú reprezentáciu gramatiky v CHNF,
- výstup v tvare kompatibilnom s nástrojmi na generovanie reťazcov [14].

Serializácia do Java formátu je realizovaná funkciou:

```
def _serialize_java_cfg(
    terminals: Set[str],
    start: str,
    productions: Dict[str, List[List[str]]],
 ) -> str:
```

Návratovou hodnotou tejto funkcie je textový reťazec vo formáte použiteľnom pre externý Java generátor slov [14]. Tento výstup má špecifický význam pre experimentálnu časť práce, pretože umožňuje generovať pozitívne a negatívne vstupné reťazce pre benchmarking.

Serializácia gramatiky v CHNF je realizovaná funkciou:

```
def _serialize_cnf_cfg(
    terminals: Set[str],
    nonterminals: Set[str],
    start: str,
    cnf_productions: Dict[str, Set[Tuple[str, ...]]],
) -> str:
```

Návratovou hodnotou je textová reprezentácia gramatiky po transformácii do CHNF. Tento výstup slúži najmä na kontrolu správnosti transformácie a na ladenie implementácie.

Hlavný priebeh funkcie `parse_grammar_file_to_chomsky`

Celý proces spracovania vstupnej gramatiky je zhrnutý vo verejnej funkcii:

```
def parse_grammar_file_to_chomsky(path: str) -> List[Grammar]:
```

Táto funkcia vykonáva nasledujúce kroky:

1. vyhľadá vstupný súbor pomocou `_resolve_input_path`,
2. načíta všetky surové gramatiky pomocou `parse_cfg_file_raw_many`,
3. pre každú gramatiku zavolá `cfg_to_cnf`,
4. vytvorí objekt `Grammar` pomocou `build_grammar_objects`,
5. doplní metadáta o zdrojovom súbore a indexe gramatiky,
6. uloží pomocné exporty pomocou `_write_export_files`,
7. vráti zoznam objektov `Grammar`.

Vstupným parametrom je cesta k súboru so vstupnou gramatikou. Výstupom je zoznam objektov `Grammar`. Ak vstupný súbor obsahuje jednu gramatiku, zoznam obsahuje jeden objekt. Ak obsahuje viacero gramatík, každá z nich je reprezentovaná samostatným objektom.

Zhrnutie funkcie parsera

Parser plní v systéme viacero dôležitých úloh. Najprv načíta vstupný súbor, odstráni komentáre, rozdelí vstup na jednotlivé gramatiky a syntakticky overí jednotlivé sekcie. Následne vytvorí surovú reprezentáciu gramatiky vo forme štvorice (T, N, S, P), transformuje ju do CHNF a vytvorí interný objekt typu `Grammar`.

Z pohľadu architektúry celého riešenia ide o vstupnú vrstvu systému, ktorá zabezpečuje, že ďalšie komponenty už pracujú s jednotnou a formálne korektnou reprezentáciou gramatiky. Výstup parsera je preto priamym vstupom pre modul `CFG_2_SAT`, ktorý následne realizuje samotnú redukciu parsovacieho problému na SAT problém.

8.3 Generovanie SAT modelu

Po spracovaní vstupnej gramatiky parserom nasleduje jadro implementácie, ktorého úlohou je previesť parsovací problém na problém splniteľnosti booleovskej formule. Táto časť systému je realizovaná triedou `CFG_2_SAT`, ktorá predstavuje hlavný modul zodpovedný za konštrukciu SAT modelu, generovanie klauzúl, export výslednej formule do DIMACS formátu a voliteľne aj za spustenie SAT riešiča. Generovanie SAT modelu je implementované v zdrojovom súbore:

`src/CFG_2_SAT.py`

Centrálnou triedou tejto časti je:

```
class CFG_2_SAT()
```

Trieda pracuje s gramatikou, ktorá už bola prevedená do Chomského normálnej formy (CHNF), a so vstupným reťazcom reprezentovaným ako postupnosť tokenov. Implementácia je navrhnutá tak, aby bolo možné samostatne vykonať konštrukciu formule, uložiť ju do súboru a následne ju použiť v experimentálnej časti práce pri benchmarkingu viacerých SAT riešičov.

Inicializácia modelu

Vstupným bodom triedy je jej konštruktor:

```
def __init__(self, grammar:Grammar, word:str,
             solve: bool = False, save_dimacs: bool = True,
             verbose: bool = True):
```

Konštruktor prijíma päť parametrov. Parameter `grammar` je objekt typu `Grammar`, ktorý obsahuje gramatiku v CHNF. Parameter `word` predstavuje vstupné slovo vo forme textového reťazca. Parameter `solve` určuje, či sa má formula po vygenerovaní okamžite riešiť pomocou SAT riešiča. Parameter `save_dimacs` určuje, či sa má vytvorená CNF formula uložiť do DIMACS súboru. Parameter `verbose` riadi výpis priebežných informačných správ.

Konštruktor nevracia samostatnú návratovú hodnotu, ale vytvára objekt `CFG_2_SAT`, ktorý po inicializácii obsahuje všetky údaje potrebné na ďalšie spracovanie. Medzi najdôležitejšie atribúty patria:

```
self.grammar
self.word
self.word_tokens
self.bools
self.clauses
self.stats
self.dimacs_path
```

Atribút `grammar` obsahuje pôvodný objekt gramatiky. Vstupný reťazec je uložený v atribúte `word` a jeho tokenizovaná podoba v atribúte `word_tokens`.

Počas inicializácie sa vytvorí aj slovník `stats`, ktorý uchováva štatistické údaje o gramatike, slove a vygenerovanej formule. Ide napríklad o počet neterminálov, počet pravidiel, počet terminálnych a binárnych pravidiel, dĺžku vstupného slova, počet základných premenných, počet Tseitinových premenných a počet klauzúl. Tieto údaje sa neskôr používajú pri benchmarkingu a experimentálnom vyhodnotení.

Spracovanie prázdneho reťazca

Implementácia osobitne ošetruje prípad, keď je vstupný reťazec prázdny. V konštruktore sa vstupné slovo najprv upraví pomocou metódy strip() a následne sa rozdelí na tokeny:

```
self.word = word.strip()
self.word_tokens = [] if self.word == "" else self.word.split()
```

Ak je zoznam `word_tokens` prázdny, systém negeneruje SAT formulu. Tento prípad možno rozhodnúť priamo na základe atribútu:

`grammar.accepts_epsilon`

Ak gramatika prijíma prázdny reťazec, systém vyhodnotí vstup ako prijatý. V opačnom prípade je vstup zamietnutý. Tento postup je dôležitý preto, že navrhovaný SAT model nepracuje s prázdnymi intervalmi vstupného slova. Prázdny vstup sa preto rieši oddelene ešte pred generovaním premenných a klauzúl.

Vytvorenie základných booleovských premenných

Základné booleovské premenné sa vytvárajú metódou:

```
def init_bools(self) -> dict:
```

Metóda nemá externé vstupné parametre, pretože pracuje s atribútmi objektu `CFG_2_SAT`, najmä s `self.grammar` a `self.word_tokens`. Návratovou hodnotou je slovník bools, ktorý obsahuje všetky základné premenné SAT modelu. V implementácii je každá premenná reprezentovaná ako položka slovníka. Kľúč má textový tvar *A,i,j* ,hodnota je dvojica *(boolean_value, variable_id)*.

Prvá zložka predstavuje fixovanú pravdivostnú hodnotu alebo hodnotu None, ak premenná zatiaľ nie je fixovaná. Druhá zložka je číselný identifikátor pre SAT riešič a DIMACS formát.

Metóda najprv vytvorí premenné pre všetky kombinácie neterminálov a intervalov vstupného slova:

```
for nonterminal in self.grammar.NonTerminals:
    for i in range(n):
        for j in range(i, n):
            bools[nonterminal.value + ',' + str(i) + ',' + str(j)]
= (None, solver_name)
            solver_name += 1
```

Každá premenná dostane jedinečné číslo. Tento identifikátor sa neskôr používa pri zápise literálov do klauzúl.

Predbežná inicializácia hodnôt

Metóda `init_bools()` zároveň fixuje niektoré premenné na hodnotu True alebo False. Tým sa znižuje priestor hľadania a zjednodušuje následná konštrukcia klauzúl.

Najprv sa nastaví premenná reprezentujúca tvrdenie, že počiatočný symbol generuje celé vstupné slovo $X_{S,0,n-1}$. Táto premenná je nastavená na hodnotu `True`.

Následne sa spracujú terminálne pravidlá tvaru: $A \rightarrow a$. Ak sa na pozícii i nachádza token zodpovedajúci terminálu a, implementácia nastaví premennú $X_{A,i,i}$ na hodnotu `True`. Všetky ostatné premenné $X_{A,i,i}$, ktoré nezodpovedajú žiadnemu terminálnemu pravidlu, sa nastavia na hodnotu `False`.

Ďalej sa kontroluje, či má daný neterminál aspoň jedno binárne pravidlo tvaru $A \rightarrow BC$ Ak pre neterminál A neexistuje žiadne binárne pravidlo, potom tento neterminál nemôže generovať interval dlhší ako jeden token. V takom prípade sa všetky premenné $X_{A,i,j}$ pre $i < j$ nastavia na hodnotu `False`.

Výstupom metódy `init_bools()` je teda slovník premenných, ktorý obsahuje základné premenné SAT modelu spolu s ich číselnými identifikátormi a prípadnými fixovanými hodnotami.

Generovanie klauzúl

Po vytvorení základných premenných systém pristupuje ku generovaniu klauzúl výslednej CNF formule. Táto fáza je implementovaná metódou:

```
def init_clauses(self) -> List[List[int]]:
```

Metóda nemá externé vstupné parametre. Pracuje s atribútom `self.bools`, ktorý obsahuje vygenerované premenné, a s atribútom `self.grammar.rules`, ktorý obsahuje pravidlá gramatiky. Návratovou hodnotou je zoznam klauzúl, pričom každá klauzula je reprezentovaná ako zoznam celých čísel.

Najprv sa vytvárajú jednotkové klauzuly pre premenné, ktoré už majú fixovanú pravdivostnú hodnotu. Ak je premenná nastavená na `True`, do formule sa vloží klauzula obsahujúca jej kladný identifikátor. Ak je premenná nastavená na `False`, do formule sa vloží klauzula obsahujúca negovaný identifikátor.

V kóde je táto časť realizovaná nasledovne:

```
for key in self.bools:
    val, var = self.bools[key]
    if val is True:
        clauses.append([var])
    elif val is False:
        clauses.append([-var])
```

Tseitinove pomocné premenné

Pre každé binárne pravidlo tvaru $A \rightarrow BC$ a pre každé možné rozdelenie intervalu (i, j) v bode k implementácia zavádza pomocnú Tseitinovu premennú. Táto premenná reprezentuje tvrdenie, že interval (i, j) možno odvodiť pomocou konkrétneho pravidla a konkrétneho deliaceho bodu.

Ak označíme pomocnú premennú ako z, potom reprezentuje vzťah:

$$z \leftrightarrow (X_{B,i,k} \wedge X_{C,k+1,j})$$

V implementácii sa tento vzťah zapisuje pomocou troch klauzúl:

$$\neg z \vee X_{B,i,k}$$
$$\neg z \vee X_{C,k+1,j}$$
$$\neg X_{B,i,k} \vee \neg X_{C,k+1,j} \vee z$$

Tieto klauzuly zabezpečujú ekvivalenciu medzi pomocnou premennou a konjunkciou dvoch podmienok. V kóde sa generujú nasledovne:

```
clauses.append([-z, Bik])
clauses.append([-z, Ckj])
clauses.append([-Bik, -Ckj, z])
```

Počet takýchto pomocných premenných sa ukladá do štatistiky:

`tseitin_variable_count`

a počet príslušných klauzúl do:

`tseitin_clause_count`

Klauzuly pre odvodenie intervalov

Po vygenerovaní pomocných premenných pre všetky pravidlá a deliace body sa pre každú premennú $X_{A,i,j}$ s intervalom dĺžky aspoň dva vytvorí klauzula typu „okno“. Táto klauzula zabezpečuje, že ak je $X_{A,i,j}$ pravdivá, potom musí existovať aspoň jeden konkrétny spôsob odvodenia intervalu.

Má tvar:

$$\neg X_{A,i,j} \vee z_1 \vee z_2 \vee \cdots \vee z_m$$

kde $z_1, z_2, \ldots, z_m$ sú pomocné premenné reprezentujúce jednotlivé možnosti odvodenia pomocou pravidiel a deliacich bodov.

V implementácii sa táto klauzula vytvára takto:

```
clauses.append([-Aij] + option_vars)
```

Ak pre daný neterminál a interval neexistuje žiadna možnosť odvodenia, systém vloží klauzulu:

```
clauses.append([-Aij])
```

To znamená, že premenná $X_{A,i,j}$ musí byť nepravdivá.

Počet týchto klauzúl sa eviduje v štatistike:

`window_clause_count`

Výstupom metódy `init_clauses()` je kompletný zoznam klauzúl CNF formule.

Evidencia štatistík

Súčasťou triedy `CFG_2_SAT` je aj evidencia štatistických údajov. Tieto údaje sú uložené v slovníku:

`self.stats`

Slovník obsahuje najmä tieto položky:

```
nonterminal_count
rule_count
singular_rule_count
double_rule_count
word_length
base_bool_variable_count
tseitin_variable_count
bool_variable_count
fixed_value_clause_count
tseitin_clause_count
window_clause_count
clause_count
```

Tieto hodnoty sa priebežne dopĺňajú počas inicializácie premenných a generovania klauzúl. Základný počet premenných sa nastavuje po zavolaní `init_bools()`:

```
self.stats["base_bool_variable_count"] = len(self.bools)
```

Celkový počet klauzúl sa nastavuje po zavolaní `init_clauses()`:

```
self.stats["clause_count"] = len(self.clauses)
```

Prístup k týmto štatistikám zabezpečuje metóda:

```
def get_stats(self) -> dict[str, int]:
```

Metóda nemá vstupné parametre a vracia kópiu slovníka `stats`. Návratová hodnota sa používa najmä v benchmarkovacích skriptoch.

Export do DIMACS formátu

Po skonštruovaní množiny klauzúl môže implementácia vytvoriť výstupný súbor v DIMACS formáte. Export je realizovaný metódou:

```
def save_dimacs(self) -> str | None:
```

Metóda nemá externé vstupné parametre. Pracuje s atribútom `self.clauses`, ktorý obsahuje vygenerované klauzuly. Návratovou hodnotou je cesta k vytvorenému DIMACS súboru vo forme textového reťazca. Ak klauzuly neexistujú, metóda vracia hodnotu `None`.

Pred samotným zápisom súboru metóda vypočíta počet premenných ako najväčšiu absolútnu hodnotu literálu v klauzulách:

```
num_variables = 0
    for clause in self.clauses:
        for literal in clause:
            num_variables = max(num_variables, abs(literal))
```

Cesta k výstupnému súboru sa získava pomocou pomocnej metódy:

```
def _dimacs_output_path(self) -> Path:
```

Táto metóda určuje cieľový adresár a názov súboru podľa pôvodného vstupného súboru gramatiky. Ak súbor obsahuje viacero gramatík, do názvu výstupného súboru sa pridáva aj index gramatiky.

DIMACS súbor obsahuje komentárové riadky s informáciou o zdrojovej gramatike, vstupnom slove a štatistikách modelu. Následne obsahuje hlavičku:

```
p cnf počet_premenných počet_klauzúl
```

a samotné klauzuly ukončené nulou:

```
1 -2 3 0
```

Takto vytvorený súbor môže byť použitý rôznymi SAT riešičmi bez potreby meniť implementáciu generovania modelu.

Riešenie formule pomocou SAT riešiča

Ak je pri inicializácii objektu nastavený parameter:

`solve=True`

po vygenerovaní klauzúl sa zavolá metóda:

```
def solve_pysat(self) -> None:
```

Metóda nemá vstupné parametre a nevracia samostatnú návratovú hodnotu. Jej úlohou je vytvoriť objekt `CNF` z klauzúl a spustiť SAT riešič prostredníctvom knižnice PySAT.

V implementácii sa CNF formula vytvára príkazom:

```
from pysat.formula import CNF

cnf = CNF(from_clauses=self.clauses)
```

Následne sa spustí SAT riešič:

```
with Solver(bootstrap_with=cnf) as solver:
    sat_result = solver.solve()
```

Výsledok `sat_result` je logická hodnota. Hodnota `True` znamená, že formula je splniteľná, a teda vstupné slovo patrí do jazyka gramatiky. Hodnota `False` znamená, že formula je nesplniteľná, a teda vstupné slovo do jazyka nepatrí.

Overenie výsledku pomocou CYK

Súčasťou metódy `solve_pysat()` je aj porovnanie výsledku so štandardným algoritmom CYK. Overenie je realizované volaním funkcie:

```
cyk_accepts(self.grammar, self.word_tokens)
```

Táto funkcia je implementovaná v súbore:

`src/modules/CYK.py`

Vstupom funkcie je objekt `Grammar` a tokenizované vstupné slovo. Návratovou hodnotou je logická hodnota určujúca, či CYK algoritmus prijíma dané slovo.

Porovnanie výsledkov sa používa ako kontrolný mechanizmus. Ak sa výsledok SAT prístupu zhoduje s výsledkom CYK algoritmu, systém vypíše informáciu, že kontrola prebehla úspešne. Ak sa výsledky líšia, systém vypíše upozornenie na nezhodu. Tento mechanizmus pomáha odhaliť prípadnú chybu v generovaní klauzúl alebo v implementácii samotného modelu.

Rekonštrukcia derivačného stromu

Ak je formula splniteľná, implementácia dokáže z výsledného modelu spätne rekonštruovať derivačný strom. Rekonštrukcia je implementovaná metódou:

```
def print_derivation(self, model: list[int]) -> None:
```

Vstupným parametrom metódy je `model`, teda zoznam literálov vrátený SAT riešičom. Návratová hodnota nie je definovaná; metóda vypisuje derivačný strom vo forme textového výstupu.

Metóda najprv prejde všetky základné premenné v slovníku `self.bools` a nastaví ich pravdivostné hodnoty podľa modelu vráteného SAT riešičom. Následne spustí rekurzívnu pomocnú funkciu:

```
def print_derivation_helper(nonterminal: NonTerminal,
                            i: int, j: int, indent: str):
```

Táto pomocná funkcia pre aktuálny neterminál a interval (i, j) hľadá pravidlo, ktoré vysvetľuje odvodenie daného podreťazca. Ak ide o terminálne pravidlo a interval má dĺžku jeden, funkcia vypíše list derivačného stromu. Ak ide o binárne pravidlo, funkcia hľadá deliaci bod k, pri ktorom sú príslušné ľavé a pravé podintervaly pravdivé, a následne pokračuje rekurzívne pre obe časti. Ak sa takéto rozdelenie nájde, funkcia vypíše použité pravidlo a pokračuje rekurzívne v ľavej a pravej vetve derivačného stromu.

Výsledkom je textový zápis derivačného stromu, ktorý ukazuje konkrétnu deriváciu vstupného slova podľa danej gramatiky.

Zhrnutie úlohy triedy `CFG_2_SAT`

Trieda `CFG_2_SAT` predstavuje centrálny implementačný modul celého riešenia. Jej vstupom je gramatika v CHNF a vstupné slovo. Výstupom je vytvorený SAT model reprezentovaný pomocou booleovských premenných a klauzúl CNF formule. Voliteľne môže byť formula uložená do DIMACS formátu alebo priamo vyriešená pomocou SAT riešiča.

Z implementačného hľadiska trieda zabezpečuje tieto logické bloky:

- inicializáciu vstupných údajov,
- ošetrenie prázdneho reťazca,
- generovanie základných premenných,
- generovanie Tseitinových pomocných premenných,
- generovanie CNF klauzúl,
- evidenciu štatistík,
- export do DIMACS formátu,
- riešenie pomocou SAT riešiča,
- kontrolu výsledku pomocou CYK,
- rekonštrukciu derivačného stromu.

Týmto spôsobom trieda prepája teoretický návrh redukcie parsovacieho problému na SAT s jeho praktickou realizáciou v implementovanom systéme.

8.4 Benchmarking

8.4.1 Porovnanie SAT riešičov

Súčasťou implementácie je samostatný modul určený na benchmarkovanie viacerých SAT riešičov nad formulami vytvorenými z bezkontextových gramatík. Cieľom tejto časti je zabezpečiť jednotný a reprodukovateľný spôsob generovania testovacích inštancií, ich

riešenia pomocou rôznych SAT riešičov a zaznamenávania štatistických údajov potrebných pre neskoršie experimentálne vyhodnotenie.

Benchmarking SAT riešičov je implementovaný v zdrojovom súbore:

`src/benchmark_sat_solvers.py`

Tento modul nadväzuje na predchádzajúce časti implementácie. Využíva parser na načítanie gramatík, triedu `CFG_2_SAT` na generovanie CNF formulí a DIMACS súborov a následne spúšťa viacero SAT riešičov nad rovnakými vstupnými formulami.

Základná myšlienka benchmarku spočíva v oddelení konštrukcie SAT modelu od procesu jeho riešenia. Najprv sa pre konkrétnu gramatiku a konkrétne vstupné slovo vytvorí DIMACS súbor. Následne sa rovnaký súbor použije ako vstup pre viaceré SAT riešiče. Tým sa zabezpečí, že všetky riešiče pracujú s identickou formulou a výsledky sú medzi sebou porovnateľné.

Vstup benchmarku

Hlavným vstupným bodom benchmarkovacieho modulu je funkcia:

```
def benchmark_from_grammar_file(
    grammar_file: str,
    min_length: int = 10,
    max_length: int = 18,
    positive_count: int = 10,
    negative_count: int = 10,
    java_jar_path: str | None = None,
) -> list[dict[str, str]]:
```

Vstupným parametrom `grammar_file` je cesta k súboru obsahujúcemu jednu alebo viac bezkontextových gramatík. Parametre `min_length` a `max_length` určujú minimálnu a maximálnu dĺžku generovaných testovacích slov. Parametre `positive_count` a `negative_count` určujú počet pozitívnych a negatívnych testovacích príkladov pre každú gramatiku. Parameter `java_jar_path` obsahuje cestu k externému Java nástroju používanému na generovanie testovacích slov [14].

Návratovou hodnotou funkcie je zoznam slovníkov:

`list[dict[str, str]]`

Každý slovník reprezentuje jeden výsledok benchmarku pre konkrétnu kombináciu gramatiky, vstupného slova a SAT riešiča. Obsahuje údaje ako názov DIMACS súboru, názov riešiča, index gramatiky, testované slovo, dĺžku slova, výsledok riešenia, čas riešenia a štatistiky vygenerovaného modelu.

Funkcia najprv načíta gramatiky pomocou funkcie:

```
parse_grammar_file_to_chomsky(grammar_file)
```

Táto funkcia je definovaná v súbore:

`src/modules/parser.py`

Vstupom je cesta k súboru s gramatikou a výstupom je zoznam objektov typu `Grammar`, pričom každá gramatika je už prevedená do CHNF.

Množina benchmarkovaných SAT riešičov

V benchmarku sa používa viacero SAT riešičov dostupných prostredníctvom knižnice PySAT. Ich skratky sú uložené v zozname:

```
PYSAT_SOLVER_NAMES = [
    "g3",    # Glucose3
    "g4",    # Glucose4
    "gc3",   # Gluecard3
    "gc4",   # Gluecard4
    "m22",   # Minisat22
    "mc",    # MapleCM
    "mgh",   # MapleGH
]
```

Okrem týchto riešičov modul voliteľne podporuje aj CryptoMiniSat prostredníctvom knižnice `pycryptosat`. Import je ošetrený pomocou bloku:

```
try:
    from pycryptosat import Solver as CryptoSolver
except Exception:
    CryptoSolver = None
```

Ak knižnica `pycryptosat` nie je dostupná, benchmark tento riešič nevyhodí ako chybu, ale označí ho ako preskočený. Tým sa zvyšuje prenositeľnosť benchmarku medzi rôznymi prostrediami.

Riešenie CNF formule pomocou PySAT

Riešenie CNF formule pomocou riešičov dostupných cez PySAT je realizované funkciou:

```
def _solve_with_pysat(cnf: CNF, solver_name) -> tuple[str, str]:
```

Vstupom funkcie je objekt `CNF`, ktorý obsahuje načítanú formulu, a parameter `solver_name`, ktorý určuje konkrétny PySAT backend. Návratovou hodnotou je dvojica:

```
(status, result)
```

Hodnota `status` je v prípade úspešného behu nastavená na `ok`. Hodnota `result` obsahuje textový výsledok `SAT` alebo `UNSAT` podľa toho, či je formula splniteľná.

Funkcia vytvorí inštanciu riešiča pomocou:

```
PySatSolver(name=solver_name, bootstrap_with=cnf.clauses)
```

a následne zavolá metódu:

```
solver.solve()
```

Výsledok tejto metódy je prevedený na textovú hodnotu `SAT` alebo `UNSAT`.

Riešenie CNF formule pomocou CryptoMiniSat

Voliteľná podpora riešiča CryptoMiniSat je implementovaná funkciou:

```
def _solve_with_pycryptosat(cnf: CNF) -> tuple[str, str]:
```

Vstupom funkcie je objekt `CNF`. Návratovou hodnotou je dvojica obsahujúca stav a výsledok riešenia. Ak knižnica `pycryptosat` nie je dostupná, funkcia vráti:

```
("skipped", "missing-pycryptosat")
```

Ak je knižnica dostupná, funkcia vytvorí inštanciu `CryptoSolver`, postupne do nej pridá všetky klauzuly pomocou metódy `add_clause` a následne zavolá `solve()`. Výsledok sa opäť prevedie na textovú hodnotu `SAT` alebo `UNSAT`.

Spustenie jedného riešiča a meranie času

Spoločná logika pre spustenie jedného SAT riešiča je implementovaná funkciou:

```
def _run_solver_inprocess(
    dimacs_path: Path,
    cnf: CNF,
    solver_name: str,
    solve_fn: Callable[[CNF], tuple[str, str]],
    grammar_index: int,
    word: str,
    word_length: int,
    positive_or_negative: str,
    stats: dict[str, int],
) -> dict[str, str]:
```

Táto funkcia predstavuje jeden beh benchmarku pre konkrétnu formulu a konkrétny SAT riešič. Vstupom je cesta k DIMACS súboru, načítaná CNF formula, názov riešiča, funkcia realizujúca samotné riešenie, index gramatiky, testované slovo, dĺžka slova, označenie pozitívneho alebo negatívneho príkladu a štatistiky modelu.

Meranie času je realizované pomocou:

```
time.perf_counter()
```

Čas sa meria pred spustením riešiča a po jeho dokončení. Výsledný čas sa prepočíta na milisekundy:

```
elapsed_ms = (time.perf_counter() - start) * 1000.0
```

Návratovou hodnotou funkcie je slovník obsahujúci kompletný záznam o jednom behu benchmarku. Ak počas riešenia nastane výnimka, funkcia ju zachytí, nastaví stav na `error` a do výsledku uloží názov výnimky.

Vytvorenie výstupného riadku benchmarku

Jednotná štruktúra výstupných údajov je zabezpečená funkciou:

```
def _make_row(
    dimacs_path: Path,
    solver_name: str,
    status: str,
    result: str,
    elapsed_ms: float,
    grammar_index: int,
    word: str,
    word_length: int,
    positive_or_negative: str,
    stats: dict[str, int],
) -> dict[str, str]:
```

Vstupom funkcie sú všetky údaje potrebné na popis jedného benchmarkovacieho behu. Návratovou hodnotou je slovník, ktorý sa neskôr uloží ako jeden riadok do CSV súboru.

Slovník obsahuje napríklad tieto položky:

- `dimacs_file`,
- `solver`,
- `grammar`,
- `word`,
- `word_length`,
- `positive_or_negative`,
- `nonterminal_count`,
- `rule_count`,
- `singular_rule_count`,
- `double_rule_count`,
- `bool_variable_count`,
- `base_bool_variable_count`,
- `tseitin_variable_count`,
- `clause_count`,
- `fixed_value_clause_count`,
- `tseitin_clause_count`,
- `window_clause_count`,
- `status`,
- `result`,
- `time_ms`.

Týmto spôsobom je zabezpečené, že všetky výsledky benchmarku majú rovnakú štruktúru bez ohľadu na použitý SAT riešič.

Benchmarkovanie jedného DIMACS súboru

Samotné spustenie všetkých podporovaných SAT riešičov nad jedným DIMACS súborom zabezpečuje funkcia:

```
def benchmark_dimacs_file(
    dimacs_path: Path,
    grammar_index: int,
    word: str,
    positive_or_negative: str,
    stats: dict[str, int],
) -> list[dict[str, str]]:
```

Vstupom funkcie je cesta k DIMACS súboru, index gramatiky, testované slovo, označenie typu príkladu a štatistiky modelu. Návratovou hodnotou je zoznam slovníkov, kde každý slovník reprezentuje výsledok jedného SAT riešiča nad danou formulou.

Funkcia najprv načíta DIMACS súbor do objektu:

```
cnf = CNF(from_file=str(dimacs_path))
```

Následne sa pokúsi z hlavičky DIMACS súboru prečítať počet premenných a počet klauzúl. Tieto údaje sa zapisujú do logu a slúžia na kontrolu veľkosti riešenej inštancie.

Potom funkcia postupne prechádza všetky riešiče uvedené v `PYSAT_SOLVER_NAMES` a pre každý z nich zavolá `_run_solver_inprocess`. Po spracovaní PySAT riešičov sa rovnakým spôsobom pridá aj beh pre `pycryptosat`. Výsledkom je zoznam riadkov benchmarku pre všetky dostupné riešiče nad jednou CNF formulou.

Generovanie testovacích slov

Pred samotným benchmarkovaním je potrebné pripraviť vstupné slová pre jednotlivé gramatiky. Táto časť implementácie využíva export gramatík v tvare kompatibilnom s externým Java generátorom slov [14]. Generovanie slov zabezpečuje funkcia:

```
def _generate_words_with_java_jar(
    java_grammar_file: str,
    min_length: int,
    max_length: int,
    target_count: int,
    jar_path: str,
    label: str,
) -> list[str]:
```

Vstupom funkcie je cesta k súboru s gramatikou vo formáte očakávanom Java generátorom, minimálna a maximálna dĺžka generovaných slov, požadovaný počet slov, cesta k súboru JAR a parameter `label`, ktorý určuje, či sa majú generovať pozitívne alebo negatívne príklady. Návratovou hodnotou je zoznam vygenerovaných slov.

Ak je požadovaný počet slov nulový alebo ak neexistuje zadaný JAR súbor, funkcia vráti prázdny zoznam. Externý nástroj je spúšťaný príkazom:

```
subprocess.run(command, ...)
```

kde príkaz obsahuje cestu k Java JAR súboru, cestu ku gramatike, minimálnu dĺžku, maximálnu dĺžku, počet opakovaní a označenie typu generovaných slov.

Keďže pri generovaní môžu vzniknúť duplicity alebo nedostatočný počet vhodných slov, funkcia realizuje najviac tri pokusy. V každom ďalšom pokuse zvyšuje počet opakovaní generátora. Vygenerované slová sa ukladajú do množiny `seen`, aby sa odstránili duplicity.

Na načítanie slov z výstupu Java programu slúži funkcia:

```
def _parse_java_generated_words(output_path: Path) -> list[str]:
```

Vstupom je cesta k výstupnému súboru generátora. Návratovou hodnotou je zoznam slov načítaných zo sekcie `Generated words:`. Funkcia ignoruje prázdne riadky a riadky obsahujúce štatistiky generátora.

Použitý Java generátor slov nebol vytvorený v rámci tejto práce. Ide o externý nástroj z predchádzajúcej študentskej práce, ktorý v našom riešení používame ako pomocný komponent na automatizované generovanie testovacích vstupov [14].

Spracovanie Java exportu gramatiky

Parser v predchádzajúcej fáze vytvára export gramatík vo formáte vhodnom pre externý Java generátor. Keďže vstupný súbor môže obsahovať viacero gramatík, benchmarkovací modul musí tento export rozdeliť na samostatné bloky. Na to slúži funkcia:

```
def _split_grammar_blocks(text: str) -> list[str]:
```

Vstupom je textový obsah Java exportu. Návratovou hodnotou je zoznam blokov, pričom každý blok reprezentuje jednu gramatiku. Bloky sú oddelené riadkom obsahujúcim „---“.

Externý Java generátor očakáva vždy jednu gramatiku v jednom vstupnom súbore. Preto sa príslušný blok pred generovaním slov uloží do dočasného súboru pomocou funkcie:

```
def _write_java_grammar_block_temp_file(grammar_block) -> Path:
```

Vstupom je text jedného gramatického bloku. Návratovou hodnotou je cesta k dočasnému súboru. Po použití sa tento súbor odstráni pomocou:

```
temp_grammar_path.unlink(missing_ok=True)
```

Týmto spôsobom je možné spracovať aj viacero gramatík v jednom vstupnom súbore a pre každú z nich generovať samostatnú množinu testovacích slov.

Príprava DIMACS inštancií

Pre každé vygenerované slovo sa v hlavnej funkcii `benchmark_from_grammar_file` vytvorí samostatná inštancia triedy:

```
cfg_sat = CFG_2_SAT(grammar, word, solve=False)
```

Vstupom je konkrétny objekt `Grammar` a konkrétne testované slovo. Parameter `solve=False` znamená, že pri vytváraní inštancie sa formula priamo nerieši, ale iba sa vygeneruje SAT model a DIMACS súbor. Po vytvorení objektu sa získajú štatistiky modelu pomocou:

```
stats = cfg_sat.get_stats()
```

Výstupom tejto metódy je slovník so štatistickými údajmi, napríklad počtom premenných, počtom klauzúl a počtom pravidiel.

Aby bol každý benchmarkovací prípad uložený samostatne a nedochádzalo k prepisovaniu DIMACS súborov, používa sa funkcia:

```
def _copy_dimacs_for_case(
    dimacs_path: Path,
    grammar_file: Path,
    grammar_index: int,
    positive_or_negative: str,
    word_index: int,
) -> Path:
```

Vstupom je pôvodná cesta k DIMACS súboru, cesta ku gramatickému súboru, index gramatiky, označenie pozitívneho alebo negatívneho príkladu a index slova. Návratovou hodnotou je cesta k novej kópii DIMACS súboru pre konkrétny benchmarkovací prípad.

Súbor sa ukladá do adresára:

`src/text/dimacs_outputs/benchmark_runs/`

a jeho názov obsahuje index gramatiky, typ príkladu a index slova.

Hlavný priebeh benchmarku

Hlavná funkcia:

```
def benchmark_from_grammar_file(...)
```

vykonáva celý benchmarkovací proces. Jej priebeh možno zhrnúť do nasledujúcich krokov:

1. načítanie gramatík pomocou parsera,
2. načítanie Java exportu gramatík,
3. rozdelenie Java exportu na bloky,
4. generovanie pozitívnych a negatívnych slov pre každú gramatiku,
5. vytvorenie objektu `CFG_2_SAT` pre každé slovo,
6. export alebo skopírovanie DIMACS súboru pre každý prípad,
7. spustenie všetkých podporovaných SAT riešičov nad každým DIMACS súborom,
8. uloženie výsledkov do zoznamu záznamov.

Výstupom funkcie je zoznam záznamov, ktorý sa následne odovzdáva funkcii `write_csv`.

Ukladanie výsledkov do CSV

Výsledky benchmarku sa ukladajú pomocou funkcie:

```
def write_csv(rows: list[dict[str, str]], output_csv: Path)->None:
```

Vstupom funkcie je zoznam výsledkov benchmarku a cesta k výstupnému CSV súboru. Funkcia nevracia žiadnu hodnotu. Jej výsledkom je vytvorený CSV súbor, ktorý obsahuje jeden riadok pre každú kombináciu testovacieho prípadu a SAT riešiča.

CSV súbor obsahuje rovnaké polia, aké vytvára funkcia `_make_row`, napríklad:

- názov DIMACS súboru,
- názov SAT riešiča,
- index gramatiky,
- testované slovo,
- dĺžku slova,
- typ príkladu,
- počet premenných,
- počet klauzúl,
- výsledok riešenia,
- čas riešenia v milisekundách.

Formát CSV bol zvolený preto, že umožňuje jednoduché ďalšie spracovanie výsledkov, napríklad v tabuľkovom procesore alebo v skriptoch určených na štatistickú analýzu a vizualizáciu.

Výpis súhrnu výsledkov

Na rýchlu kontrolu výsledkov počas behu programu slúži funkcia:

```
def print_summary(rows: list[dict[str, str]]) -> None:
```

Vstupom je zoznam výsledkov benchmarku. Funkcia nevracia hodnotu, ale vypisuje zjednodušený prehľad výsledkov na štandardný výstup. Tento výpis obsahuje najdôležitejšie

údaje, ako názov riešiča, index gramatiky, dĺžku slova, počet premenných, počet klauzúl, výsledok riešenia, čas a názov súboru.

Spustenie benchmarku z príkazového riadku

Benchmarkovací modul obsahuje aj funkciu:

```
def main() -> int:
```

Táto funkcia umožňuje spustiť benchmark priamo z príkazového riadku. Vstupné parametre sa spracúvajú pomocou knižnice `argparse`.

Podporované argumenty sú:

- `grammar_file` – cesta ku gramatickému súboru,
- `--min-length` – minimálna dĺžka generovaných slov,
- `--max-length` – maximálna dĺžka generovaných slov,
- `--positive-count` – počet pozitívnych príkladov pre každú gramatiku,
- `--negative-count` – počet negatívnych príkladov pre každú gramatiku,
- `--java-jar` – cesta k Java generátoru slov.

Funkcia `main()` najprv skontroluje platnosť rozsahu dĺžok, existenciu vstupného gramatického súboru a existenciu Java JAR súboru. Následne nastaví logovanie, spustí funkciu `benchmark_from_grammar_file`, uloží výsledky do CSV súboru, vypíše súhrn a vráti návratový kód. Ak benchmark prebehne úspešne, funkcia vráti hodnotu `0`. Ak nastane chyba, napríklad neplatné vstupné údaje alebo chýbajúci súbor, funkcia vráti hodnotu `1`.

Logovanie priebehu benchmarku

Benchmark používa modul `logging`, ktorý zaznamenáva priebeh experimentu do logovacieho súboru aj na štandardný výstup. Logovací súbor sa vytvára v adresári:

```
text/results/<názov_gramatiky>/
```

a jeho názov má tvar:

```
<názov_gramatiky>_benchmark.log
```

Do logu sa zapisujú informácie o použitom gramatickom súbore, rozsahu generovaných slov, počte pozitívnych a negatívnych príkladov, použitom Java generátore, aktuálne spracovávanej gramatike, počte vygenerovaných slov a stave jednotlivých riešičov.

Logovanie je dôležité najmä pri dlhších benchmarkoch, pretože umožňuje spätne skontrolovať, či experiment prebehol korektne a kde prípadne nastala chyba.

Architektúra benchmarkovacieho procesu

Z hľadiska architektúry možno benchmarkovací proces opísať nasledovne:

```
gramatika → Java export → generovanie slov → CFG_2_SAT → DIMACS →
SAT riešiče → CSV záznam
```

Najprv sa vstupná gramatika spracuje parserom a zároveň sa vytvorí Java export. Tento export slúži ako vstup pre externý generátor slov. Vygenerované pozitívne a negatívne slová sa následne použijú na vytvorenie SAT inštancií pomocou triedy `CFG_2_SAT`. Výsledné CNF formuly sa uložia vo formáte DIMACS. Nad každým DIMACS súborom sa postupne spustia podporované SAT riešiče a výsledky sa uložia do CSV súboru.

Takto navrhnutý proces zabezpečuje, že všetky riešiče pracujú nad identickými formulami a že každý benchmarkovací prípad je reprodukovateľný. Zároveň sa oddeľuje generovanie dát, konštrukcia modelu a samotné riešenie, čo zvyšuje prehľadnosť celej implementácie.

Význam benchmarkovacej vrstvy

Benchmarkovacia vrstva má v kontexte tejto práce dve hlavné úlohy. Po prvé, umožňuje experimentálne porovnať správanie rôznych SAT riešičov na formulách vzniknutých redukciou parsovacieho problému. Po druhé, poskytuje systematický spôsob zberu dát o veľkosti formulí a časovej náročnosti riešenia.

Vďaka tomu benchmark nepredstavuje iba technický doplnok implementácie, ale dôležitú súčasť metodiky práce. Umožňuje overiť, do akej miery je navrhovaný prístup prakticky použiteľný a ako sa jeho správanie mení v závislosti od typu gramatiky, dĺžky vstupného slova, veľkosti výslednej formule a použitého SAT riešiča.

8.4.2 Porovnanie SAT prístupu a algoritmu CYK

Okrem benchmarku zameraného na porovnanie viacerých SAT riešičov implementujeme aj samostatný benchmark, ktorého cieľom je porovnať navrhovaný SAT prístup s klasickým algoritmom CYK pri riešení membership problému. Na rozdiel od predchádzajúceho benchmarku, ktorý spúšťa viacero SAT riešičov nad rovnakými DIMACS formulami, tento benchmark porovnáva dva odlišné prístupy k riešeniu toho istého rozhodovacieho problému.

Táto časť je implementovaná v zdrojovom súbore:

`src/benchmark_membership.py`

Benchmark opätovne využíva časti opísané v predchádzajúcej kapitole, najmä načítanie gramatík pomocou parsera, Java export gramatík a generovanie pozitívnych a negatívnych testovacích slov pomocou externého Java nástroja. Tieto kroky preto v tejto časti neopisujeme opakovane podrobne. Hlavný rozdiel spočíva v tom, že pre každý vygenerovaný testovací prípad sa nespúšťa množina rôznych SAT riešičov, ale dve vetvy výpočtu: SAT vetva a CYK vetva.

Hlavnou funkciou porovnávacieho benchmarku je:

```
def benchmark_from_grammar_file(
    grammar_file: str,
    min_length: int,
    max_length: int,
    positive_count: int,
    negative_count: int,
    java_jar_path: str,
    sat_solver_name: str,
) -> list[dict[str, str]]:
```

Vstupné parametre sú podobné ako pri benchmarku SAT riešičov. Parameter `grammar_file` určuje vstupný súbor s gramatikami, parametre `min_length` a `max_length` určujú rozsah dĺžok generovaných slov a parametre `positive_count` a `negative_count` určujú počet pozitívnych a negatívnych príkladov. Parameter `java_jar_path` označuje cestu k externému Java generátoru slov. Novým dôležitým parametrom je `sat_solver_name`, ktorým sa určuje konkrétny SAT riešič použitý v SAT vetve benchmarku.

Návratovou hodnotou funkcie je zoznam slovníkov. Každý slovník reprezentuje jeden výsledkový riadok. Keďže pre každé testované slovo sa vykonávajú dva výpočty, výsledný zoznam obsahuje pre každé slovo dva záznamy: jeden pre SAT prístup a jeden pre algoritmus CYK.

Rozdiel oproti benchmarku SAT riešičov

Zásadný rozdiel medzi oboma benchmarkmi je v tom, čo sa porovnáva. Benchmark SAT riešičov porovnáva viacero riešičov nad rovnakou CNF formulou uloženou v DIMACS

formáte. Jeho cieľom je zistiť, ktorý SAT riešič je najvhodnejší pre formule vzniknuté redukciou parsovacieho problému.

Porovnávací benchmark SAT prístupu a CYK algoritmu má iný cieľ. Neporovnáva viacero SAT riešičov medzi sebou, ale porovnáva celý navrhovaný SAT prístup s klasickým algoritmom CYK. Preto sa v tejto časti sleduje nielen samotné riešenie SAT formule, ale aj celkový čas SAT vetvy, ktorý zahŕňa vytvorenie objektu `CFG_2_SAT`, generovanie premenných, generovanie klauzúl a riešenie formule pomocou zvoleného SAT riešiča.

Ďalším rozdielom je, že v tomto benchmarku sa CNF formula neukladá do DIMACS súboru. Objekt `CFG_2_SAT` sa vytvára s parametrom:

```
save_dimacs=False
```

Formula sa teda rieši priamo v pamäti pomocou knižnice PySAT. Tento prístup je vhodnejší pre porovnanie s algoritmom CYK, pretože cieľom nie je testovať prácu s DIMACS súbormi, ale porovnať celkový čas oboch metód pri rozhodovaní, či dané slovo patrí do jazyka gramatiky.

SAT vetva benchmarku

SAT vetva je implementovaná funkciou:

```
def _run_sat_solver(grammar, word: str,
                    sat_solver_name: str
) -> tuple[str, str, float, dict[str, int]]:
```

Vstupom funkcie je objekt gramatiky, testované slovo a názov SAT riešiča. Funkcia najprv spustí meranie času pomocou `time.perf_counter()`. Následne vytvorí objekt:

```
CFG_2_SAT(grammar, word, solve=False, save_dimacs=False,
         verbose=False)
```

Tým sa vygenerujú základné premenné a klauzuly SAT modelu, ale formula sa neuloží do DIMACS súboru a nevypisujú sa pomocné informácie. Po vytvorení modelu sa štatistiky získajú volaním:

```
cfg_sat.get_stats()
```

Ak vstupné slovo nie je prázdne, z vygenerovaných klauzúl sa vytvorí objekt:

```
CNF(from_clauses=cfg_sat.clauses)
```

a formula sa vyrieši zvoleným PySAT backendom:

```
PySatSolver(name=sat_solver_name, bootstrap_with=cnf.clauses)
```

Návratovou hodnotou funkcie je štvorica:

```
(status, result, elapsed_ms, stats)
```

kde `status` označuje stav výpočtu, `result` obsahuje hodnotu `ACCEPT` alebo `REJECT`, `elapsed_ms` obsahuje čas výpočtu v milisekundách a `stats` obsahuje štatistiky vytvoreného SAT modelu.

CYK vetva benchmarku

Druhá vetva benchmarku používa klasický algoritmus CYK. Je implementovaná funkciou:

```
def _run_cyk(grammar, word)->tuple[str,str,float,dict[str, int]]:
```

Vstupom funkcie je rovnaký objekt gramatiky a rovnaké testované slovo ako pri SAT vetve. Funkcia najprv rozdelí slovo na tokeny a následne zavolá:

```
cyk_accepts(grammar, word_tokens)
```

Táto funkcia je implementovaná v súbore:

`src/modules/CYK.py`

Návratová hodnota funkcie `cyk_accepts` je logická hodnota, ktorá sa v benchmarku prevedie na jednotný výstup `ACCEPT` alebo `REJECT`. Výstupom funkcie `_run_cyk` je rovnako štvorica:

```
(status, result, elapsed_ms, stats)
```

Na rozdiel od SAT vetvy však štatistiky obsahujú iba základné údaje o gramatike, konkrétne počet neterminálov a počet pravidiel. Tieto údaje poskytuje pomocná funkcia:

```
def _grammar_stats(grammar) -> dict[str, int]:
```

Jednotná reprezentácia výsledkov

Aby bolo možné výsledky oboch prístupov ukladať do jedného CSV súboru, používa sa jednotná pomocná funkcia:

```
def _build_row(
    solver: str,
    grammar_index: int,
    word: str,
    positive_or_negative: str,
    status: str,
    result: str,
    elapsed_ms: float,
    stats: dict[str, int],
) -> dict[str, str]:
```

Táto funkcia vytvorí jeden výsledkový riadok. Pri SAT vetve má položka `solver` tvar napríklad:

```
sat:m22
```

Pri CYK vetve má hodnotu:

```
cyk
```

Týmto spôsobom možno v jednom CSV súbore rozlíšiť, ktorý riadok patrí SAT prístupu a ktorý algoritmu CYK. Výsledný riadok obsahuje index gramatiky, testované slovo, dĺžku slova, typ príkladu, základné štatistiky, stav výpočtu, výsledok a čas riešenia.

Výstup benchmarku

Výsledky sa ukladajú funkciou:

```
def write_csv(rows: list[dict[str,str]], output_csv: Path)-> None:
```

Na rozdiel od benchmarku SAT riešičov je štruktúra CSV súboru jednoduchšia, pretože cieľom nie je analyzovať veľkosť DIMACS formulí pre každý riešič, ale porovnať časové správanie dvoch prístupov. CSV súbor obsahuje stĺpce:

`solver, grammar, word, word_length, positive_or_negative, nonterminal_count, rule_count, status, result, time_ms`

Výstupný súbor sa ukladá pod názvom:

`<názov_gramatiky>_membership_solver_benchmark_results.csv`

do adresára:

`text/results/<názov_gramatiky>/`

Súčasťou implementácie je aj funkcia:

```
def print_summary(rows: list[dict[str, str]]) -> None:
```

ktorá vypisuje zjednodušený prehľad výsledkov na štandardný výstup.

Spustenie z príkazového riadku

Benchmark je možné spustiť aj priamo z príkazového riadku pomocou funkcie:

```
def main() -> int:
```

Táto funkcia spracúva vstupné argumenty pomocou knižnice `argparse`. Oproti benchmarku SAT riešičov obsahuje navyše parameter:

`--sat-solver`

ktorý určuje, ktorý PySAT backend sa použije pri SAT vetve. Predvolenou hodnotou je:

`m22`

Funkcia zároveň kontroluje existenciu vstupného súboru, platnosť rozsahu dĺžok generovaných slov a existenciu Java JAR súboru. Pri úspešnom dokončení vracia hodnotu `0`, pri chybe hodnotu `1`.

Architektúra porovnávacieho benchmarku

Architektúru tejto časti možno zapísať nasledovne:

$$\text{gramatika} \rightarrow \text{generovanie slov} \rightarrow \begin{cases} \text{CFG_2_SAT} \rightarrow \text{SAT riešič} \\ \text{CYK} \end{cases} \rightarrow \text{CSV záznam}$$

Najprv sa vstupná gramatika načíta a transformuje do CHNF. Následne sa pomocou rovnakého mechanizmu ako pri predchádzajúcom benchmarku vygenerujú pozitívne a negatívne testovacie slová. Každé slovo sa potom spracuje dvoma vetvami: SAT vetvou a CYK vetvou. Výsledky oboch vetiev sa zapíšu do rovnakého CSV súboru.

Hlavným prínosom tejto benchmarkovacej vrstvy je, že umožňuje porovnať navrhovaný SAT model s klasickým parsovacím algoritmom na rovnakých vstupných dátach. SAT prístup aj CYK algoritmus dostávajú rovnakú gramatiku a rovnaké slovo, takže výsledky možno priamo porovnávať z hľadiska správnosti aj časovej náročnosti.

9 Experimentálne vyhodnotenie

V tejto kapitole sa zameriavame na experimentálne vyhodnotenie navrhovaného prístupu založeného na redukcii parsovacieho problému na SAT problém. Cieľom tejto časti je analyzovať správanie implementovaného riešenia z hľadiska výpočtovej náročnosti, veľkosti generovaných formulí a efektivity ich riešenia pomocou rôznych SAT riešičov.

Experimenty sú navrhnuté tak, aby umožnili systematické porovnanie dvoch hlavných aspektov. Prvým je porovnanie viacerých SAT riešičov aplikovaných na rovnakú množinu CNF formulí, čím možno sledovať ich výkonnosť a vhodnosť pre daný typ problému. Druhým aspektom je porovnanie navrhovaného SAT prístupu s klasickým algoritmom CYK pri riešení membership problému, čo umožňuje posúdiť praktickú použiteľnosť redukcie parsovania na SAT.

Cieľom tejto kapitoly nie je len prezentovať namerané hodnoty, ale aj identifikovať hlavné faktory ovplyvňujúce výkonnosť navrhovaného prístupu. Osobitná pozornosť je venovaná vzťahu medzi veľkosťou generovanej CNF formule a časom riešenia, ako aj porovnaniu tohto správania s klasickým parsovacím algoritmom.

9.1 Metodológia experimentov

Pred samotnou analýzou výsledkov je potrebné opísať metodológiu experimentov, spôsob generovania testovacích dát, použitú množinu gramatík, počet vykonaných meraní a technické prostredie, v ktorom boli experimenty realizované. Cieľom experimentálnej časti bolo vyhodnotiť praktické správanie navrhovaného SAT prístupu pri riešení parsovacieho problému a zároveň porovnať výkonnosť viacerých SAT riešičov nad vygenerovanými CNF formulami.

V experimentoch sme použili štyri vstupné súbory s gramatikami: `g1`, `g2`, `grammars` a `big_grammar`. Súbory `g1`, `g2` a `big_grammar` obsahovali vždy jednu gramatiku. Súbor `grammars` obsahoval sedemnásť samostatných gramatík, ktoré boli v súbore oddelené samostatnými blokmi. Gramatika `big_grammar` predstavovala komplexnejší vstup a bola použitá aj pri porovnaní navrhovaného SAT prístupu s algoritmom CYK. Pre benchmark SAT riešičov sme každú gramatiku testovali na viacerých intervaloch dĺžok vstupných slov. Použité boli intervaly 1–10, 10–20, 20–30 a 30–50 tokenov. Pre každý interval a každú gramatiku bolo vygenerovaných desať pozitívnych a desať negatívnych testovacích slov. Pozitívne slová predstavovali reťazce patriace do jazyka danej gramatiky, zatiaľ čo negatívne slová predstavovali reťazce, ktoré daná gramatika nemá generovať.

Testovacie slová boli generované pomocou externého Java generátora slov [14]. Tento nástroj nebol vytvorený v rámci tejto práce, ale bol použitý ako pomocný komponent na prípravu vstupných dát pre experimenty. Parser preto okrem výstupu v CHNF vytváral aj export gramatiky vo formáte kompatibilnom s týmto generátorom. Vďaka tomu bolo možné automatizovane generovať pozitívne aj negatívne vstupy pre jednotlivé gramatiky a dĺžkové intervaly.

Pri benchmarku SAT riešičov sme porovnávali riešiče dostupné prostredníctvom knižnice PySAT, konkrétne Glucose3, Glucose4, Gluecard3, Gluecard4, Minisat22, MapleCM a MapleGH. Okrem nich sme použili aj riešič CryptoMiniSat prostredníctvom knižnice `pycryptosat`. Každý vygenerovaný testovací prípad bol najprv prevedený na CNF formulu a uložený do DIMACS formátu. Následne bola rovnaká DIMACS formula riešená všetkými použitými SAT riešičmi. Tým sme zabezpečili, že porovnanie riešičov prebiehalo nad identickými vstupnými formulami.

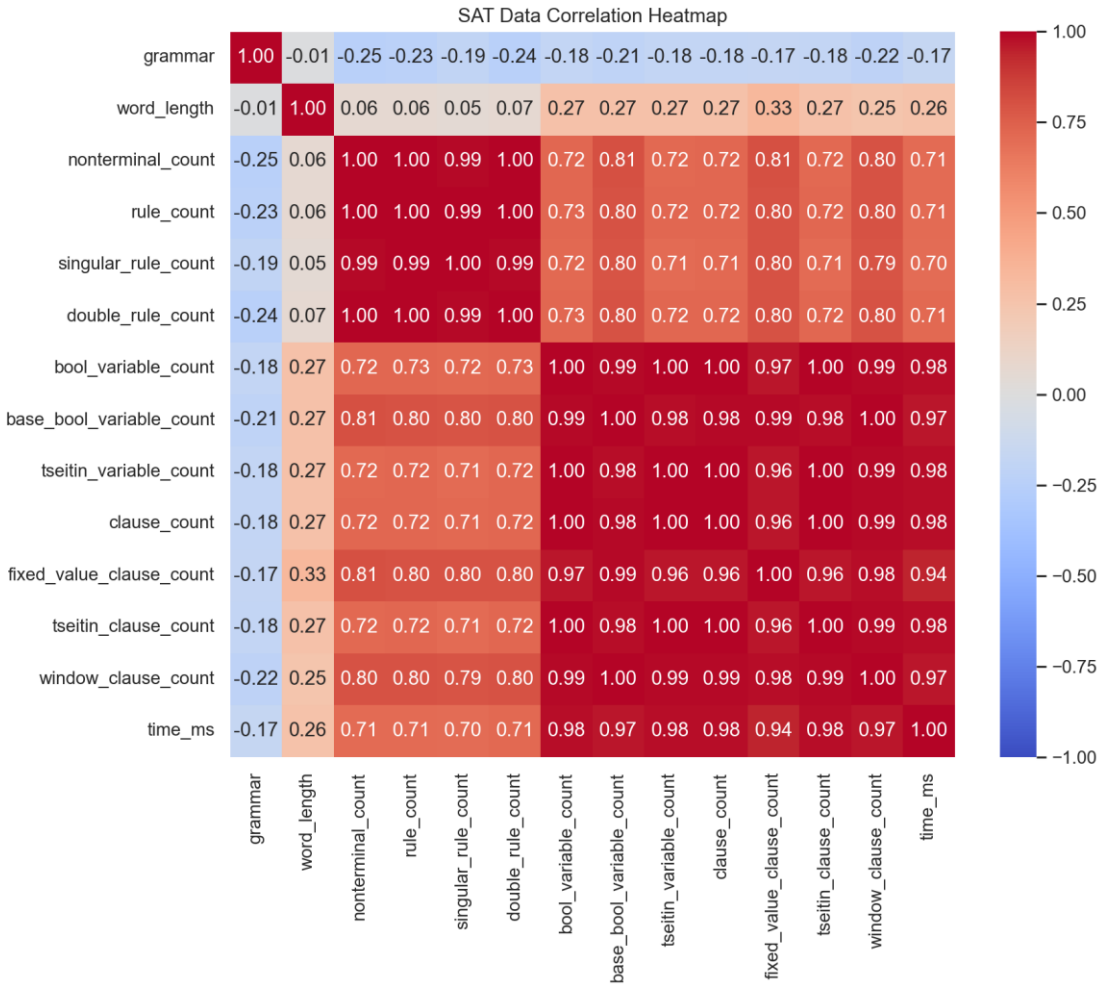
Okrem benchmarku SAT riešičov sme realizovali aj porovnanie navrhovaného SAT prístupu s algoritmom CYK. Na toto porovnanie bola použitá gramatika `big_grammar`, ktorá reprezentovala komplexnejší vstup. Testovanie prebehlo na intervale dĺžok 10–20, pričom bolo použitých desať pozitívnych a desať negatívnych slov. V SAT vetve bol použitý riešič `m22`, teda Minisat22 dostupný cez knižnicu PySAT. Pri tomto porovnaní sa pre každé slovo vykonal jeden beh SAT prístupu a jeden beh algoritmu CYK nad rovnakou gramatikou a rovnakým vstupným slovom.

Každá konkrétna kombinácia gramatiky, vstupného slova a SAT riešiča bola spustená jedenkrát. Nevykonávali sme viacnásobné opakovania rovnakého experimentu a pri vyhodnocovaní sme preto nepočítali priemery z opakovaných behov tej istej kombinácie. V grafoch a analýzach sme použili všetky namerané výsledky. Agregované hodnoty, napríklad pri porovnaní riešičov alebo pri analýze časov podľa dĺžky vstupu, boli počítané z dostupných nameraných záznamov v CSV súboroch. Výsledky experimentov boli ukladané do CSV súborov. Pri benchmarku SAT riešičov každý riadok reprezentoval jednu kombináciu gramatiky, vstupného slova a SAT riešiča. Pri porovnaní SAT prístupu a algoritmu CYK každý vstup generoval dva záznamy: jeden pre SAT vetvu a jeden pre CYK vetvu. Všetky tieto namerané hodnoty boli následne použité pri tvorbe grafov a pri analýze výsledkov v nasledujúcich častiach kapitoly.

Experimenty boli realizované na počítači s procesorom Intel(R) Core(TM) i5-9300H CPU @ 2.40GHz so štyrmi fyzickými jadrami a ôsmimi logickými procesormi. Počítač mal k dispozícii 16 GB operačnej pamäte RAM. Použitým operačným systémom bol Microsoft Windows 11 Home. Implementácia bola spúšťaná v prostredí Python 3.10. Pre prácu so SAT riešičmi bola použitá knižnica `python-sat` vo verzii 1.8.dev30 a riešič CryptoMiniSat bol dostupný prostredníctvom knižnice `pycryptosat` vo verzii 5.11.21.

9.2 Analýza korelácií medzi parametrami modelu

V tejto časti analyzujeme vzťahy medzi jednotlivými parametrami generovaného SAT modelu a časom riešenia pomocou korelačnej analýzy a grafických vizualizácií. Cieľom je identifikovať faktory, ktoré najvýznamnejšie ovplyvňujú výpočtovú náročnosť navrhovaného prístupu.

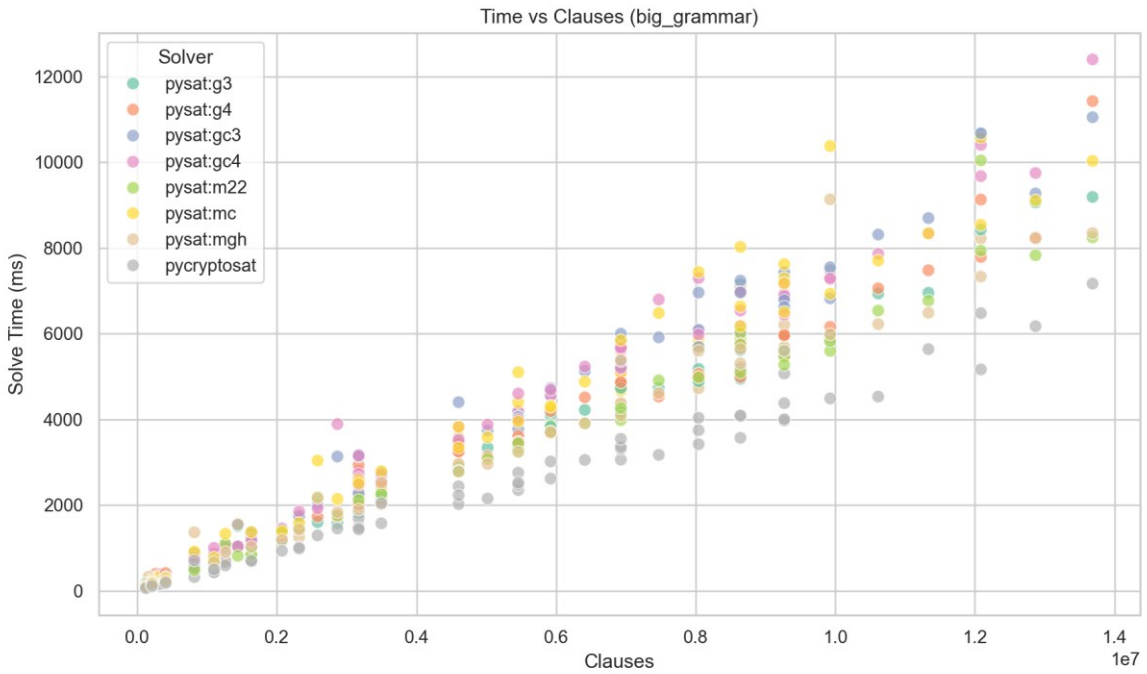


Obrázok 9-1 Korelácie medzi parametrami modelu

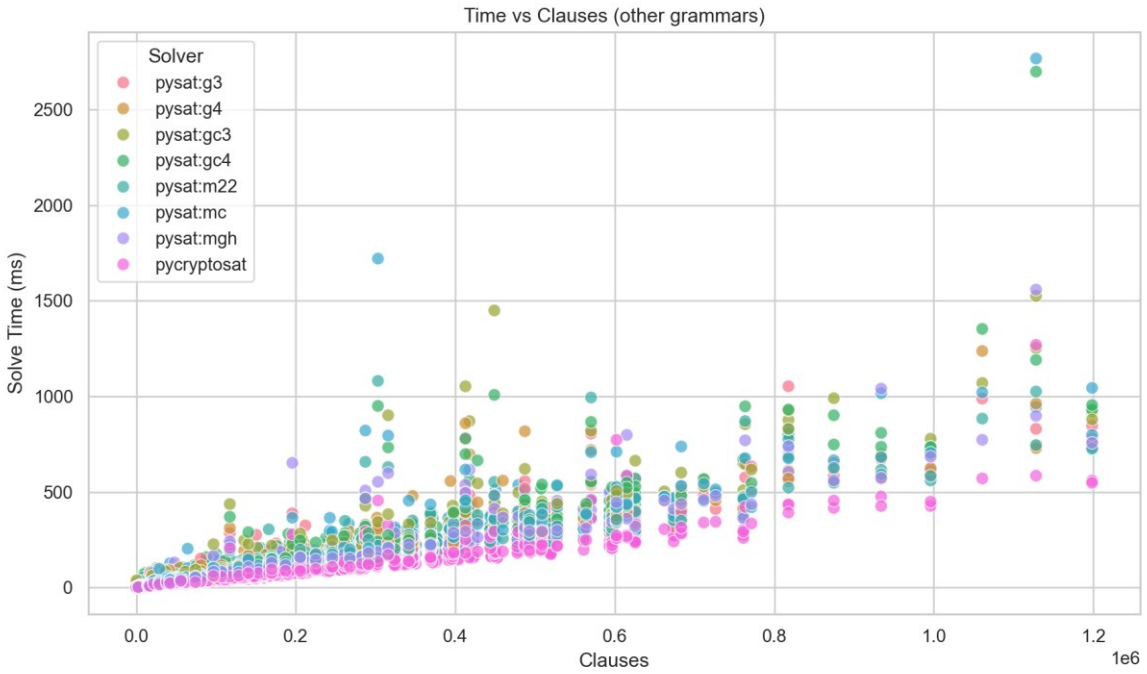
9.2.1 Vzťah medzi veľkosťou formule a časom riešenia

Najvýraznejším výsledkom je veľmi silná korelácia medzi časom riešenia a veľkosťou generovanej CNF formule. Konkrétne pozorujeme takmer lineárnu závislosť medzi časom riešenia a:

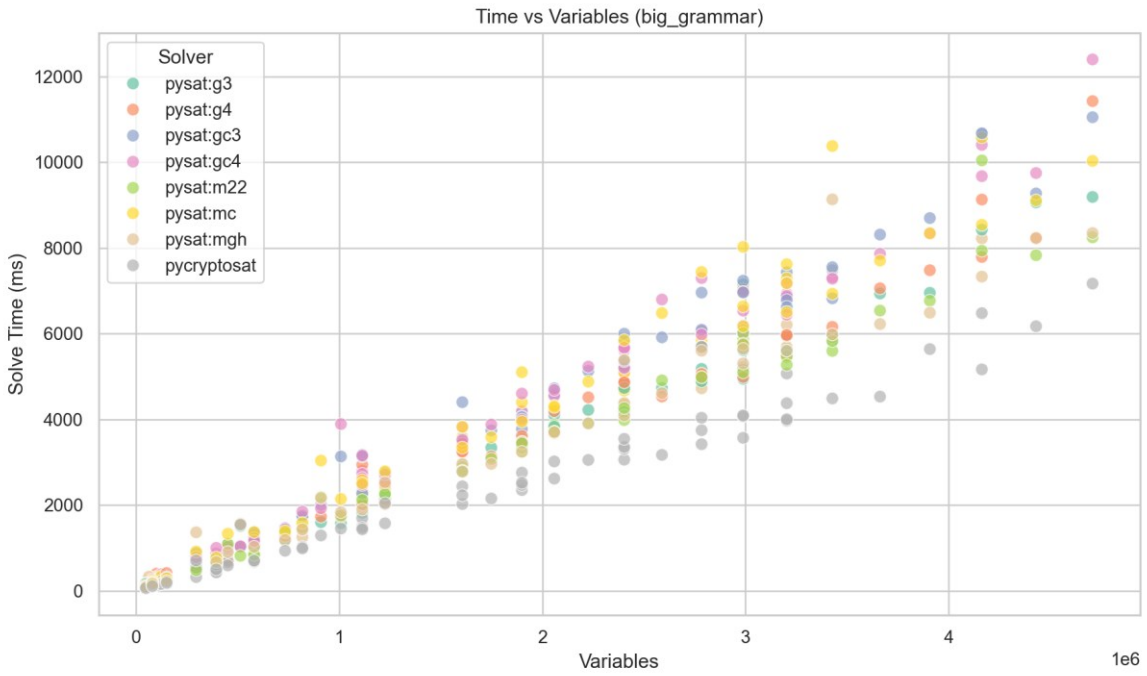
- počtom klauzúl,
- počtom booleovských premenných.



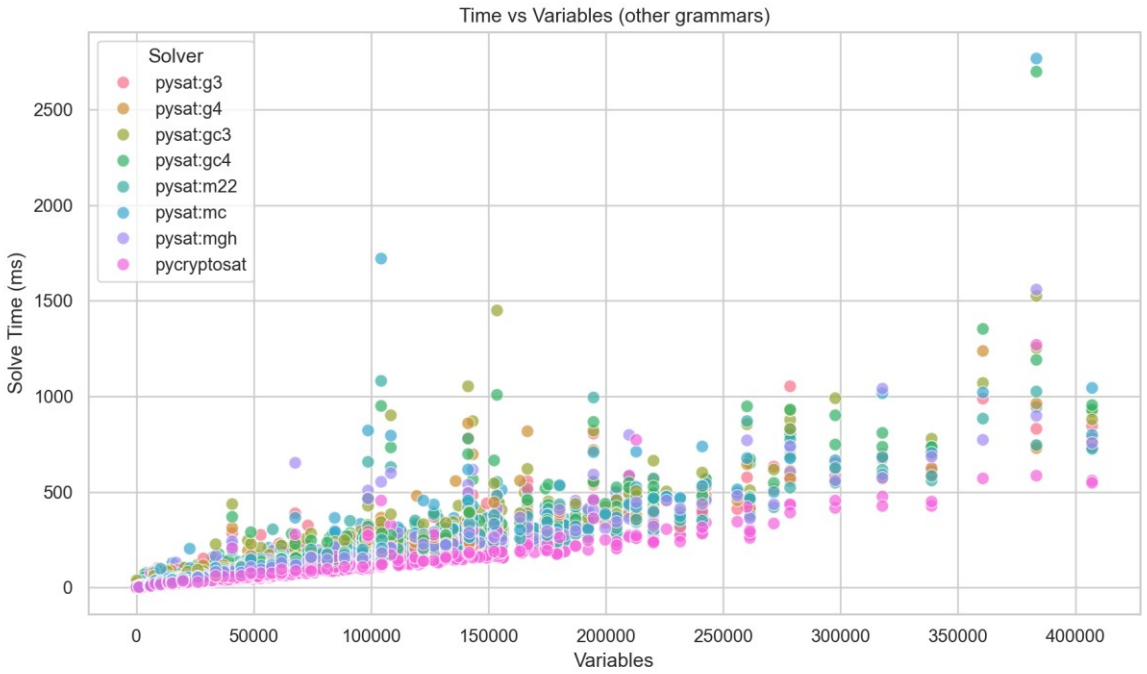
Obrázok 9-2 Závislosť času riešenia od počtu klauzúl pre gramatiku big_grammar pri jednotlivých SAT riešičoch.



Obrázok 9-3 Závislosť času riešenia od počtu klauzúl pre ostatné testované gramatiky pri jednotlivých SAT riešičoch.



Obrázok 9-4 Závislosť času riešenia od počtu booleovských premenných pre gramatiku big_grammar pri jednotlivých SAT riešičoch.



Obrázok 9-5 Závislosť času riešenia od počtu booleovských premenných pre ostatné testované gramatiky pri jednotlivých SAT riešičoch.

Táto závislosť naznačuje, že dominantným faktorom ovplyvňujúcim výkon SAT riešiča je veľkosť vytvoreného modelu. Inými slovami, čím väčšia je CNF formula, tým vyššia je výpočtová náročnosť riešenia. Formálne môžeme túto závislosť vyjadriť ako:

$$T \approx f(|V|, |C|)$$

kde T predstavuje čas riešenia, $|V|$ počet premenných a $|C|$ počet klauzúl.

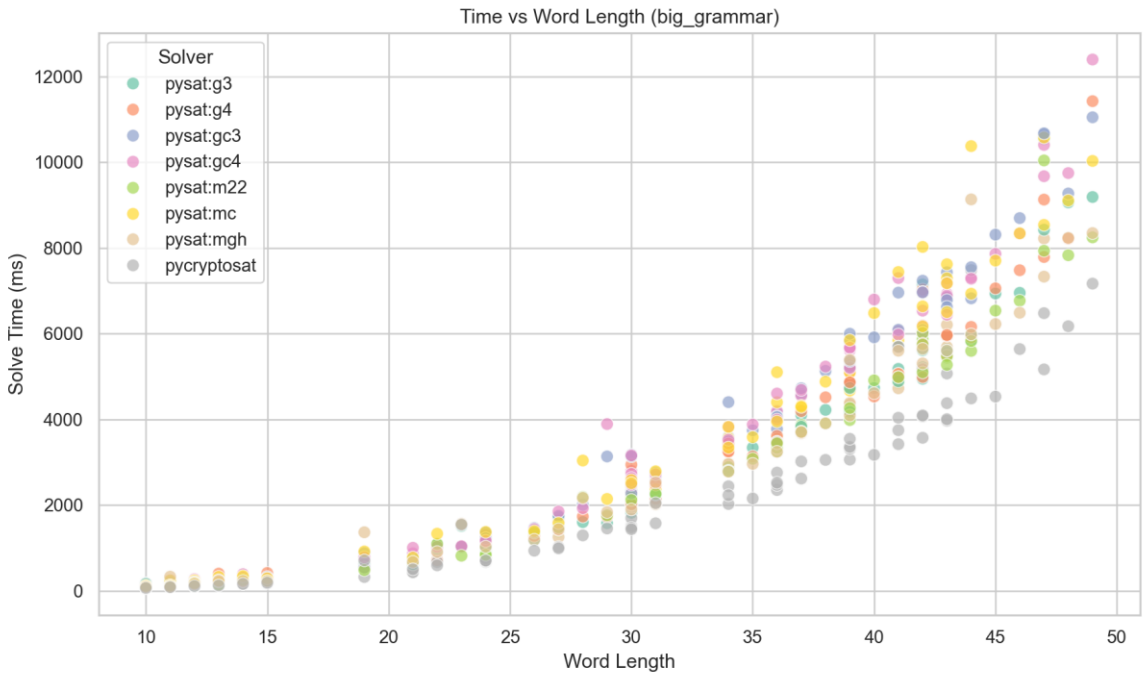
9.2.2 Vzťah medzi premennými a klauzulami

Medzi počtom premenných a počtom klauzúl pozorujeme takmer dokonalú koreláciu. To znamená, že tieto dve veličiny rastú spoločne a ich nárast je priamo spôsobený konštrukciou modelu.

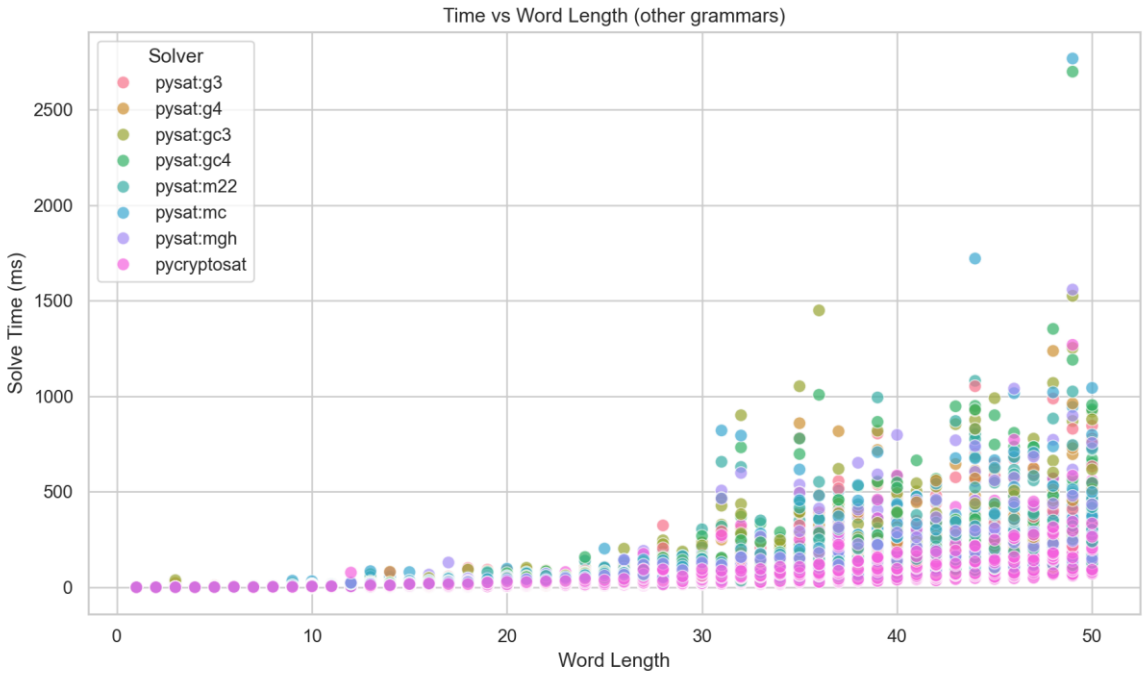
Tento výsledok je v súlade s implementáciou, kde každá kombinácia pravidla a rozdelenia reťazca generuje nové pomocné premenné (Tseitinove premenné) a zodpovedajúce klauzuly. Preto je veľkosť formule prirodzene úzko spätá s počtom premenných.

Vplyv gramatiky

Počet neterminálov a počet pravidiel vykazujú veľmi silnú vzájomnú koreláciu, čo je očakávané. Zároveň majú tieto parametre výrazný vplyv na veľkosť generovaného SAT modelu a tým aj na čas riešenia.



Obrázok 9-6 Závislosť času riešenia od dĺžky vstupného slova pre gramatiku big_grammar pri jednotlivých SAT riešičoch.



Obrázok 9-7 Závislosť času riešenia od dĺžky vstupného slova pre ostatné testované gramatiky pri jednotlivých SAT riešičoch.

Obrázok 9-8 Súhrnné porovnanie závislosti času riešenia od dĺžky vstupného slova pre gramatiku big_grammar a ostatné testované gramatiky.

Z grafov vyplýva, že rôzne gramatiky vedú k výrazne odlišným časom riešenia aj pri rovnakej dĺžke vstupného reťazca. Tento jav naznačuje, že štruktúra gramatiky – konkrétne počet pravidiel a možných kombinácií derivácií – má zásadný vplyv na náročnosť problému.

Vplyv dĺžky vstupného reťazca

Na prvý pohľad vykazuje dĺžka vstupného reťazca iba slabú koreláciu s časom riešenia. Pri detailnejšej analýze grafov však pozorujeme, že dáta netvoria jednu súvislú trendovú krivku, ale rozdeľujú sa do viacerých skupín (Obrázok 9-8).

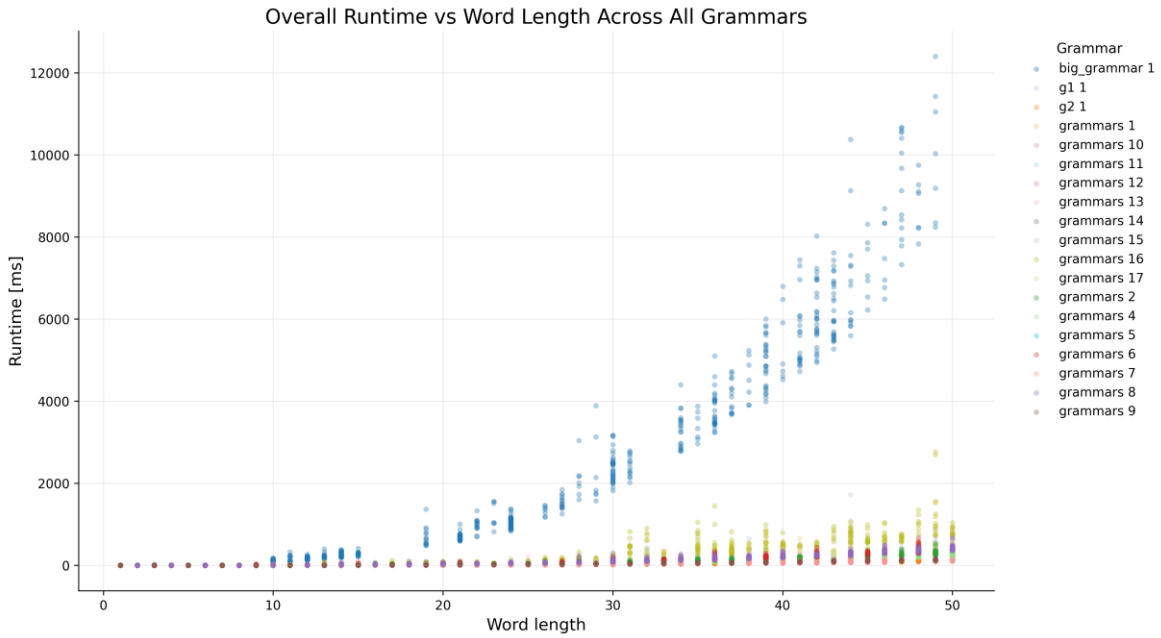
Tieto skupiny zodpovedajú rôznym gramatikám. Pre každú konkrétnu gramatiku možno pozorovať rastúci trend medzi dĺžkou reťazca a časom riešenia, avšak pri zmiešaní všetkých gramatík do jedného datasetu dochádza k prekrytiu viacerých trendov.

Tento efekt spôsobuje, že celkový korelačný koeficient medzi dĺžkou reťazca a časom riešenia je relatívne nízky. Nejde však o dôkaz, že dĺžka reťazca nemá vplyv, ale o dôsledok heterogénnosti dát spôsobenej rozdielnou štruktúrou gramatík.

Vplyv štruktúry gramatiky na vzťah medzi dĺžkou vstupu a časom riešenia

Pri celkovom vyhodnotení všetkých gramatík spolu sa ukázalo, že korelácia medzi dĺžkou vstupného slova a časom riešenia je pomerne nízka. Celkový korelačný koeficient

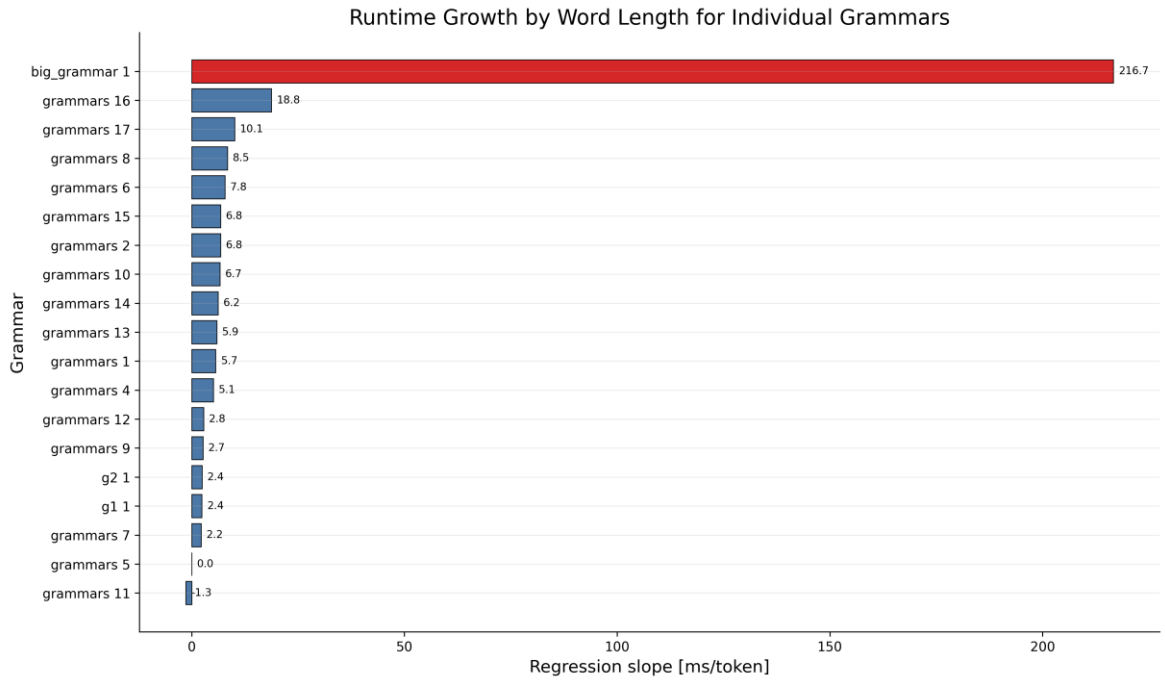
dosiahol hodnotu približne 0,2623. Tento výsledok by na prvý pohľad mohol naznačovať, že dĺžka vstupu nemá výrazný vplyv na čas riešenia. Detailnejšia analýza podľa jednotlivých gramatík však ukazuje, že takáto interpretácia by nebola správna.



Obrázok 9-9 Závislosť času riešenia od dĺžky vstupného slova pri spoločnom zobrazení všetkých gramatík.

Hlavným dôvodom nižšej globálnej korelácie je heterogénnosť testovaných gramatík. Jednotlivé gramatiky nemajú rovnakú štruktúru, rovnaký počet pravidiel ani rovnaký počet možných odvodení pre dané intervaly vstupného slova. Preto dve slová rovnakej dĺžky nemusia viesť k podobne veľkej SAT formule. Pri jednoduchej gramatike môže aj dlhšie slovo vytvoriť relatívne malý počet premenných a klauzúl, zatiaľ čo pri komplexnejšej gramatike môže aj porovnateľne dlhý vstup viesť k výrazne väčšej formule. Z tohto dôvodu sa pri spoločnom zobrazení všetkých gramatík zmiešavajú dáta s odlišnými trendmi, čo znižuje celkový korelačný koeficient.

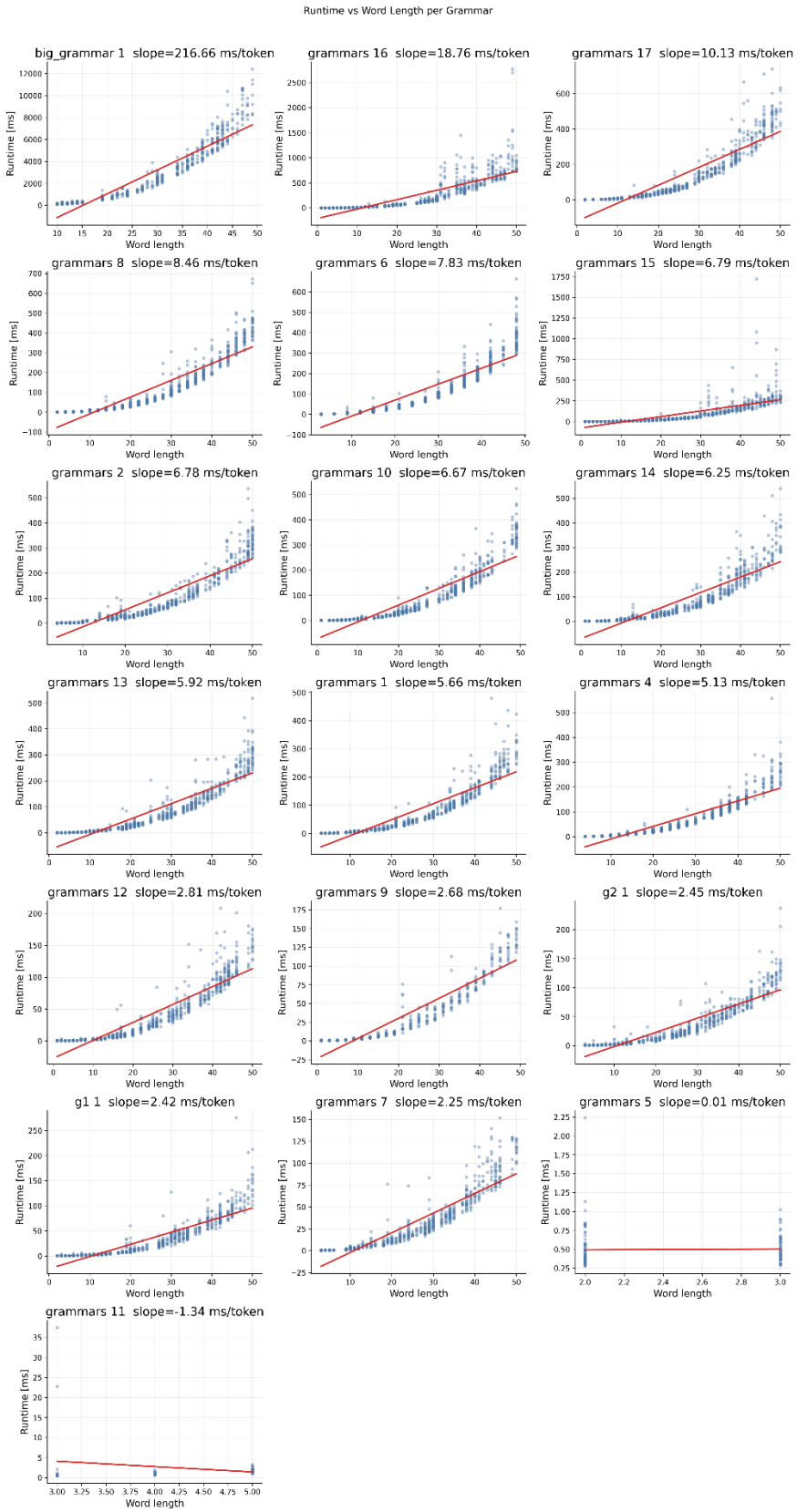
Na kvantifikáciu tohto efektu bol pre každú gramatiku samostatne vypočítaný lineárny trend medzi dĺžkou vstupného slova a časom riešenia. Smernica tohto trendu vyjadruje, o koľko milisekúnd sa priemerne zvýši čas riešenia pri predĺžení vstupu o jeden token. Výsledky ukazujú, že rozdiely medzi gramatikami sú výrazné. Najvyšší nárast bol nameraný pri gramatike `big_grammar`, ktorej smernica dosiahla približne 216,6574ms/token.



Obrázok 9-10 Porovnanie nárastu času riešenia podľa jednotlivých gramatík.

Tento výsledok potvrdzuje, že samotná dĺžka vstupného slova nie je jediným faktorom ovplyvňujúcim čas riešenia. Dôležitú úlohu zohráva aj štruktúra gramatiky, najmä počet neterminálov, počet binárnych pravidiel a počet možných spôsobov rozdelenia intervalov pri generovaní klauzúl. Pri komplexnejších gramatikách vzniká pre rovnakú dĺžku vstupu výrazne viac možností odvodenia, čo vedie k väčšiemu počtu Tseitinových premenných, väčšiemu počtu klauzúl a následne aj k vyššiemu času riešenia.

Rozdiel medzi globálnym a individuálnym pohľadom je dobre viditeľný aj pri samostatnom zobrazení trendov pre jednotlivé gramatiky. Kým spoločný graf všetkých gramatík pôsobí rozptýlene, pri mnohých jednotlivých gramatikách je vzťah medzi dĺžkou vstupu a časom riešenia výrazne silnejší. Viaceré gramatiky dosahujú samostatné korelácie približne v rozsahu 0,8 až 0,94, čo znamená, že v rámci konkrétnej gramatiky má dĺžka vstupu významný vplyv na čas riešenia.



Obrázok 9-11 : Závislosť času riešenia od dĺžky vstupného slova zobrazená samostatne pre jednotlivé gra-matiky.

Z uvedených výsledkov vyplýva, že nízky celkový korelačný koeficient nie je dôkazom toho, že dĺžka vstupu nemá vplyv na čas riešenia. Skôr ide o dôsledok toho, že experimentálne dáta obsahujú gramatiky s veľmi rozdielnou štruktúrou a rozdielnou citlivosťou na rast dĺžky vstupu. Pri jednoduchších gramatikách môže byť nárast času veľmi mierny, zatiaľ čo pri komplexnejších gramatikách, najmä pri `big_grammar`, rastie čas riešenia podstatne rýchlejšie. Táto analýza preto ukazuje, že pri hodnotení efektivity SAT prístupu je potrebné zohľadniť nielen dĺžku vstupného slova, ale aj štruktúrnu zložitosť samotnej gramatiky.

9.2.3 Interpretácia výsledkov

Na základe vykonanej analýzy môžeme formulovať niekoľko kľúčových záverov:

- čas riešenia je silne závislý od veľkosti CNF formule,
- štruktúra gramatiky predstavuje hlavný faktor ovplyvňujúci veľkosť modelu,
- dĺžka vstupného reťazca má významný vplyv v rámci konkrétnej gramatiky, avšak tento vplyv je oslabený pri agregovanej analýze,
- celková výpočtová náročnosť je výsledkom kombinácie dĺžky vstupu a komplexity gramatiky.

Zhrnutie

Výsledky ukazujú, že navrhovaný prístup je citlivý najmä na veľkosť generovaného SAT modelu, ktorá je priamo ovplyvnená štruktúrou gramatiky. Dĺžka vstupného reťazca zohráva úlohu, avšak jej vplyv je sekundárny v porovnaní s komplexitou gramatiky.

Z toho vyplýva, že prípadná optimalizácia algoritmu by sa mala zamerať predovšetkým na redukciu veľkosti generovanej formule, napríklad obmedzením počtu generovaných kombinácií alebo efektívnejšou reprezentáciou pravidiel.

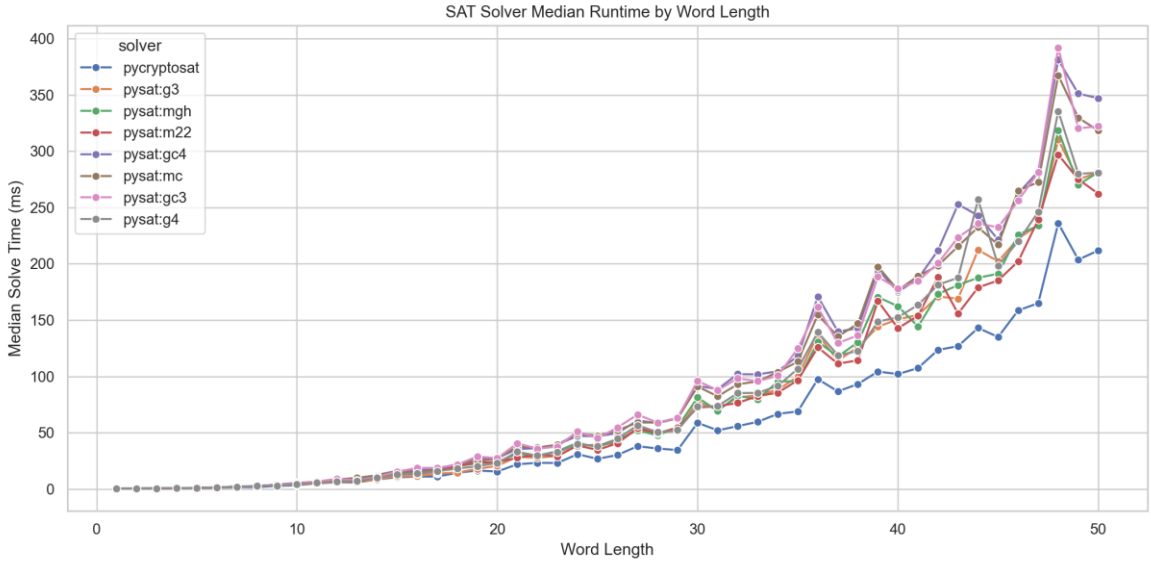
9.3 Porovnanie SAT riešičov

V tejto časti analyzujeme výkonnosť rôznych SAT riešičov aplikovaných na CNF formulách vzniknutých redukciou parsovacieho problému. Cieľom je identifikovať rozdiely v správaní jednotlivých riešičov a posúdiť ich vhodnosť pre daný typ problému.

Závislosť času riešenia od dĺžky vstupu

Z prvého grafu vyplýva, že všetky testované SAT riešiče vykazujú podobný trend rastu času riešenia v závislosti od dĺžky vstupného reťazca. So zvyšujúcou sa dĺžkou reťazca

dochádza k postupnému nárastu času riešenia, čo je v súlade s rastúcou veľkosťou generovanej CNF formule.



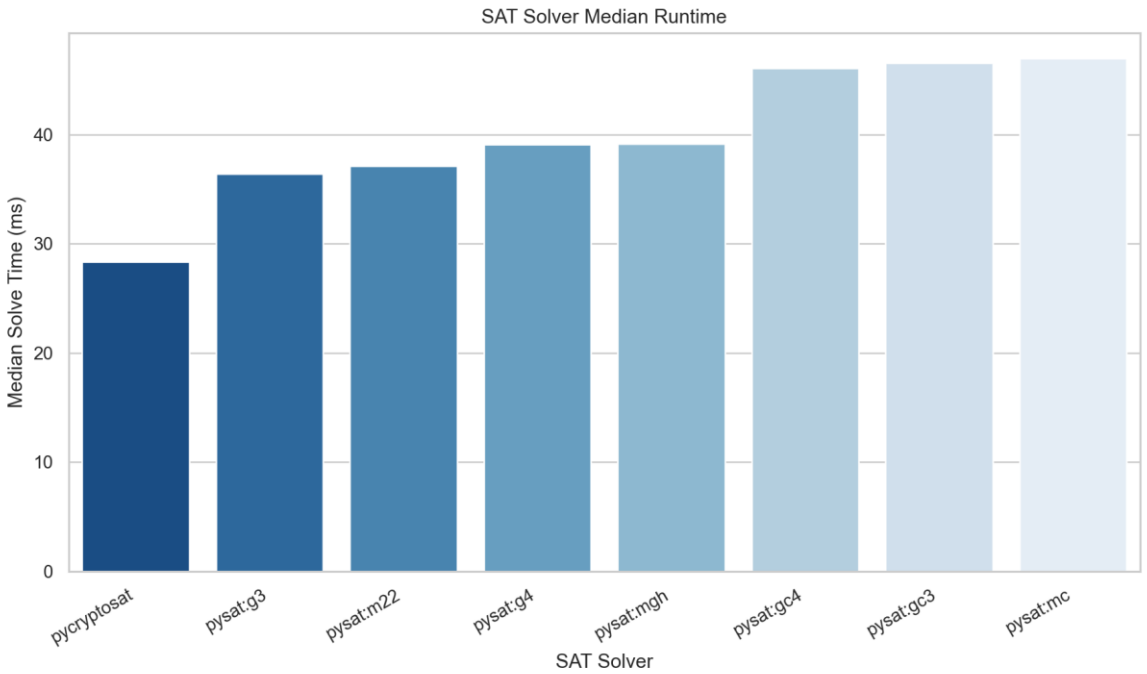
Obrázok 9-12 Závislosť mediánového času riešenia od dĺžky vstupného reťazca pre jednotlivé SAT riešiče

Aj keď trend rastu je spoločný pre všetky riešiče, medzi nimi možno pozorovať rozdiely v absolútnych hodnotách času. Niektoré riešiče dosahujú systematicky nižšie časy, zatiaľ čo iné vykazujú vyššiu výpočtovú náročnosť najmä pri väčších vstupoch.

Zaujímavé je, že rozdiely medzi riešičmi sa zväčšujú s rastúcou dĺžkou vstupu. Pri krátkych reťazcoch sú časy riešenia veľmi podobné, avšak pri väčších inštanciách sa výkonnosť jednotlivých riešičov začína výraznejšie diverzifikovať.

Porovnanie mediánových časov riešenia

Druhý graf zobrazuje mediánový čas riešenia pre jednotlivé SAT riešiče naprieč všetkými testovacími prípadmi. Tento pohľad umožňuje eliminovať vplyv extrémnych hodnôt a poskytuje stabilnejšie porovnanie výkonu.



Obrázok 9-13 Porovnanie mediánového času riešenia jednotlivých SAT riešičov naprieč všetkými testovacími prípadmi

Z grafu vyplýva, že najlepšie výsledky dosahuje riešič **pycryptosat**, ktorý vykazuje najnižší mediánový čas riešenia. Ostatné riešiče dosahujú mierne vyššie hodnoty, pričom rozdiely medzi nimi nie sú extrémne veľké, ale sú konzistentné.

Interpretácia výsledkov

Na základe vykonanej analýzy možno formulovať nasledujúce závery:

- všetky riešiče vykazujú podobný rastový trend v závislosti od veľkosti vstupu,
- rozdiely medzi riešičmi sa prejavujú najmä pri väčších inštanciách,
- mediánový čas riešenia poskytuje stabilný ukazovateľ výkonu,
- riešič **pycryptosat** dosahuje najlepšie výsledky v rámci testovanej množiny,
- rozdiely medzi jednotlivými PySAT backendmi sú relatívne malé, avšak konzistentné.

Diskusia

Výsledky naznačujú, že výber konkrétneho SAT riešiča môže mať významný vplyv na celkový čas riešenia, najmä pri väčších formulách. Napriek tomu je potrebné zdôrazniť, že rozdiely medzi riešičmi sú menšie než rozdiely spôsobené samotnou veľkosťou formulí.

To znamená, že optimalizácia generovania CNF modelu má potenciálne väčší dopad na výkon než výber konkrétneho riešiča. Výber vhodného riešiča však môže predstavovať dodatočné zlepšenie výkonu, najmä pri praktickom nasadení.

Zhrnutie

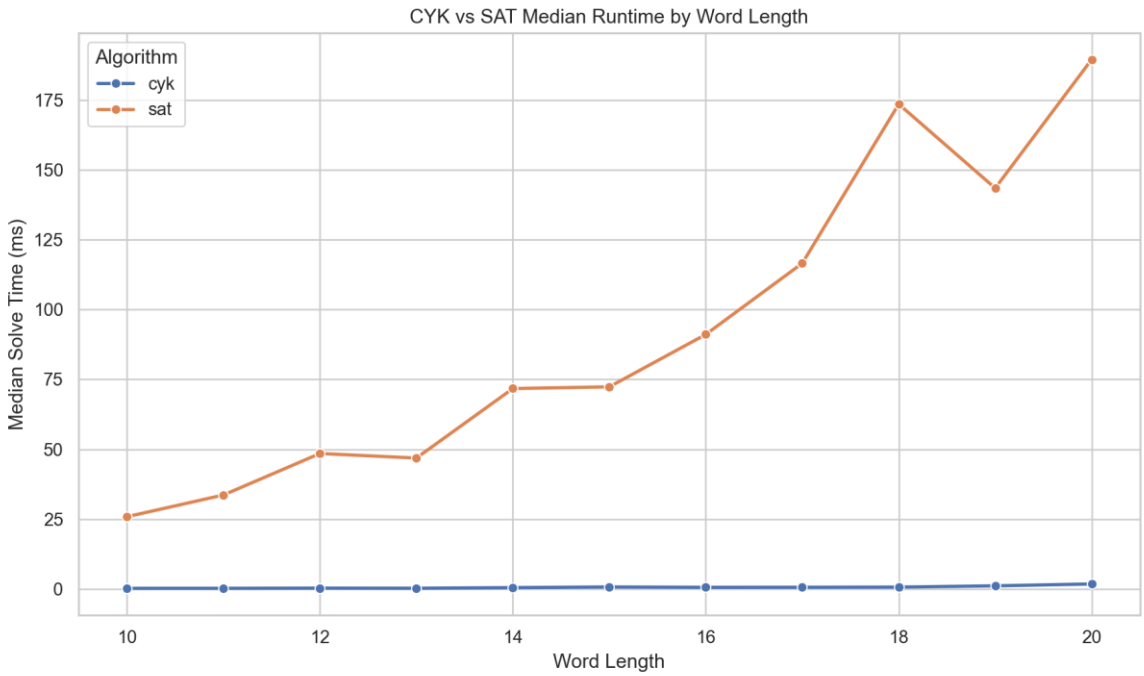
Porovnanie SAT riešičov ukazuje, že navrhovaný prístup je kompatibilný s rôznymi modernými riešičmi a nevykazuje závislosť od konkrétnej implementácie. Zároveň však výsledky potvrdzujú, že niektoré riešiče sú vhodnejšie pre daný typ problému než iné, čo je potrebné zohľadniť pri praktickom použití.

9.4 Porovnanie SAT prístupu a algoritmu CYK

V tejto časti porovnávame navrhovaný SAT prístup s klasickým algoritmom CYK pri riešení membership problému. Cieľom je posúdiť praktickú efektivitu redukcie parsovania na SAT v porovnaní s tradičným dynamickým programovacím prístupom. Experiment bol vykonaný na gramatike big_grammar v intervale dĺžok vstupných slov $10–20$, pričom pre každú dĺžku boli použité pozitívne aj negatívne testovacie prípady.

Závislosť času riešenia od dĺžky vstupu

Na grafe pozorujeme výrazný rozdiel v správaní oboch prístupov. Algoritmus CYK vykazuje veľmi nízke a stabilné časy riešenia naprieč celým rozsahom dĺžok vstupného reťazca. Rast času je mierny a prakticky lineárny v rámci testovaného rozsahu.

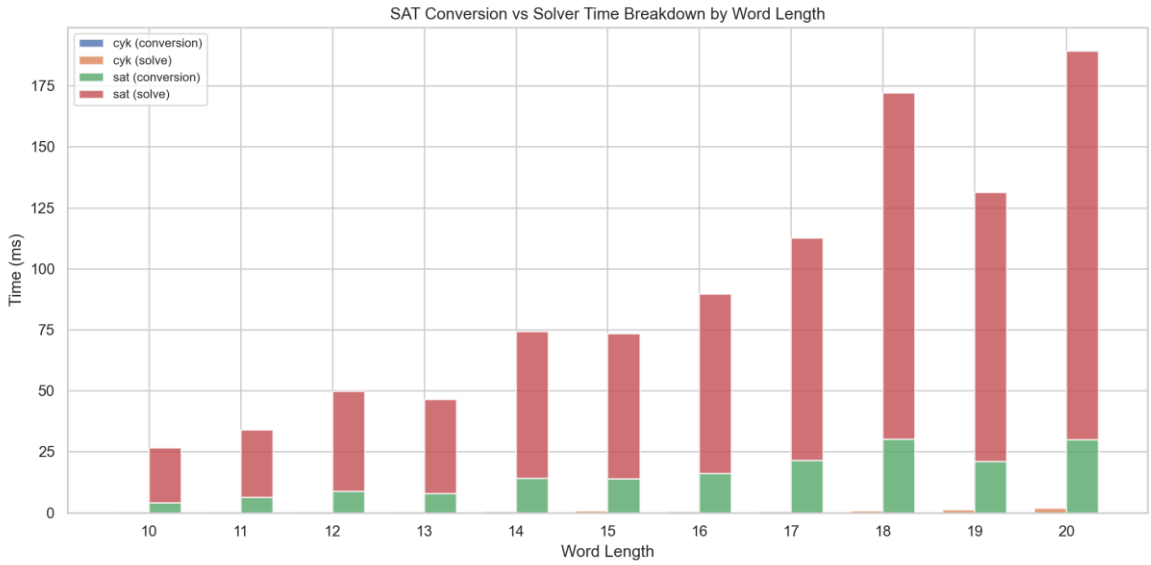


Obrázok 9-14 Porovnanie mediánu času riešenia algoritmu CYK a SAT prístupu v závislosti od dĺžky vstupného slova pre gramatiku big_grammar.

Naopak, SAT prístup vykazuje výrazne vyššiu časovú náročnosť. Už pri kratších vstupoch je jeho čas vyšší než pri algoritme CYK a so zvyšujúcou sa dĺžkou vstupného slova rastie podstatne rýchlejšie. Pri dĺžke vstupu približne 10 tokenov sa SAT prístup pohybuje v desiatkach milisekúnd, zatiaľ čo CYK zostáva blízko nulovej hodnoty. Pri dĺžke 20 tokenov dosahuje SAT prístup hodnoty približne v stovkách milisekúnd, zatiaľ čo CYK zostáva stále výrazne rýchlejší.

Tento rozdiel však nemožno interpretovať iba ako rozdiel medzi SAT riešičom a CYK algoritmom. SAT prístup totiž pozostáva z dvoch častí: najprv je potrebné skonštruovať CNF formulu a až následne ju odovzdať SAT riešiču. Preto je vhodné pozrieť sa aj na rozklad celkového času SAT prístupu.

Rozklad času SAT prístupu



Obrázok 9-15 Rozklad času SAT prístupu na čas konštrukcie formule a čas samotného riešenia v porovnaní s algoritmom CYK.

Naopak, SAT prístup vykazuje výrazne vyššiu výpočtovú náročnosť. Čas riešenia rastie podstatne rýchlejšie so zvyšujúcou sa dĺžkou vstupu a pri väčších hodnotách dosahuje rádovo vyššie hodnoty než CYK algoritmus.

Kvantitatívne porovnanie

Pri menších vstupoch (napr. dĺžka reťazca okolo 10) je SAT prístup približne o jeden rád pomalší než CYK. S rastúcou dĺžkou vstupu sa tento rozdiel ďalej zväčšuje. Pri väčších vstupoch (napr. dĺžka okolo 20) dosahuje čas riešenia SAT prístupu niekoľkonásobne vyššie hodnoty než CYK.

Zatiaľ čo CYK algoritmus spracúva vstupy v milisekundách s minimálnym nárastom, SAT prístup vykazuje výrazný rast až do stoviek milisekúnd. Tento rozdiel je spôsobený najmä tým, že SAT prístup vyžaduje najprv konštrukciu CNF formule a následne jej riešenie pomocou všeobecného SAT riešiča.

Interpretácia výsledkov

Rozdiel v správaní oboch prístupov možno vysvetliť ich teoretickými vlastnosťami. CYK algoritmus má deterministickú časovú zložitosť $O(n^3)$, pričom pracuje priamo nad tabuľkou veľkosti $n \times n$.

Navrhovaný SAT prístup síce vychádza z podobnej dynamickej štruktúry, avšak túto štruktúru transformuje do booleovskej formule. Výsledná CNF formula obsahuje veľké množstvo premenných a klauzúl, čo vedie k výrazne vyššej výpočtovej náročnosti pri jej riešení.

Zároveň je potrebné zdôrazniť, že SAT riešič rieši všeobecný problém splniteľnosti, čo je výrazne zložitejšia úloha než špecializovaný parsovací algoritmus. Preto je očakávané, že v základnom nastavení bude SAT prístup menej efektívny než CYK.

Význam výsledkov

Aj napriek vyššej výpočtovej náročnosti má SAT prístup svoje výhody. Umožňuje flexibilné rozšírenie modelu o dodatočné obmedzenia, jednoduché formulovanie rôznych variantov problému a využitie pokročilých heuristík moderných SAT riešičov.

Výsledky preto neznamenajú, že SAT prístup je nevhodný, ale skôr to, že jeho využitie je opodstatnené najmä v situáciách, kde je potrebná vyššia flexibilita alebo kombinácia s inými obmedzeniami.

Záver

Cieľom tejto práce bolo preskúmať možnosť riešenia parsovacieho problému pre bezkontextové gramatiky prostredníctvom redukcie na SAT problém. V práci sme sa zamerali na prepojenie teórie formálnych jazykov s modernými prístupmi založenými na riešení problémov splniteľnosti Booleovských formulí. V teoretickej časti sme najprv predstavili základné pojmy formálnych jazykov a bezkontextových gramatík, ako aj klasické prístupy k parsovaniu, najmä algoritmus CYK. Následne sme sa venovali SAT problému, jeho vlastnostiam a významu v súčasnej informatike. Osobitnú pozornosť sme venovali princípu redukcie problémov a motivácii použitia SAT prístupu pri riešení parsovania. Na základe existujúcich prác, najmä prístupu navrhnutého Axelsson et al. [1], sme implementovali vlastné riešenie, ktoré transformuje bezkontextovú gramatiku v Chomského normálnej forme na konjunktívnu normálnu formu vhodnú pre SAT riešič. Navrhnutá implementácia zahŕňa kompletný pipeline spracovania – od parsovania vstupnej gramatiky, cez jej transformáciu, až po generovanie DIMACS reprezentácie a riešenie pomocou SAT riešiča. Dôležitou súčasťou riešenia je aj schopnosť rekonštrukcie derivačného stromu, čím sa rozširuje využiteľnosť prístupu aj na konštrukčný variant parsovania. V experimentálnej časti sme analyzovali správanie implementovaného riešenia z viacerých hľadísk. Najprv sme porovnali výkonnosť rôznych SAT riešičov, pričom sme zistili, že ich správanie je vo všeobecnosti podobné a rozdiely sa prejavujú najmä pri väčších inštanciách. Následne sme analyzovali vzťah medzi parametrami modelu a časom riešenia, pričom sa ukázalo, že dominantným faktorom ovplyvňujúcim výpočtovú náročnosť je veľkosť generovanej CNF formule. Táto veľkosť je priamo závislá od štruktúry gramatiky a počtu možných derivácií. Kľúčovým výsledkom práce je porovnanie navrhovaného SAT prístupu s klasickým algoritmom CYK. Experimenty ukázali, že CYK algoritmus dosahuje výrazne lepšie časové výsledky pri riešení membership problému. Na druhej strane SAT prístup ponúka vyššiu flexibilitu, keďže umožňuje jednoduché rozšírenie modelu o dodatočné obmedzenia a riešenie komplexnejších variantov problému. Z tohto pohľadu možno SAT prístup vnímať nie ako náhradu klasických parsovacích algoritmov, ale ako alternatívny nástroj vhodný pre špecifické typy úloh.

Na základe získaných výsledkov možno konštatovať, že redukcia parsovacieho problému na SAT je prakticky realizovateľná a poskytuje zaujímavý pohľad na riešenie tohto problému. Zároveň však prináša aj určité obmedzenia, najmä v podobe veľkosti generovanej formule a s tým súvisiacej výpočtovej náročnosti. Záverom možno povedať, že hoci SAT prístup nedosahuje výkon klasických algoritmov pri základnom parsovaní, predstavuje flexibilný a univerzálny rámec, ktorý má potenciál uplatniť sa pri riešení komplexnejších problémov nad bezkontextovými gramatikami.

Literatúra

[1] AXELSSON, R. et al. Analyzing Context-Free Grammars Using an Incremental SAT Solver. In *Automata, Languages and Programming* . Berlin, Heidelberg: Springer Berlin Heidelbergs. 410–422. .

[2] HOPCROFT, J.E.. et al. *Introduction to automata theory, languages, and computation*. . [s.l.]: Pearson Education : Dorling Kindersley, 2008. ISBN 9788131720479.

[3] SIPSER, M. *Introduction to the theory of computation*. . [s.l.]: Cengage Learning, 2013. 458 s. ISBN 113318779X.

[4] AHO, A.V.. et al. *Compilers : principles, techniques, & tools*. . [s.l.]: Pearson/Addison Wesley, 2007. 1009 s. ISBN 0321486811.

[5] CHOMSKY, N. On certain formal properties of grammars. In *Information and control* . 1959. Vol. 2, no. 2, s. 137–167. .

[6] GRUNE, D. - JACOBS, C.J.H.. *Parsing techniques : a practical guide*. . [s.l.]: Springer, 2008. 662 s. ISBN 9780387202488.

[7] JURAFSKY, D. - MARTIN, J.H. *Speech and Language Processing: An Introduction toNatural Language Processing, Computational Linguistics,and Speech Recognition with Language Models* [online]. . 3rd. vyd. 2026. .

[8] INTERNATIONAL, E. Standard ECMA-404: The JSON Data Interchange Syntax. In . 2017. .

[9] BIERE, A. et al. *Handbook of satisfiability*. . [s.l.]: IOS Press, 2009. 966 s. ISBN 160750376X.

[10] COOK, S.A. The complexity of theorem-proving procedures. In *Logic, automata, and computational complexity: The works of Stephen A. Cook* . 2023. s. 143–152. .

[11] KARP, R.M. Reducibility among combinatorial problems. In *50 Years of Integer Programming 1958-2008: from the Early Years to the State-of-the-Art* . [s.l.]: Springer, 2009. s. 219–241. .

[12] MARQUES-SILVA, J.P. - SAKALLAH, K.A. GRASP: A search algorithm for propositional satisfiability. In *IEEE Transactions on computers* . 2002. Vol. 48, no. 5, s. 506–521. .

[13] IMADA, K. - NAKAMURA, K. Learning Context Free Grammars by Using SAT Solvers. In *2009 International Conference on Machine Learning and Applications* . [s.l.]: IEEE, 2009. s. 267–272. .

[14] REVIĽÁKOVÁ, A. [online]. .2023. Zverejnená bakalárska práca. Použitá ako zdroj externého Java nástroja WordGenerator na generovanie testovacích reťazcov. Dostupné na internete:
<https://opac.crzp.sk/?fn=detailBiblioForm&sid=925BFB95FAA30F229C9AC7019F0C>.

Použitie nástrojov umelej inteligencie

OpenAI (2026), ChatGPT, preklady medzi jazykmi vo viacerých častiach práce

OpenAI (2026), ChatGPT, jazyková korekcia a gramatická úprava textu v celom dokumente

OpenAI (2026), ChatGPT, pomoc pri štruktúrovaní textu a formulovaní opisných častí

Prílohy

Príloha A: Zdrojový kód algoritmov II

Príloha A: Zdrojový kód algoritmov

Súčasťou práce je elektronická príloha vo forme archívu ZIP, ktorá obsahuje kompletné implementačné a experimentálne materiály vytvorené v rámci tejto práce. Príloha slúži na reprodukovateľnosť riešenia, kontrolu implementácie a overenie výsledkov uvedených v experimentálnej časti práce.

Archív obsahuje najmä:

- kompletný zdrojový kód implementácie,
- vstupné gramatiky použité pri experimentoch,
- CSV súbory s výsledkami benchmarkov,
- vygenerované grafy použité pri vyhodnotení,
- pomocné skripty na analýzu výsledkov,
- návod na spustenie implementácie a benchmarkov,
- informácie o použitých knižniciach a požiadavkách prostredia.

Zdrojový kód obsahuje moduly na spracovanie vstupných gramatík, transformáciu gramatiky do CHNF, generovanie SAT modelu, export CNF formule do DIMACS formátu, referenčnú implementáciu algoritmu CYK a benchmarkovacie skripty. Súčasťou prílohy sú aj experimentálne výstupy, ktoré boli použité pri tvorbe tabuliek, grafov a interpretácií v kapitole 9.

Okrem elektronickej prílohy je projekt dostupný aj vo verejnom repozitári GitHub. Repozitár obsahuje zdrojový kód a súvisiace súbory potrebné na spustenie implementácie. Odkaz na repozitár:

https://github.com/juszuf69/DP

Elektronická príloha a GitHub repozitár tak umožňujú overiť postup implementácie, zopakovať experimenty a ďalej rozšíriť navrhnuté riešenie.